



CV项目

目标检测一

课程内容

⌚ 目标检测概述

⌚ R-CNN、Fast R-CNN、Faster-R-CNN、SPP-Net、R-FCN

⌚ YOLO、SSD

⌚ 目标检测应用案例

目标检测

- ⌚ 图像分类(Image Classification)的目标是识别出图中出现的物体类别是什么，其功能主要是用于判断是什么？
- ⌚ 图像定位(Object Location)的目标不仅仅需要识别出是什么物体(即分类)，同时需要预测物体的位置信息，也就是单个目标在哪里？是什么？
- ⌚ 目标检测(Object Detection)实质上上多目标的定位，即在一个图片中定位多个目标物体，包括分类和定位，也就是多个目标分别在哪里？分别属于那个类别？

目标检测

⌚ 图像分类:

⌚ VGG

⌚ GoogleNet

⌚ ResNet



0.99 – Dinosaur

0.01 – Rooster

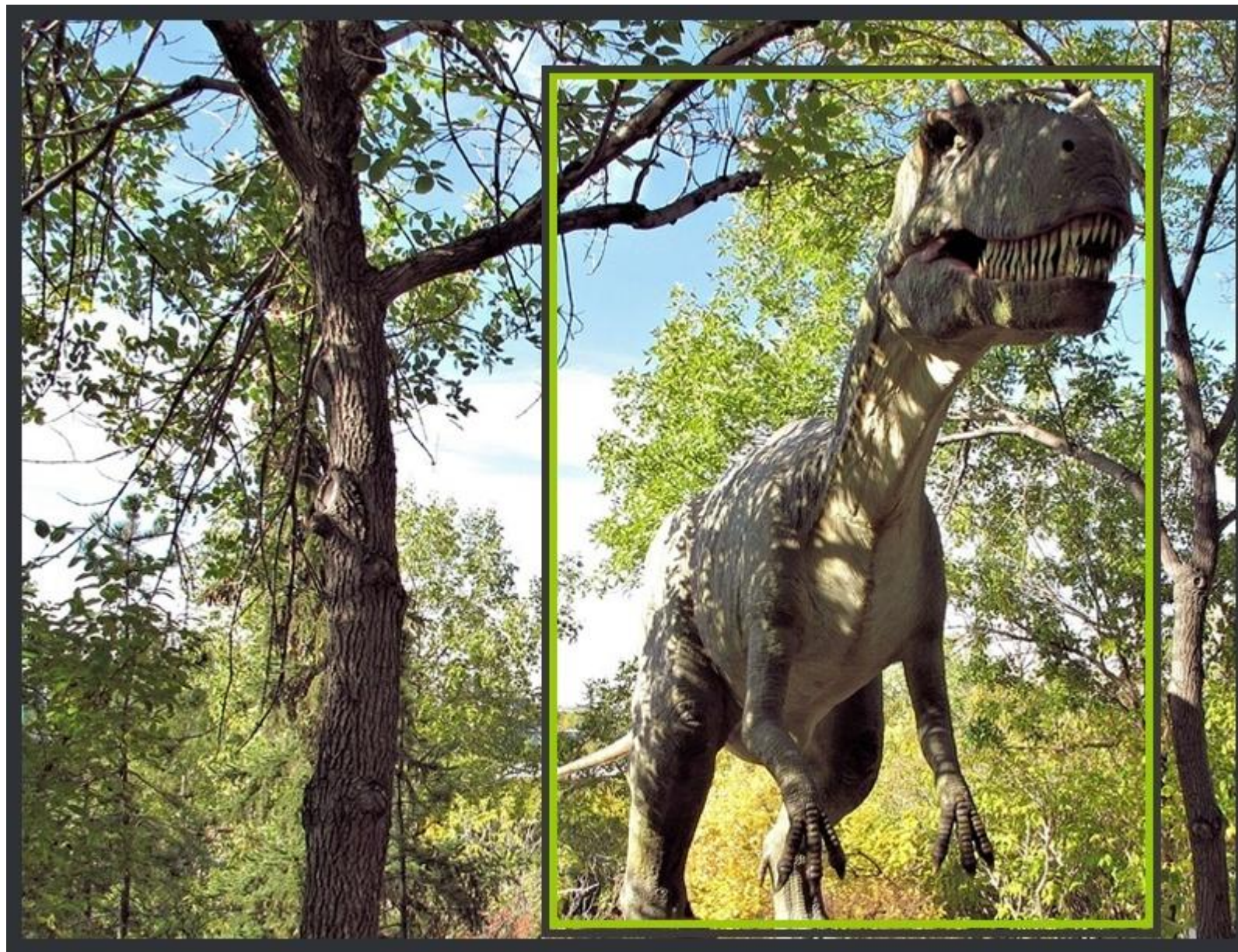
目标检测

⌚ 图像定位

⌚ RCNN

⌚ Fast RCNN

⌚ Faster RCNN



目标检测

⌚ 目标检测

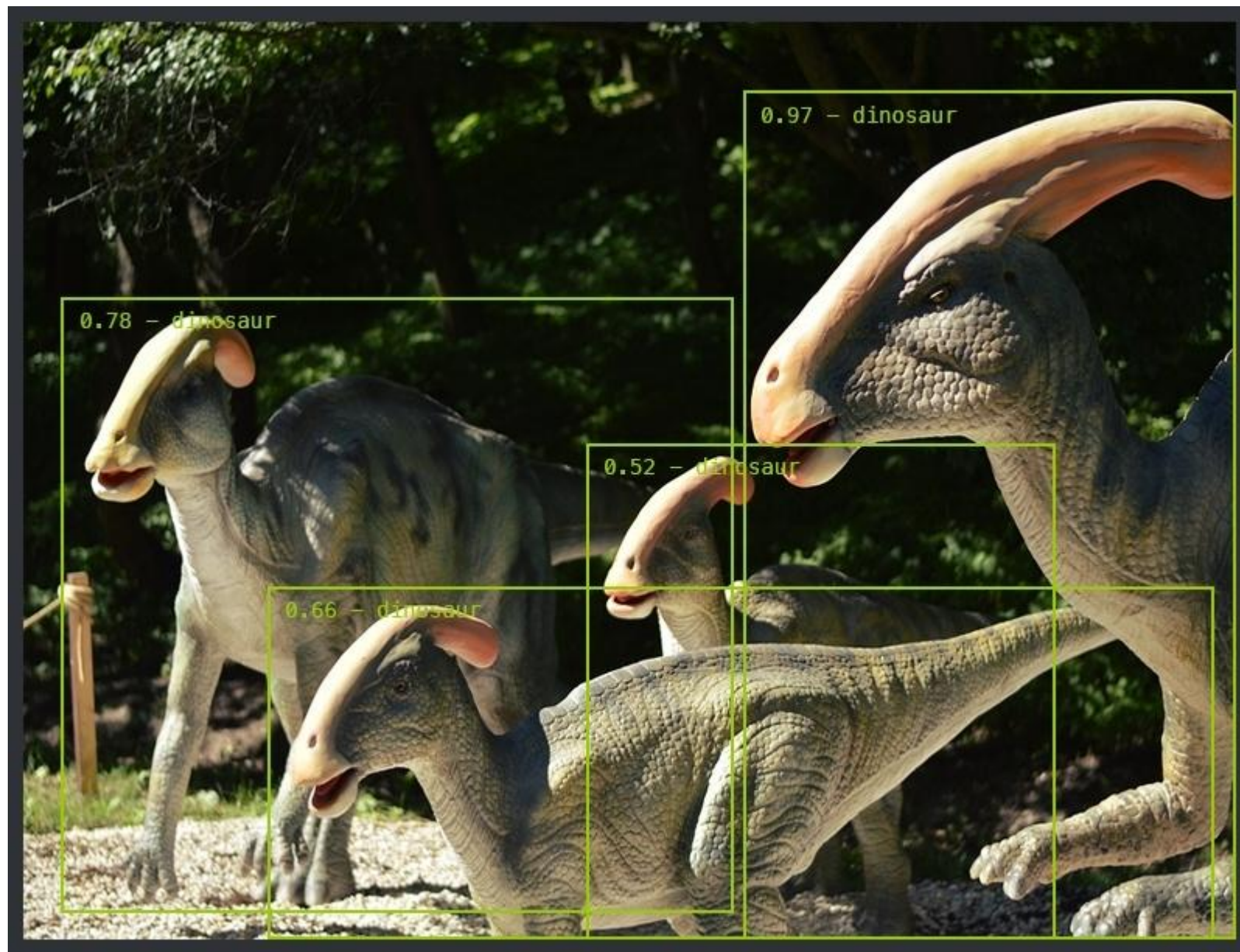
⌚ RCNN

⌚ Fast RNN

⌚ Faster RCNN

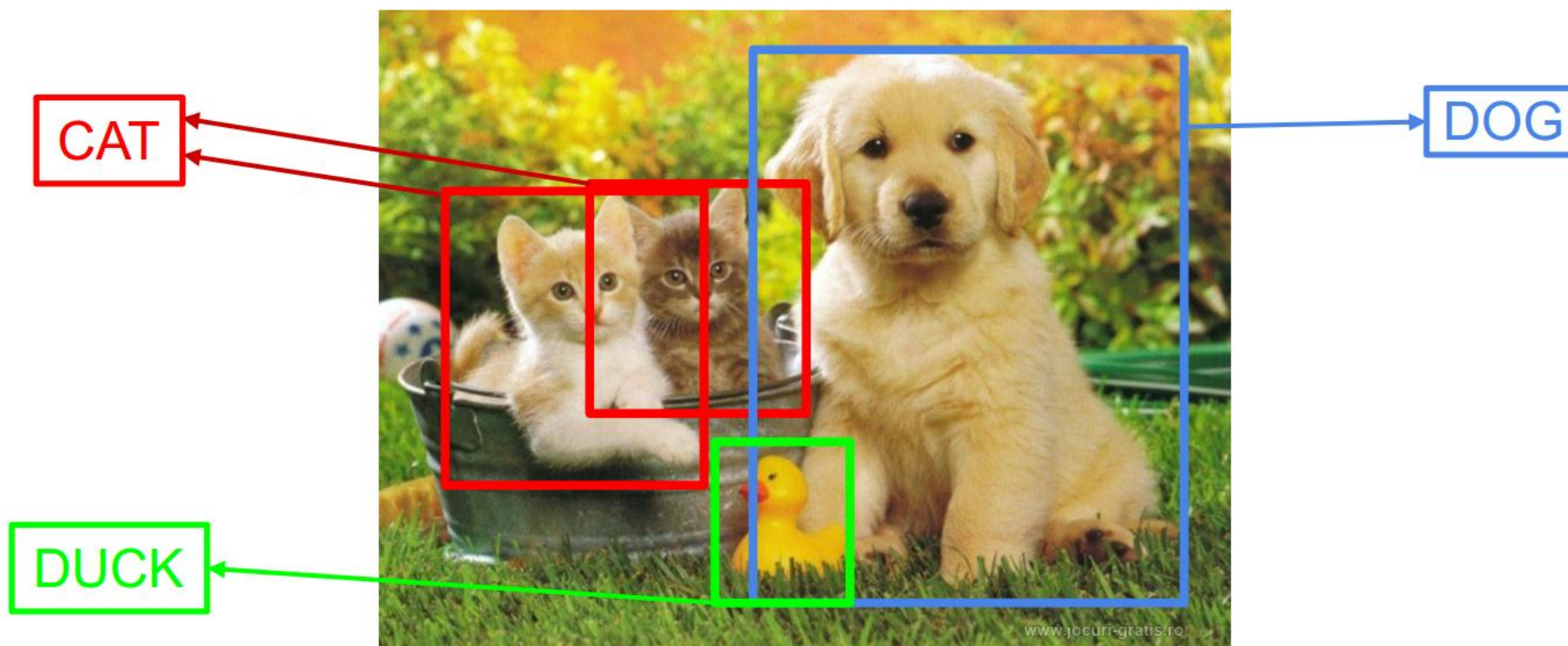
⌚ SSD

⌚ YOLO



目标检测(Object Detection)

- ⌚ 检测图片中所有物体(**模型关注的物体**)的
 - ⌚ 类别标签(**Category Label**)
 - ⌚ 位置(最小外接矩形/**Bounding Box**)



目标检测(Object Detection)

⌚ 和其它任务的区别

⌚ 单目标任务

⌚ 分类

⌚ 分类&定位

⌚ 多目标任务

⌚ 目标检测/物体检测

⌚ 图像分割/实例分割

分类
(Classification)

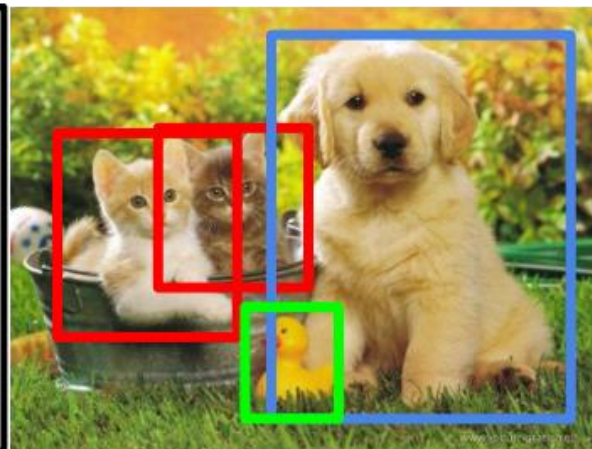
CAT



分类&定位
(Classif. & Localization)



CAT
DOG
DUCK



物体检测
(Object Detection)



实例分割
(Instance Segmentation)

目标检测(Object Detection)

⌚ ILSVRC竞赛 Imagenet Large Scale Visual Recognition Challenge

⌚ 物体检测(Object Detection)

⌚ 竞赛历史: 2013-2017

⌚ 物体类别: 200

⌚ 每个图片多组标签:

⌚ 类别+ Bounding Box(x,y,w,h)

⌚ 图像分类(Image Classification)

⌚ 场景分类(Scene Classification)

⌚ 物体定位(Object Localization)

⌚ NOTE:

⌚ <http://image-net.org/challenges/LSVRC/2017/index>

⌚ <http://www.image-net.org/challenges/LSVRC/2012/nonpub-downloads>

目标检测(Object Detection)

⌚ PASCAL VOC

⌚ 竞赛历史: 2005-2012

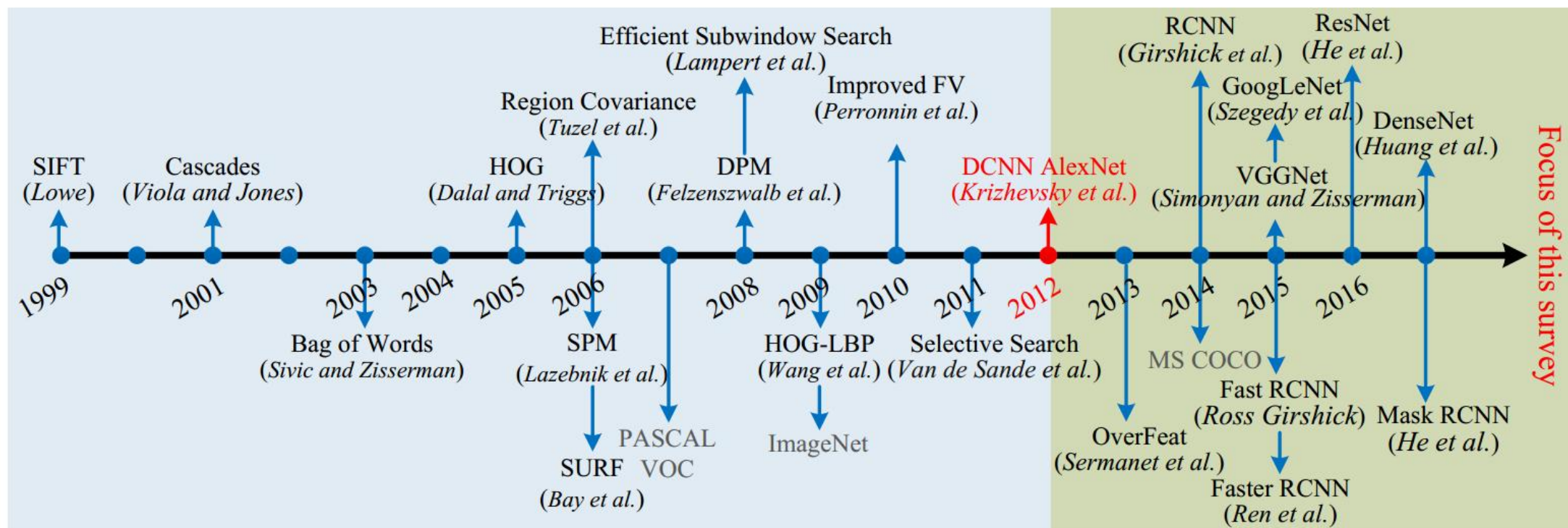
⌚ NOTE:

⌚ <http://host.robots.ox.ac.uk/pascal/VOC/>

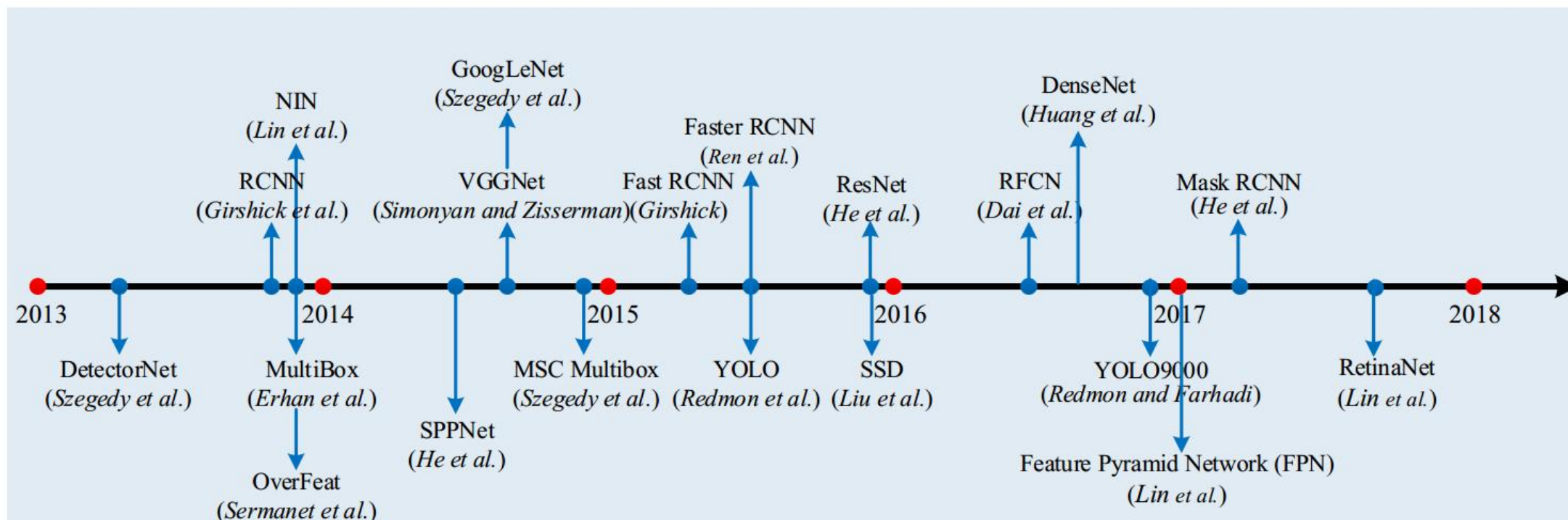
⌚ <https://pjreddie.com/projects/pascal-voc-dataset-mirror/>

		PASCAL VOC 2012	ILSVRC
	物体类别	20	200
训练集 Training	图片数量	5717	456567
	标注数量	13609	478807
验证集 Validation	图片数量	5823	20121
	标注数量	13841	55502
测试集 Testing	图片数量	10991	40152
	标注数量	-	-

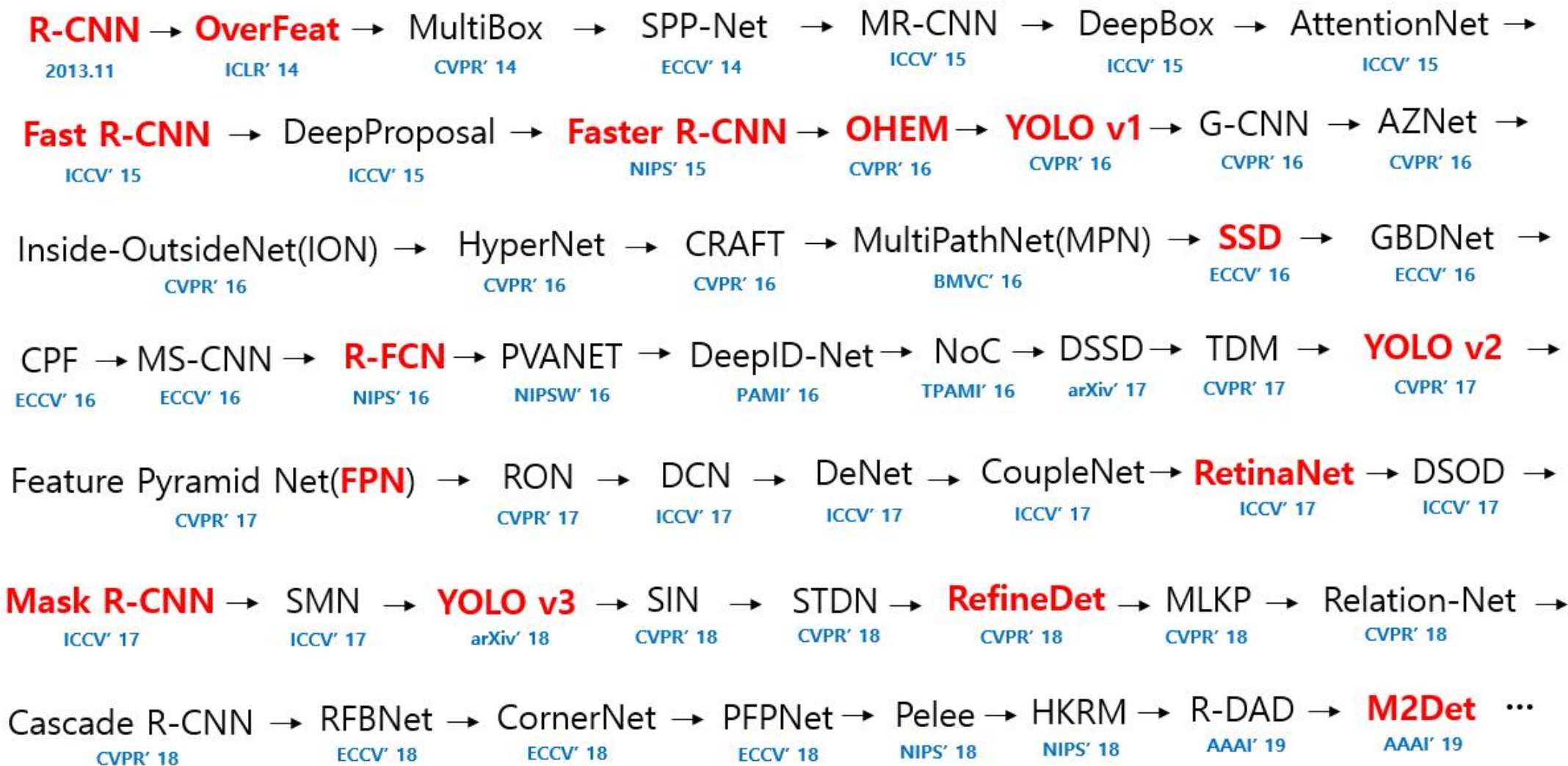
目标检测(Object Detection)



目标检测(Object Detection)



目标检测(Object Detection)



目标检测(Object Detection)

⌚ 目标检测对于人类来说并不困难，通过对图片中不同颜色模块的感知很容易定位并分类出其中目标物体，但对于计算机来说，面对的是RGB像素矩阵，很难从图像中直接得到狗和猫这样的抽象概念并定位其位置，再加上有时候多个物体和杂乱的背景混杂在一起，目标检测更加困难。在传统视觉领域，目标检测就是一个非常热门的研究方向，一些特定目标的检测，比如人脸检测和行人检测已经有非常成熟的技术了。普通的目标检测也有过很多的尝试，但是效果总是差强人意。

目标检测(Object Detection)

- ⌚ 传统的目标检测一般使用滑动窗口的框架，主要包括三个步骤：
 - ⌚ 利用不同尺寸的滑动窗口框出图中的某一部分作为候选区域；
 - ⌚ 提取候选区域相关的视觉特征。比如人脸检测常用的Harr特征；行人检测常用HOG特征等；
 - ⌚ 利用分类器进行识别，比如常用的SVM模型。
- ⌚ 在传统目标检测算法中，多尺度形变部件模型DPM的效果相对是比较优秀的；在DPM中将物体看成是多个不同组件的结合，其效果不错，但是检测速度相对比较慢。

总结：

第一步：从原始图像中获取候选的区域；

第二步：针对候选区域进行分类判断，是否属于物体以及如果属于物体的话，具体属于哪个类别 --> $n+1$ 的分类

目标检测(Object Detection)

- ⌚ 随着深度学习模型效果的优化，基于深度学习的目标检测算法盖过了传统计算机视觉中的目标检测算法(DPM)，从而导致很多之前研究传统目标检测算法的人员转向深度学习，并且现在工业界中应用最多的目标检测方式以深度学习为主。
- ⌚ 其中R-CNN是奠定深度学习方向发展目标检测的基础算法，是第一个真正可以工业级应用的目标检测解决方案，是结合区域提名(Region Proposal)和卷积神经网络(CNN)的一种目标检测算法。

目标检测(Object Detection)

⌚ 目前主流的目标检测算法主要是基于深度学习模型，主要可以分为两大类：**two-stage检测算法**和**one-stage检测算法**。

⌚ two-stage检测算法将检测问题划分为两个阶段，首先产生候选区域(region proposals, 基于图像特征来产生-->比较耗时的), 然后对候选区域进行分类(一般需要进行位置精修), 这类算法实现主要有: R-CNN、SPPNET、Fast R-CNN、Faster R-CNN等。

⌚ one-stage检测算法是一种端到端的检测算法，直接进行区域定位与分类，这类算法实现主要有: SSD、YOLO、FPN等。

- Anchor Box检测算法: 基于规则产生的Anchor Box(候选框), 然后对Anchor Box直接做分类和回归修正, 代表算法: SSD、YOLOv5、YOLOv8等。

- Anchor Free检测算法: 不产生Anchor Box候选框, 直接对图像进行前向过程, 直接输出边框位置以及坐标, 代表算法: YOLOv1、CornerNet等

选择性搜索 (Selective Search)



(a)



(b)



(c)



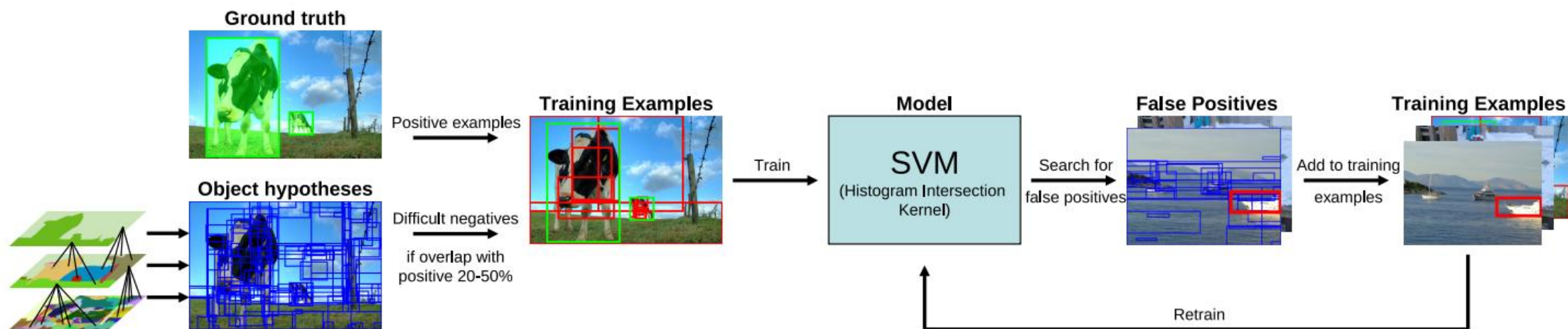
(d)

选择性搜索 (Selective Search)

- ⌚ 一种目标用于目标检测的暴力方法就是从左往右，从上往下滑动窗口，利用分类识别目标，为了识别不同距离出检测不同的目标类型，可以使用不同大小、高宽比的窗口进行滑动，然后将窗口内的图片进行特征提取后送入分类器，进行分类判断即可完成目标检测的操作。很显然这种方法复杂度太高，会产生太多的冗余候选区域，实际应用中是不太可行的。
- ⌚ 在穷举暴力法的基础上，进行一些剪枝操作，只选择固定大小和高宽比的窗口，这种方式在某些特定应用场景中是有效的，但是对于普通的目标检测而言，计算复杂度还是比较高的。

选择性搜索 (Selective Search)

- ⌚ 为了解决刚刚的区域窗口获取过程中的问题，选择性搜索算法被提出来了，其问题的解决核心在于如何有效的去除冗余候选区域；在选择性搜索算法中，利用冗余候选区域大多是发生重叠的这个特征，**自底向上的合并相邻的相似区域，从而减少冗余。**



选择性搜索 (Selective Search)

- ⌚ 1. 将图像划分为细粒度的小块；使用算法：felzenszwalb；
- ⌚ 2. 使用贪婪策略，计算两个相邻的区域的相似度，然后合并最相似的两块区域，直到最终只剩下一个完整的图像(S集合中没有相似度就结束区域的合并)；
- ⌚ 3. 获取相似度最相似的形成最终的候选区域。

Algorithm 1: Hierarchical Grouping Algorithm

Input: (colour) image

Output: Set of object location hypotheses L

Obtain initial regions $R = \{r_1, \dots, r_n\}$ using [13]

Initialise similarity set $S = \emptyset$

foreach *Neighbouring region pair* (r_i, r_j) **do**

 Calculate similarity $s(r_i, r_j)$

$S = S \cup s(r_i, r_j)$

while $S \neq \emptyset$ **do**

 Get highest similarity $s(r_i, r_j) = \max(S)$

 Merge corresponding regions $r_t = r_i \cup r_j$

 Remove similarities regarding r_i : $S = S \setminus s(r_i, r_*)$

 Remove similarities regarding r_j : $S = S \setminus s(r_*, r_j)$

 Calculate similarity set S_t between r_t and its neighbours

$S = S \cup S_t$

$R = R \cup r_t$

Extract object location boxes L from all regions in R

选择性搜索 (Selective Search)

⌚ 区域合并采用了多样性的策略，如果仅仅考虑单一策略，容易导致合并不相似的区域，比如仅考虑纹理的时候，不同颜色的区域很可能会误合并，所以在选择性搜索中采用三种多样性策略来增加候选区域的可能性：

- ⌚ 多种颜色空间：考虑RGB、灰度、HSV等
- ⌚ 多种相似度度量标准，既考虑颜色相似度，又考虑纹理、大小、重叠情况等。
- ⌚ 通过更改阈值初始化原始区域，阈值越大，分割区域的块越少。

选择性搜索 (Selective Search)

- 通过色彩空间变换，将原始色彩空间转换到多达八中的色彩空间。作者采用了8中不同的颜色方式，主要是为了考虑场景以及光照条件等。这个策略主要应用于中图像分割算法中原始区域的生成(两个像素点的相似度计算时，计算不同颜色空间下的两点距离)。主要使用的颜色空间有：（1）RGB，（2）灰度I，（3）Lab，（4）rgl（归一化的rg通道加上灰度），（5）HSV，（6）rgb（归一化的RGB），（7）C，（8）H（HSV的H通道）

选择性搜索 (Selective Search)

<i>colour channels</i>	R	G	B	I	V	L	a	b	S	r	g	C	H
Light Intensity	-	-	-	-	-	-	+/-	+/-	+	+	+	+	+
Shadows/shading	-	-	-	-	-	-	+/-	+/-	+	+	+	+	+
Highlights	-	-	-	-	-	-	-	-	-	-	-	+/-	+

<i>colour spaces</i>	RGB	I	Lab	rgI	HSV	rgb	C	H
Light Intensity	-	-	+/-	2/3	2/3	+	+	+
Shadows/shading	-	-	+/-	2/3	2/3	+	+	+
Highlights	-	-	-	-	1/3	-	+/-	+

选择性搜索 (Selective Search)

⌚ 颜色相似度:

- ⌚ 使用L1-norm归一化获取图像每个颜色通道的25bins直方图，这样每个区域可以得到一个75维的向量(以RGB为例)，区域之间的相似度通过下列公式计算；值越大表示越相似，值越小表示越不相似。

$$C_i = \{c_i^1, \dots, c_i^n\} \quad s_{colour}(r_i, r_j) = \sum_{k=1}^n \min(c_i^k, c_j^k).$$

$$C_t = \frac{\text{size}(r_i) \times C_i + \text{size}(r_j) \times C_j}{\text{size}(r_i) + \text{size}(r_j)}.$$

选择性搜索 (Selective Search)

⌚ 纹理相似度:

- ⌚ 采用SIFT特征(尺度不变特征变换), 具体特征求解做法为: 对每个颜色通道的8个不同方向计算方差 $\sigma=1$ 的高斯微分; 使用L1-norm归一化获取图像每个颜色通道每个方向的10个bins的直方图, 这样可以得到一个240维的向量($3*8*10$), 其相似度计算方式以及合并后的纹理特征计算方式同颜色相似度计算过程。

$$T_i = \{t_i^1, \dots, t_i^n\} \quad s_{texture}(r_i, r_j) = \sum_{k=1}^n \min(t_i^k, t_j^k)$$

选择性搜索 (Selective Search)

- ⌚ 如果仅仅基于颜色和纹理特征来进行合并的话，会导致合并之后的区域不断吞并周围的区域，后果就是多尺度仅应用与某个局部，而不是全局的多尺度，因此，需要给小的区域更多的权重，这样可以保证图像每个位置都是多尺度的合并，也就是优先合并小的区域。

$$s_{size}(r_i, r_j) = 1 - \frac{\text{size}(r_i) + \text{size}(r_j)}{\text{size}(im)}$$

选择性搜索 (Selective Search)

- ⌚ 如果区域 r_i 包含在 r_j 内，那么应该被合并；如果 r_i 和 r_j 是很难相接的，那么就不应该合并到一起，所以可以通过定义区域之间的距离来衡量两个区域是否应该进行合并，其主要指标采用合并后区域的Bounding Box(最小矩形面积)越小，表示相似度越高。

$$fill(r_i, r_j) = 1 - \frac{\text{size}(BB_{ij}) - \text{size}(r_i) - \text{size}(r_j)}{\text{size}(im)}$$

选择性搜索 (Selective Search)

⌚ 合并上述四种相似度，即可得到合并区域的相似度的值：

$$s(r_i, r_j) = a_1 s_{colour}(r_i, r_j) + a_2 s_{texture}(r_i, r_j) + a_3 s_{size}(r_i, r_j) + a_4 s_{fill}(r_i, r_j),$$

$$a_i \in \{0, 1\}$$

选择性搜索 (Selective Search)

- ⌚ 给予最先合并的区域一个比较大的权重，给予最后一个合并的区域一个最小的权重系数，如果存在一样合并的方式的时候，乘以一个随机数来给定权重，如果相同区域被多次进行合并重叠，那么对应的权重进行权重叠加操作，最终根据权重系数进行排序，从而得到最优的候选区域。
- ⌚ 通过这种方式合并候选区域后，将每个候选区域对应的图像提取图像特征向量后，将特征向量输入SVM分类器即可进行目标检测操作。

目标检测_深度学习

⌚ 图像识别(Classification)

⌚ 输入：图片

⌚ 输出：物体的类别

⌚ 评估方式：准确率



→ CAT

⌚ 图像定位(Localization)

⌚ 输入：图片

⌚ 输出：方框在图片中的位置(x,y,w,h)

⌚ 评估方式：检测评价函数 Intersection-Over-Union(IOUS)

(x, y)



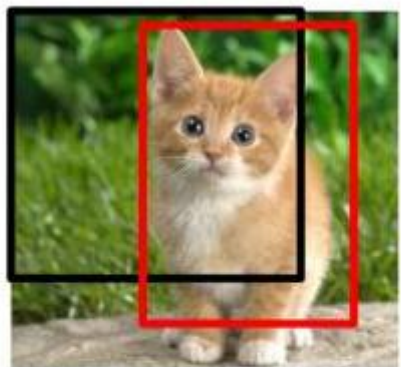
h,height,
高度

w,width,宽度

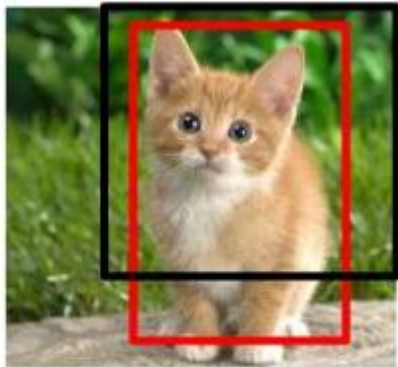
→ (x, y, w, h)

目标检测_深度学习_思路一

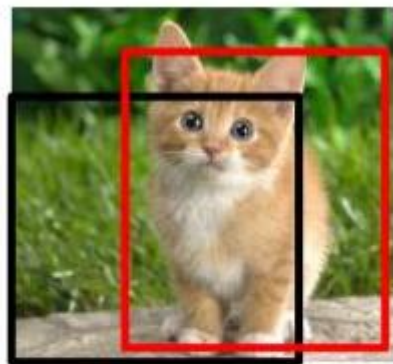
- ⌚ 采用候选框 + classification的思路设计网络;
- ⌚ 取大小不同的框, 让框出现在不同位置, 得出这个框的判定得分, 获取得分最高的那个框。NOTE: 输入模型的就是框的区域。



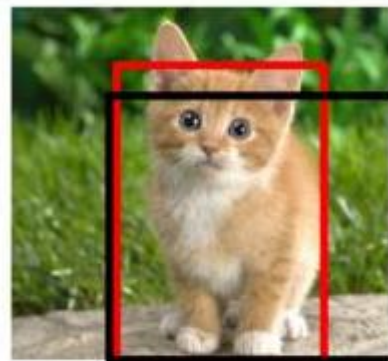
Larger image:
3 x 257 x 257



Larger image:
3 x 257 x 257



Larger image:
3 x 257 x 257



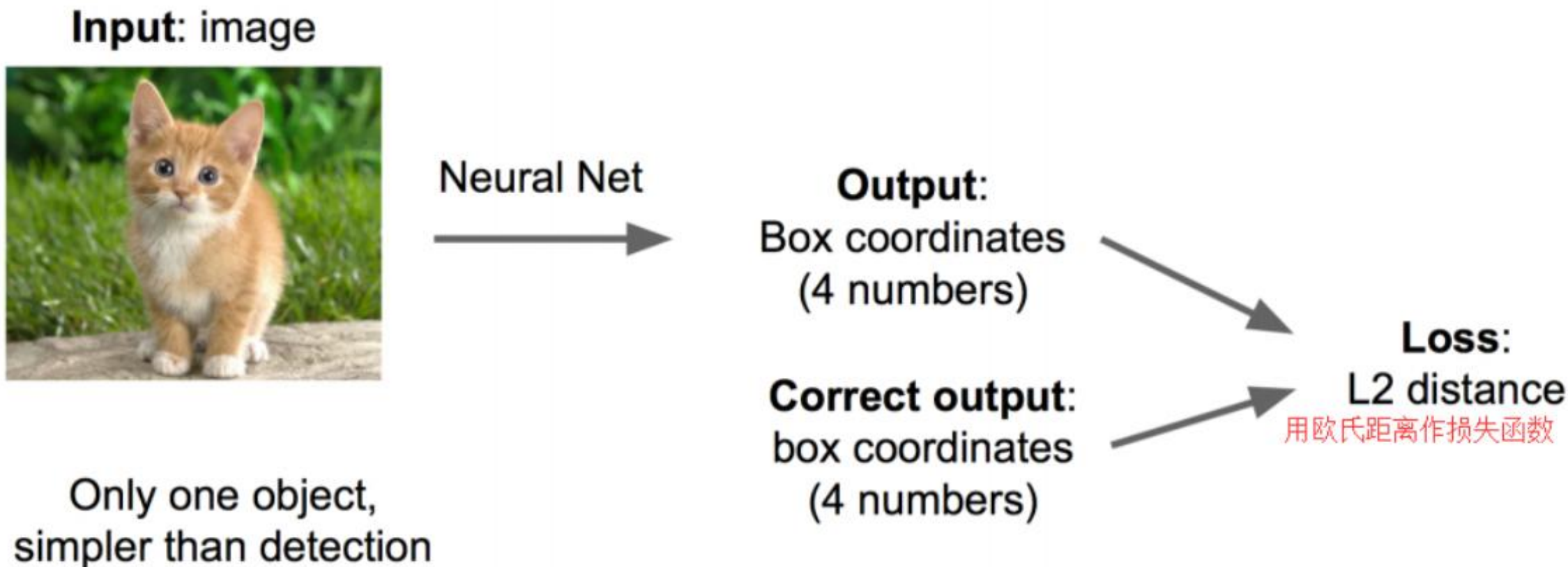
Larger image:
3 x 257 x 257

0.5	0.75
0.6	0.8

Classification scores:
P(cat)

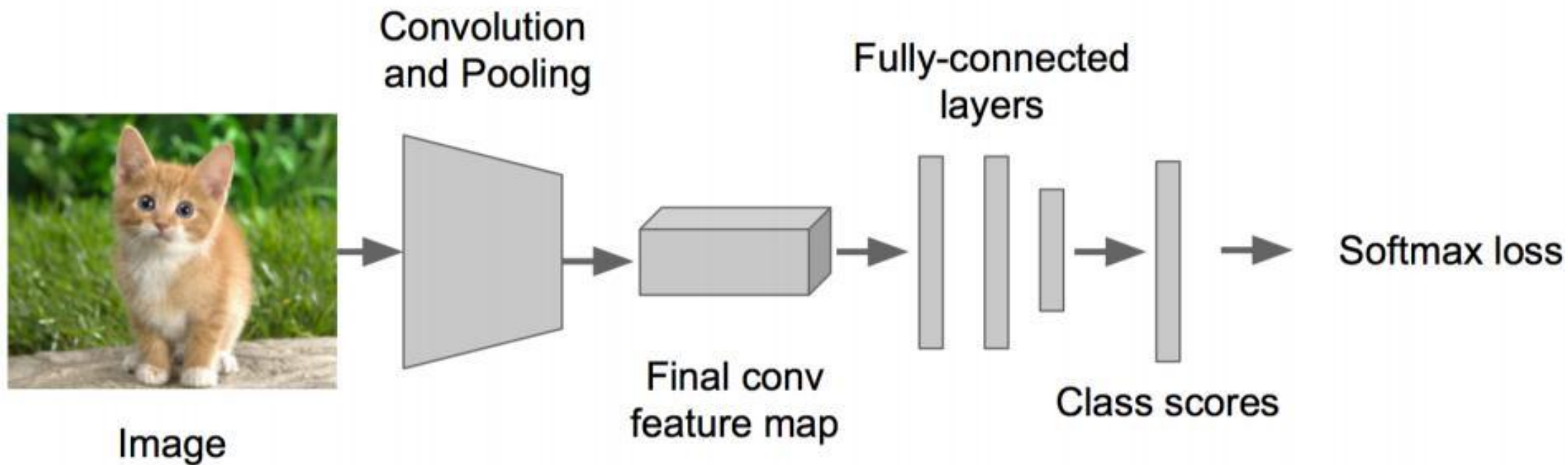
目标检测_深度学习_思路二

- ⌚ 将定位问题看成回归问题，也即是需要预测出(x,y,w,h)四个数值，从而得到方框的位置。



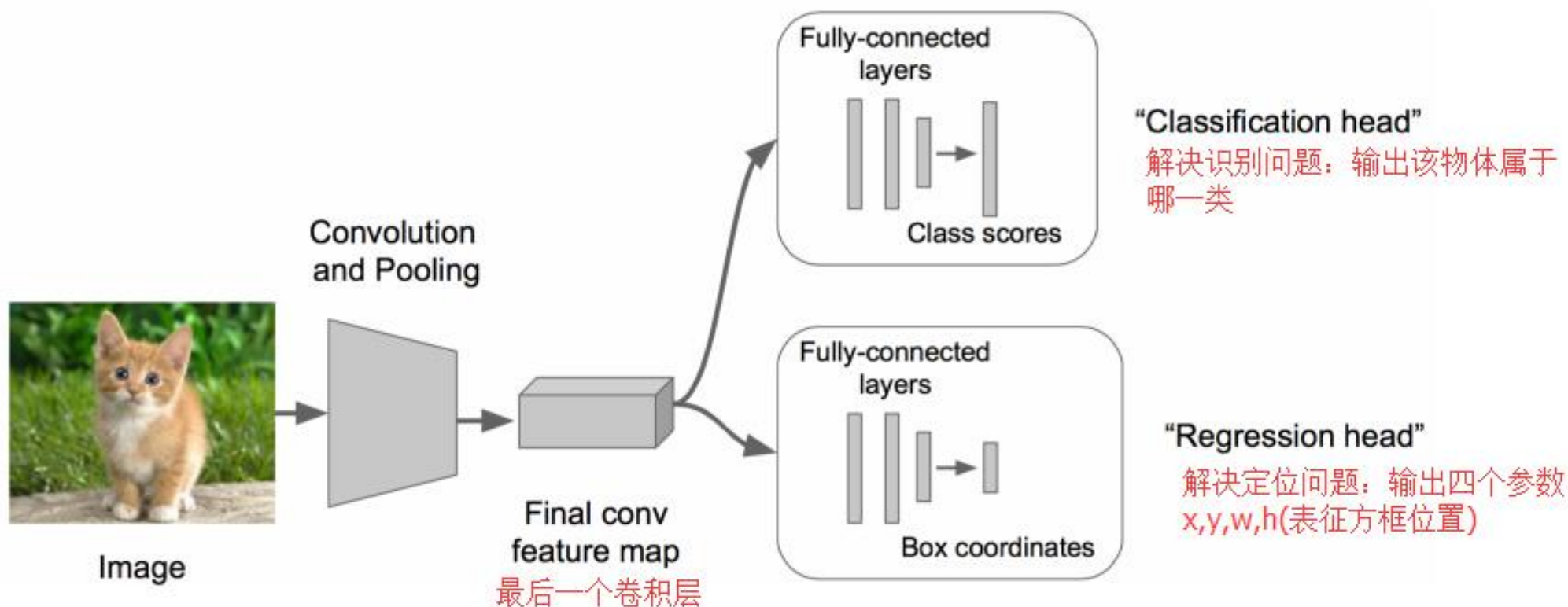
目标检测_深度学习_思路二

- ⌚ 步骤一：搭建一个图像分类的神经网络，可以在AlexNet、VGG、GoogleNet基础上做一个fine-tuning产生一个新的神经网络。



目标检测_深度学习_思路二

⌚ 步骤二：在上述神经网络的尾部展开(也就是前面CNN特征提取部分保持不变，而针对于分类和回归分别构建不同的分支)，变成 classification+regression 的模式。



目标检测_深度学习_思路二

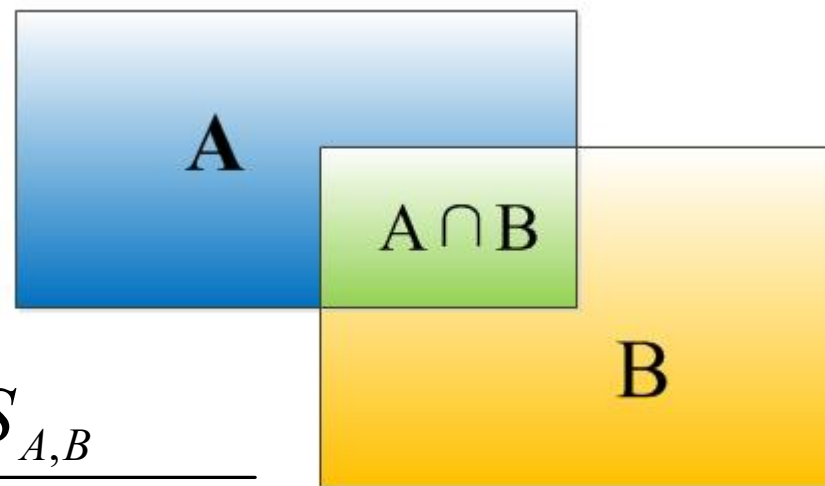
- ⌚ 步骤三：对于Regression部分，使用欧式距离作为损失函数(可改)，然后做一个fine-tuning来进行网络的训练。
- ⌚ 步骤四：在预测的时候，分别使用两个分支完成不同的功能。
- ⌚ NOTE: 这里进行了两次fine-tuning操作。

目标检测_深度学习

- ⌚ 物体检测(Object Detection): 相比于分类和定位任务, 多物体的识别+定位会更加的复杂, 结合刚刚说的分类、定位任务, 有如下比较好的方法:
 - ⌚ 找出可能包含物体的框(候选框), 然后这些框之间是可以互相重叠、互相包含的, 然后对这些框进行分类+回归的操作, 这种方式可以避免暴力枚举所有的框。
 - ⌚ 常见获取候选框的方式有: EdgeBoxes、Selective Search等。

目标检测_深度学习_IOU

🕒 物体检测需要定位出物体的bounding box，就像右边的图片一样，我们不仅要定位出车辆的bounding box 我们还要识别出bounding box里面的物体就是车辆。对于bounding box的定位精度，有一个很重要的概念，因为我们算法不可能百分百跟人工标注的数据完全匹配，因此就存在一个定位精度评价公式：IOU(Intersection over Union)。**IOU定义了两个bounding box的重叠度，也就是两个bounding box重叠区域的面积占总面积的比值。**



$$IOU = \frac{A \cap B}{A \cup B} = \frac{S_{A,B}}{S_A + S_B - S_{A,B}}$$

目标检测_mAP

⌚ TP: 实际为正例, 预测也为正例;

⌚ FP: 实际为负例, 预测为正例;

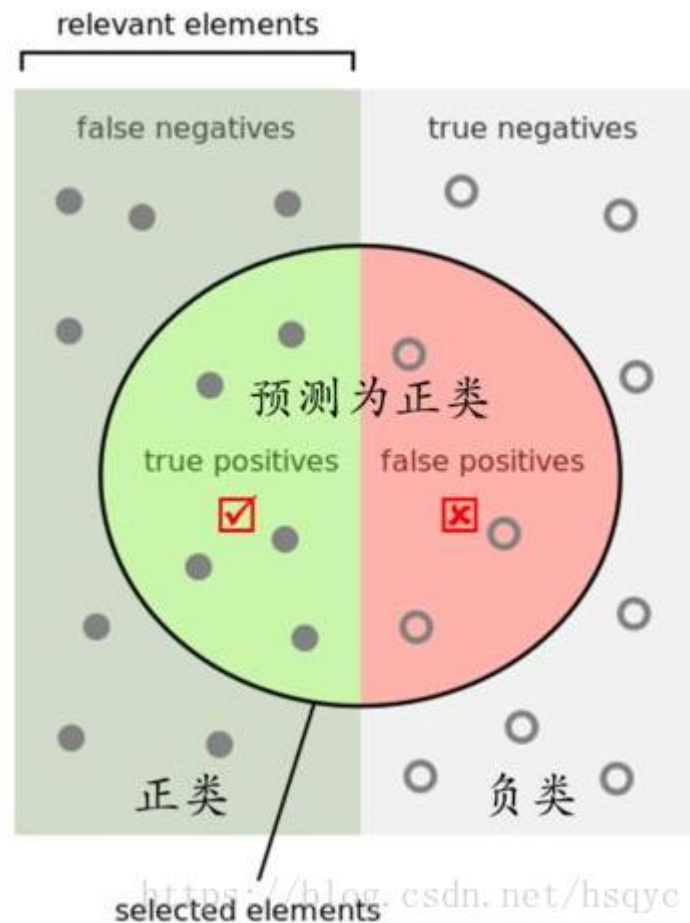
⌚ TN: 实际为负例, 预测也为负例;

⌚ FN: 实际为正例, 预测为负例;

⌚ 精度:
$$precision = \frac{TP}{TP + FP}$$

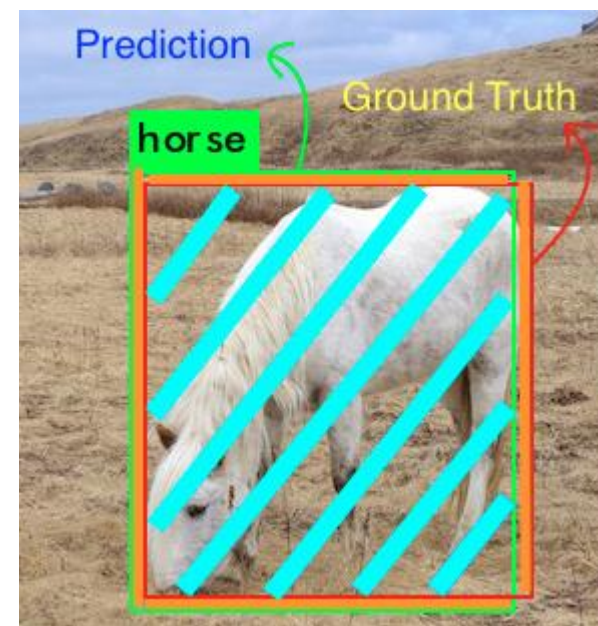
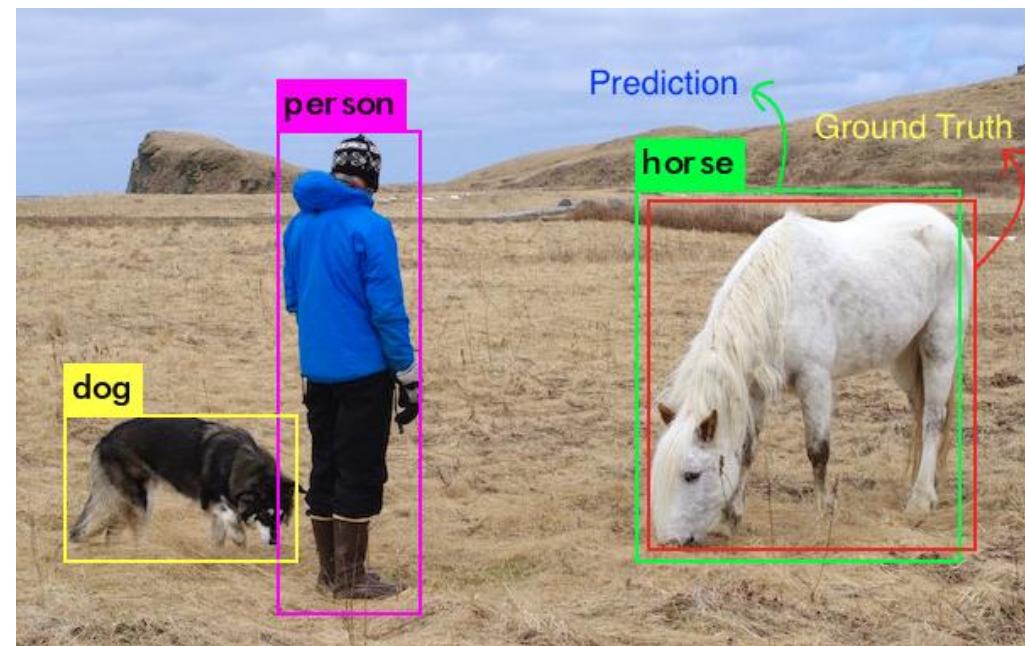
⌚ 召回率:

$$recall = \frac{TP}{TP + FN}$$



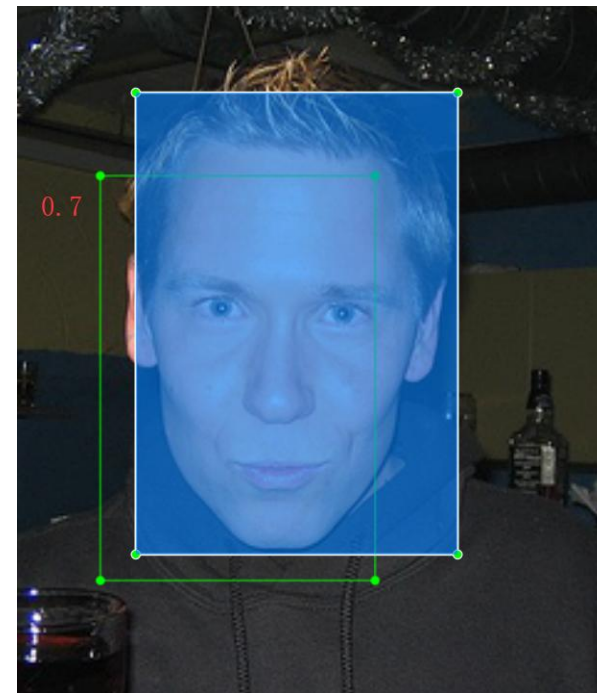
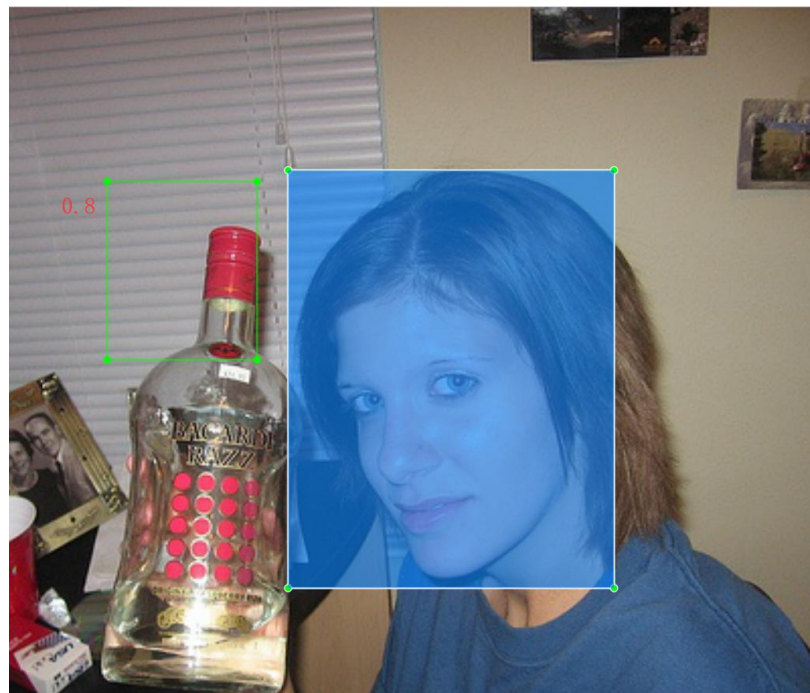
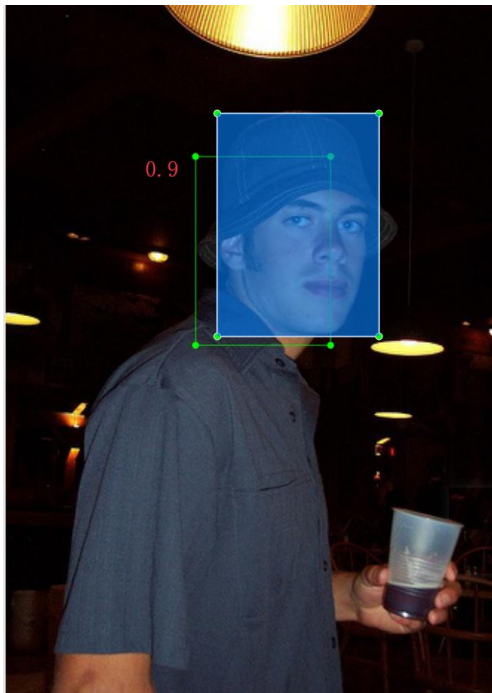
目标检测_mAP

- ⌘ 对于精度(Precision)和召回率(Recall)的计算和机器学习一样，都是需要判断出TP(真正例)、FP(假正例)、TN(真负例)、FN(假负例)
- ⌘ 为了得到TP和FP，需要用到IOU，以判断检测结果正确还是错误，一般以0.5作为IOU的阈值，如果 $IOU > 0.5$ ，认为检测结果为TP，否则为FP。在COCO数据集中，建议使用不同的阈值分别计算，在PASCAL VOC中建议直接0.5阈值。
- ⌘ 为了计算Recall，可以直接将漏检的物体数目当做FN。

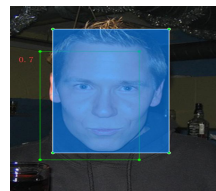
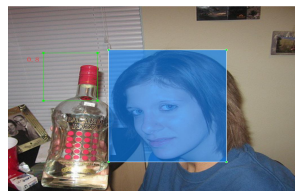


目标检测_mAP

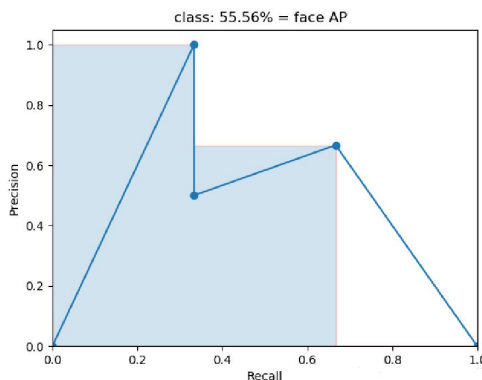
⌚ mAP(mean Average Precision)在目标检测中定义为平均精度评估指标值;



目标检测_mAP



- 根据predict的置信度阈值进行排序，这里刚好就是predict1(0.9)、predict2(0.8)、predict3(0.7)的顺序；
- 计算在不同阈值的情况下，Precision和Recall的值。
 - 阈值0.9，无视所有小于0.9的predict，那么此时TP=1, FP=0, precision=1，所有标签数目为3，那么recall=1/3；
 - 阈值0.8，无视所有小于0.8的predict，那么此时TP=1, FP=1, precision=1/2，所有标签数目为3，那么recall=1/3
 - 阈值0.7，无视所有小于0.7的predict，那么此时TP=2, FP=1, precision=2/3，所有标签数目为3，那么recall=2/3
- 绘制PR曲线，计算AP值。
 - 在每个“峰值点”往左画一条直线，和上一个“峰值点”的垂直线相交，这样和坐标轴框出来的面积就是AP值。



$$AP = 1 * \frac{1}{3} + \frac{2}{3} * \frac{1}{3} \approx 0.55$$

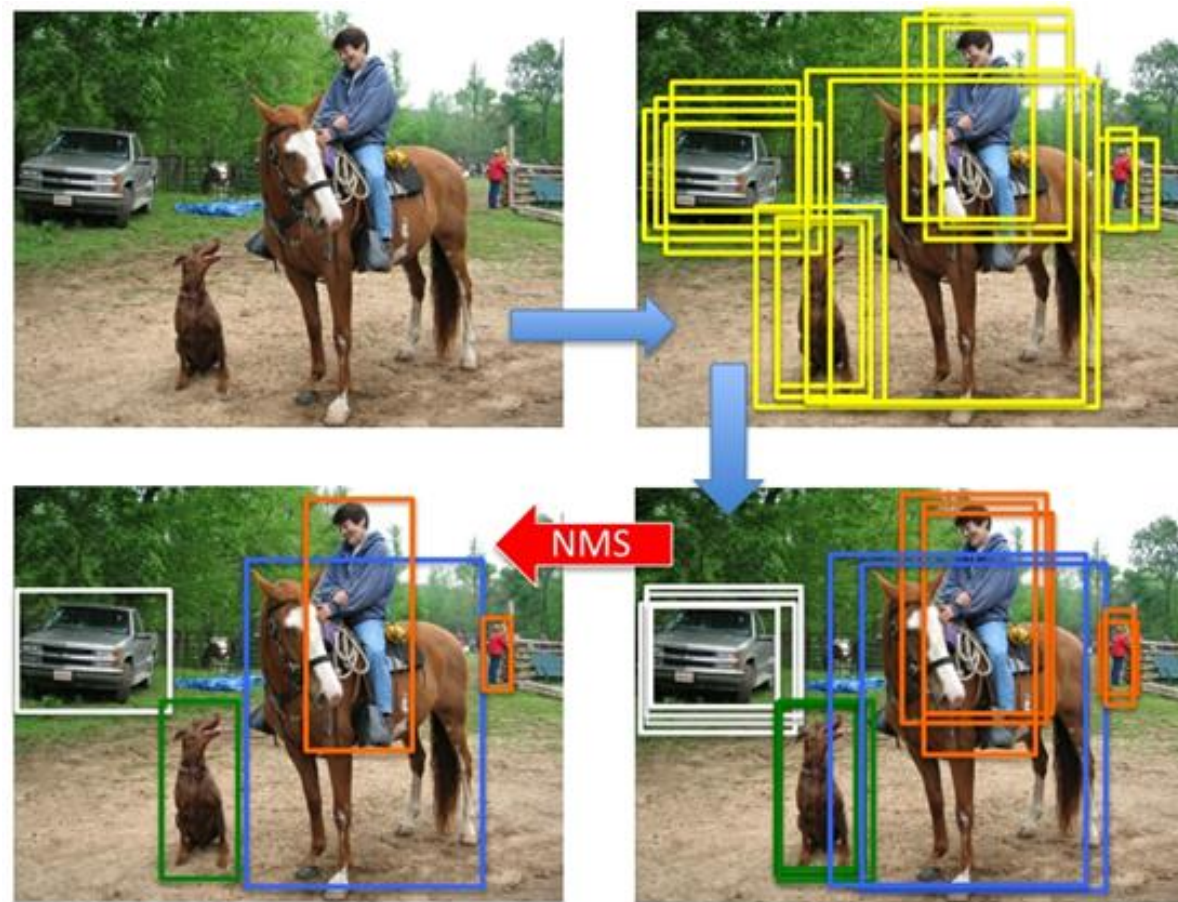
目标检测_mAP

⌚ AP值是衡量一个类别的检测好坏情况的，mAP就是对多个类别的检测好坏情况进行评估，简单来讲就是将所有类别的AP值进行平均取值即可。比如现在有两个类别，类别A的AP值为0.5，类别B的AP值为0.2，那么
 $mAP = (0.5 + 0.2) / 2 = 0.35$

$$mAP = \frac{\sum AP}{N(classes)}$$

RCNN_NMS

- ⌚ 非极大值抑制(Non-Maximum Suppression, NMS), 顾名思义就是抑制不是极大值的元素, 可以理解为局部最大搜索。是目标检测中常用的一种精修 bounding box 的方式。
- ⌚ 产生 proposal 后使用分类网络给出每个框的每类置信度, 使用回归网络修正位置, 最终应用 NMS 返回最终位置。

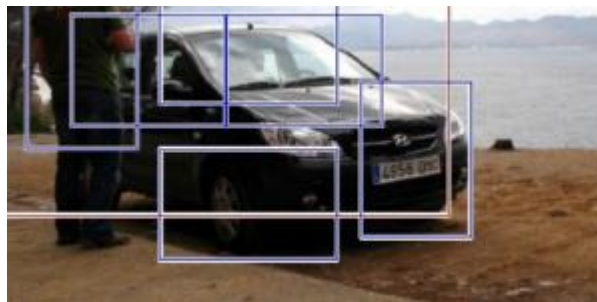


RCNN_NMS

- ✂ 对于CNN网络提取出来的Bounding Box列表B以及其对应的置信度S，采用下列执行过程来进行NMS操作：
 - ✂ 从S和B中选择最大置信度score(至少大于0)以及对应Bounding Box框M，将其从B/S集合中移除，并将M加入到最终检测结果集合D中。
 - ✂ 计算B中剩下所有的Bounding Box和M之间的IOU值，如果IOU值大于阈值t，那么表示这两个框比较相似，则将其从集合B中移除(将置信度重置为0)，否则不进行任何操作。
 - ✂ 重复上述两个操作，直到集合B为空或者S中的最大置信度为0。
 - ✂ NOTE: 上述操作针对每个类别进行操作。

RCNN_NMS

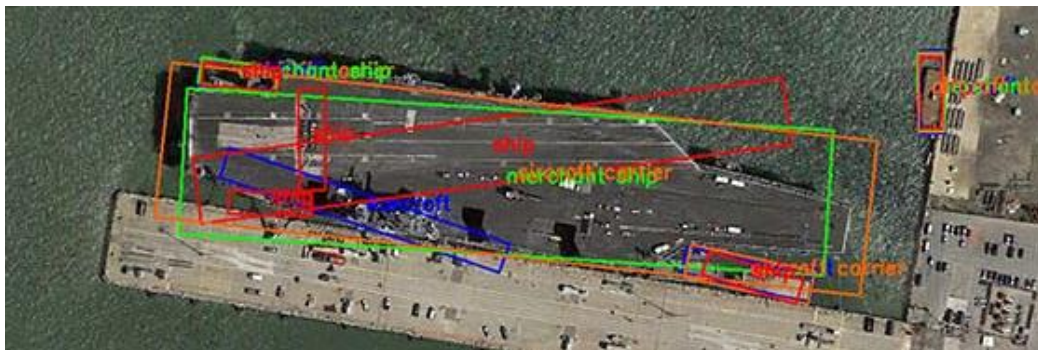
- ✂ 假设有6个矩形框，根据分类器的类别分类概率做排序，假设从小到大属于车辆的概率分别为A、B、C、D、E、F；
 - ✂ 从最大概率矩形框F开始，分别计算A~E与F的重叠度IOU是否大于某个设定的阈值；
 - ✂ 假设B、D与F的重叠度超过阈值，那么就删除B、D；并标注F矩形框保留下来；
 - ✂ 从剩下的A、C、E中，选择概率最大的E，然后判断E与A、C的重叠度，重叠度大于阈值，则删除，并标记E为保留下来的第二个矩形框；
 - ✂ 重复上述操作，直到所有被保留下来的矩形框被找出来。



RCNN NMS

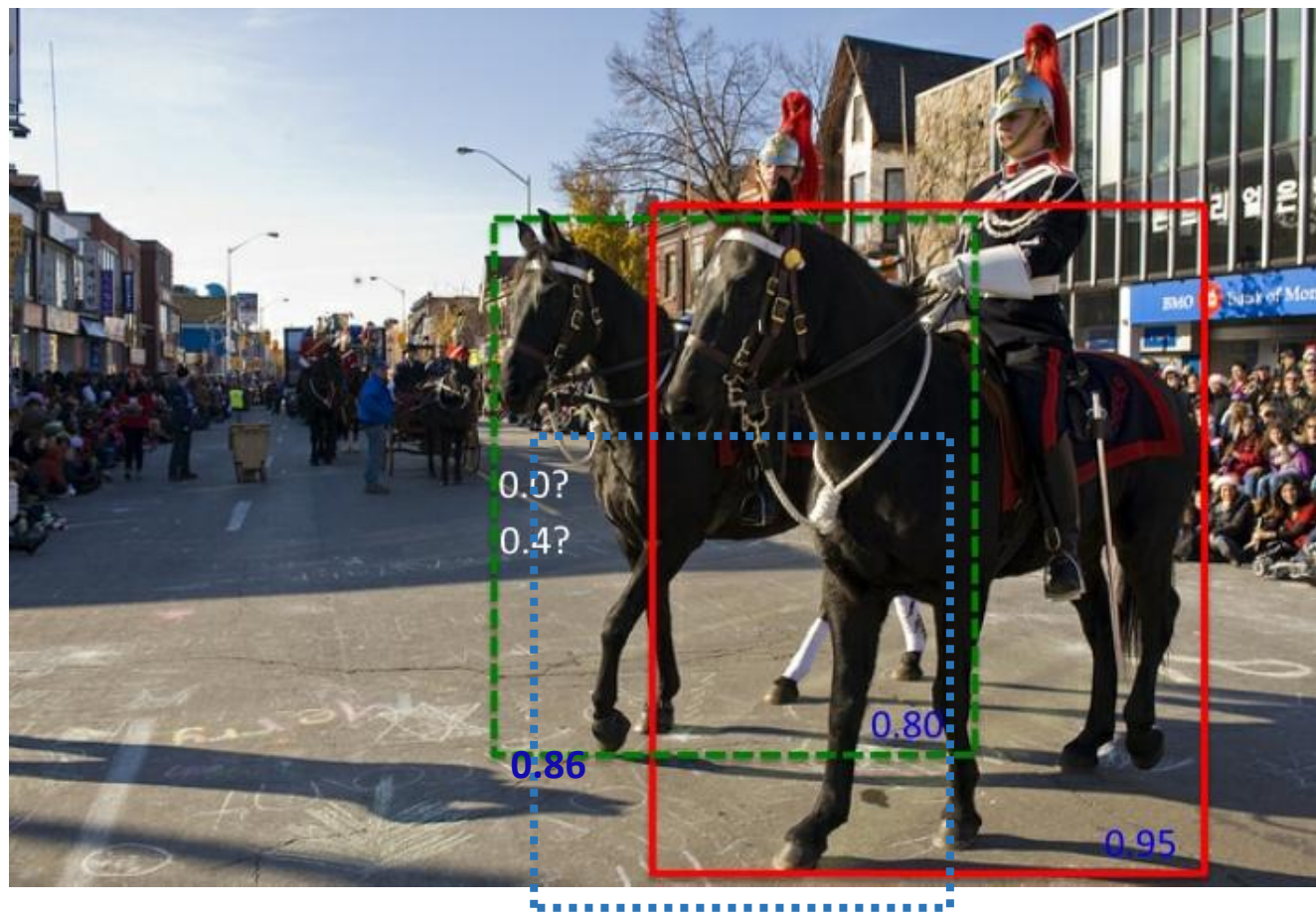
🕒 对于多类别的检测任务，如果对每个类别分别进行NMS，那么当检测结果中包含两个被分到不同类别的目标且IOU较大的时候，会导致结果合并，出现不可接受的结果。针对这个问题主要解决方案为：

Soft-NMS



RCNN_NMS

- ⌚ 在增加NMS损失的方法中，其主要问题是对于两个ground truth (真实)的 bounding box，如果其IOU很高的时候，NMS会将其中低置信度的框去除 (置信度重置为0)。



RCNN_NMS

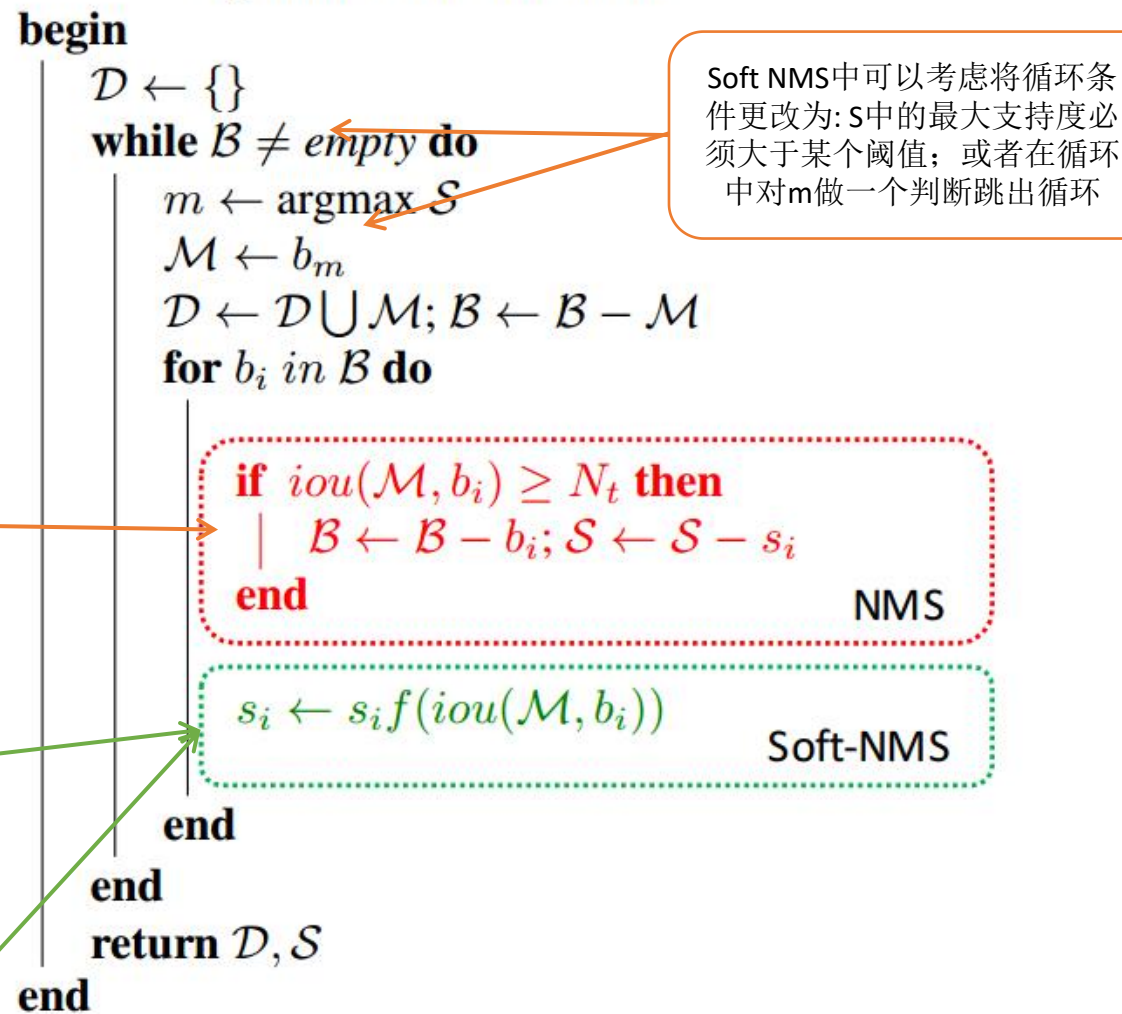
⌚ 在Soft-NMS算法中，其主要修改方式为：在重叠区域比较大的两个框中对于置信度较低值的框，不进行删除操作，而是进行置信度更新操作。

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ 0, & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases},$$

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ s_i(1 - \text{iou}(\mathcal{M}, b_i)), & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases}$$

$$s_i = s_i e^{-\frac{\text{iou}(\mathcal{M}, b_i)^2}{\sigma}}, \forall b_i \notin \mathcal{D}$$

Input : $\mathcal{B} = \{b_1, \dots, b_N\}$, $\mathcal{S} = \{s_1, \dots, s_N\}$, N_t
 $\mathcal{B}$ is the list of initial detection boxes
 $\mathcal{S}$ contains corresponding detection scores
 N_t is the NMS threshold



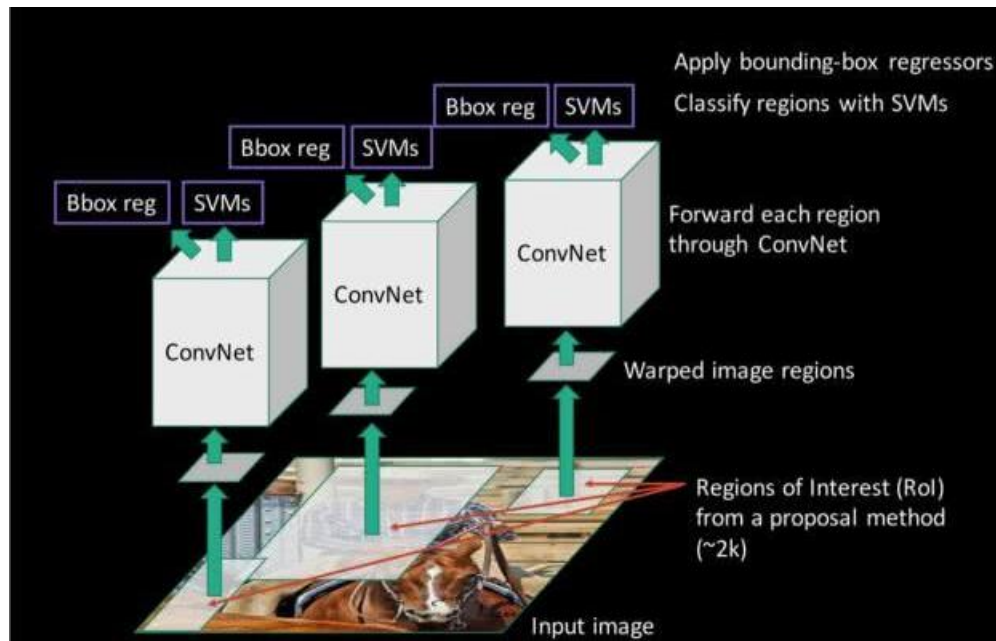
RCNN

- 🕒 CVPR2014经典paper: 《Rich feature hierarchies for accurate object detection and semantic segmentation》 ,
<https://arxiv.org/abs/1311.2524>, 这篇论文的算法思想被称为R-CNN(Regions with Convolutional Neural Network Features), 该算法思想更改了物体检测的总思路, 现在很多物体检测算法均基于该算法。
- 🕒 简单来讲: **R-CNN是一种基于Region Proposal的CNN网络结构。**

⌚ R-CNN算法中采用和传统目标检测类似的执行过程，主要包含以下几个步骤：

- ⌚ 1. 区域提名(Region Proposal)：通过区域提名的方法从原始图片中提取2000个左右的区域候选框，一般选择Selective Search(SS)；
- ⌚ 2. 区域大小归一化：将所有候选框大小缩放固定大小(eg:227*227)；
- ⌚ 3. 特征提取：通过CNN网络，对每个候选区域框提取高阶特征，将高阶特征保存磁盘；
- ⌚ 4. 分类与回归：在高阶特征的基础上添加两个全连接层，再使用SVM分类器来进行识别，用线性回归来微调边框位置与大小，其中每个类别单独训练一个边框回归器。

RCNN



R-CNN: *Regions with CNN features*

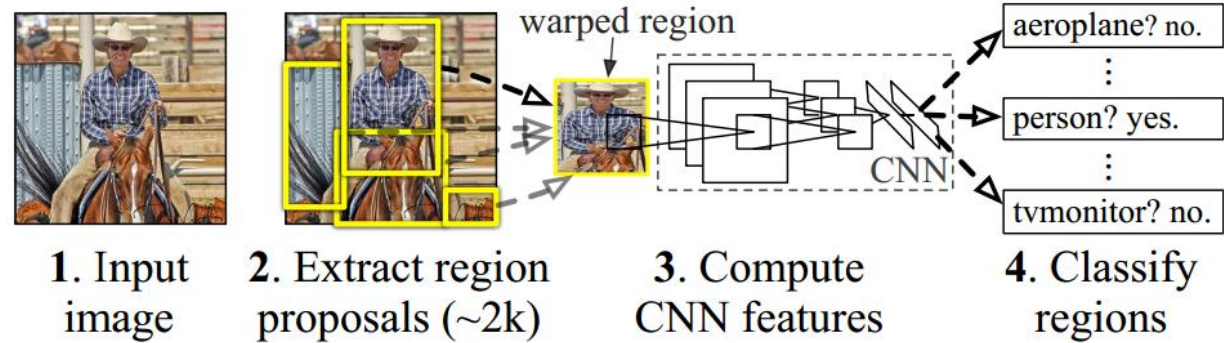


Figure 1: Object detection system overview. Our system (1) takes an input image, (2) extracts around 2000 bottom-up region proposals, (3) computes features for each proposal using a large convolutional neural network (CNN), and then (4) classifies each region using class-specific linear SVMs. R-CNN achieves a mean average precision (mAP) of **53.7% on PASCAL VOC 2010**. For comparison, [39] reports 35.1% mAP using the same region proposals, but with a spatial pyramid and bag-of-visual-words approach. The popular deformable part models perform at 33.4%. On the 200-class **ILSVRC2013 detection dataset**, **R-CNN's mAP is 31.4%**, a large improvement over OverFeat [34], which had the previous best result at 24.3%.

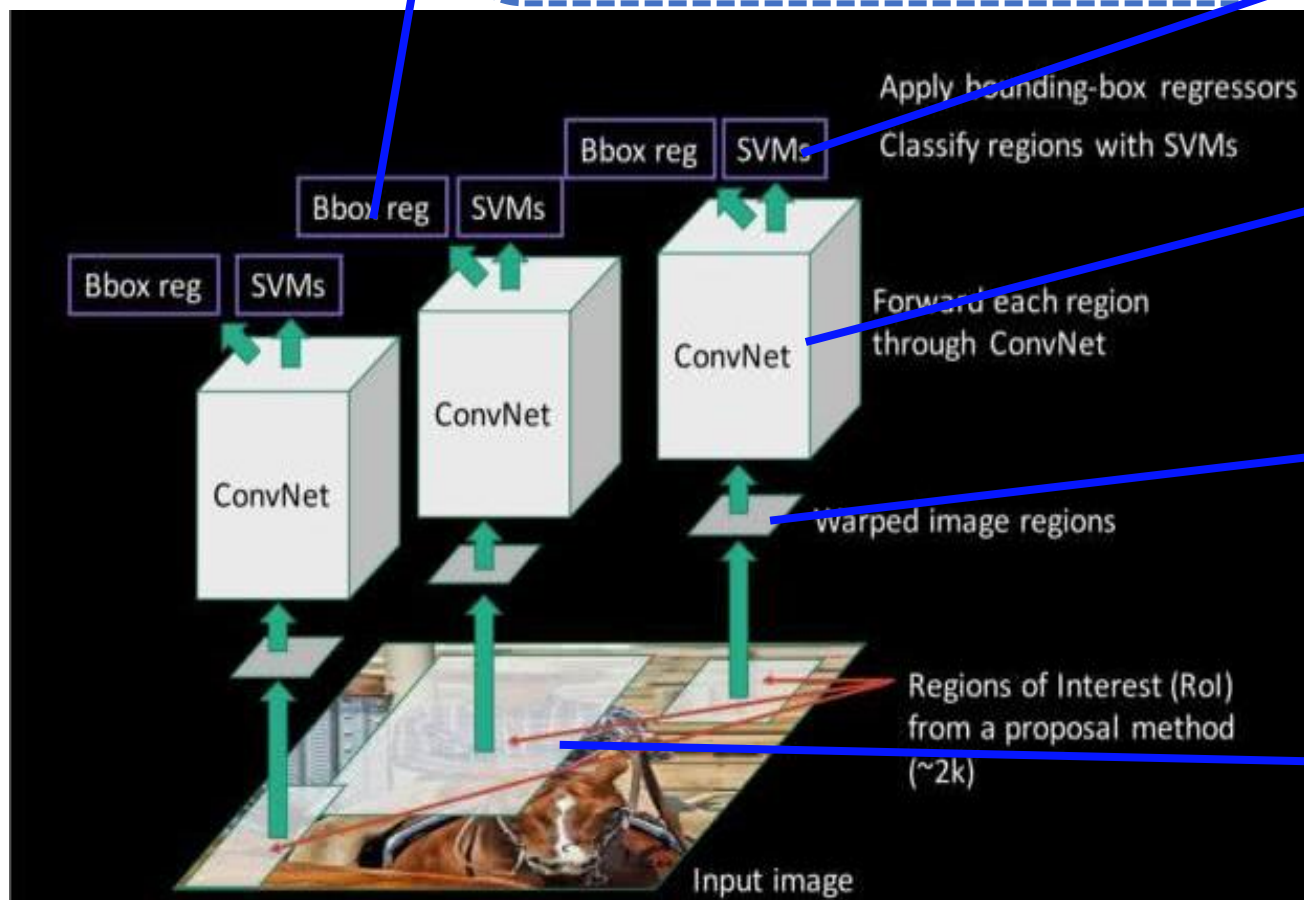
RCNN

模块4: Bounding Box回归模型

- * 对SS提供的ROI进行精化
- * 基于AlexNet的Conv5层输出的特征进行线性转换
- * IoU阈值0.6(>0.6)才进行训练
- * 每个类别一个BB回归模型
- * 使用NMS对SS提供的ROI进行剔除

模块3: 线性SVMs分类器

- * 基于AlexNet的FC7层输出的特征(4096)进行分类(就是特征向量 x)
- * 每个类别一个SVM
- * 样本IoU阈值0.3(GT $\rightarrow$ 正, $<0.3 \rightarrow$ 负)



模块2: AlexNet网络

- * 预训练(加载执行训练好的AlexNet模型)
- * fine-tune微调(为了训练特征提取功能)
- * 样本IoU阈值0.5(GT $+ \geq 0.5 \rightarrow$ 正, $<0.5 \rightarrow$ 负)

ROI预处理

- * Bounding Box膨胀(16px)
- * 尺寸缩放变换 227×227

模块1: Selective Search(SS)获取ROI

- * 2000个区域Region Proposals
- * 均包含可能的物体, 和分类无关

RCNN

- ⌚ 步骤一：利用Selective Search得到若干候选区域(所有可能是物体的区域);
- ⌚ 步骤二：对于每个候选区域(2k、缩放变换)分别使用CNN提取高阶特征向量，并且特征向量存储到磁盘中；
- ⌚ 步骤三：对于每个候选区域提取的高阶特征向量使用SVM进行分类(二分类)；
- ⌚ 步骤四：边框回归，对边框位置进行修正；
- ⌚ 步骤五：使用非极大值抑制剔除重复框。

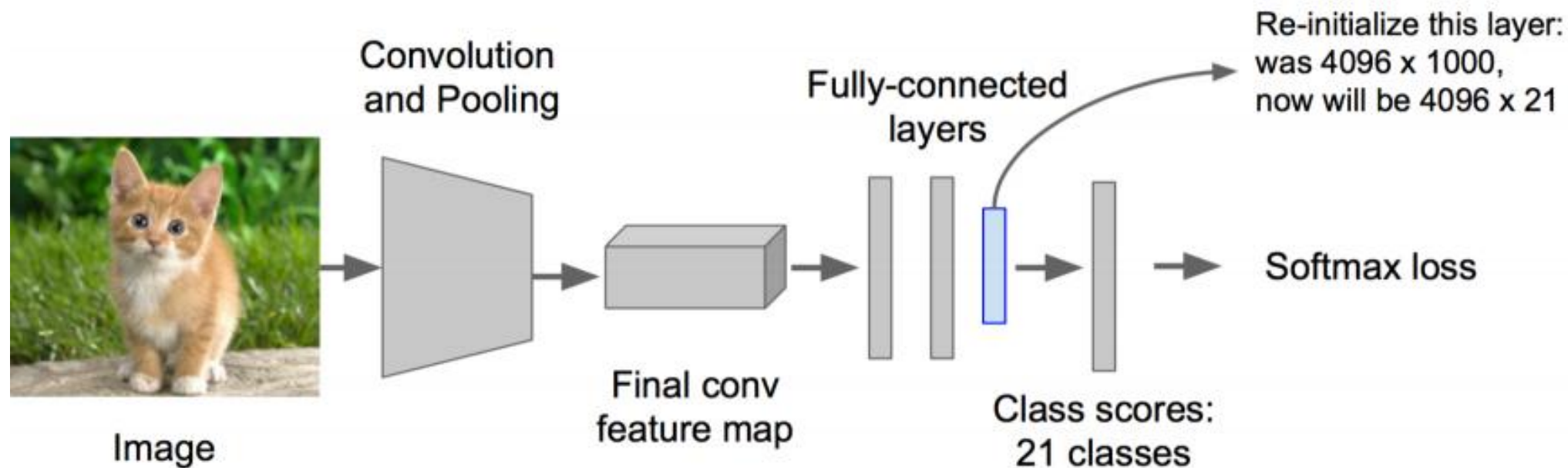
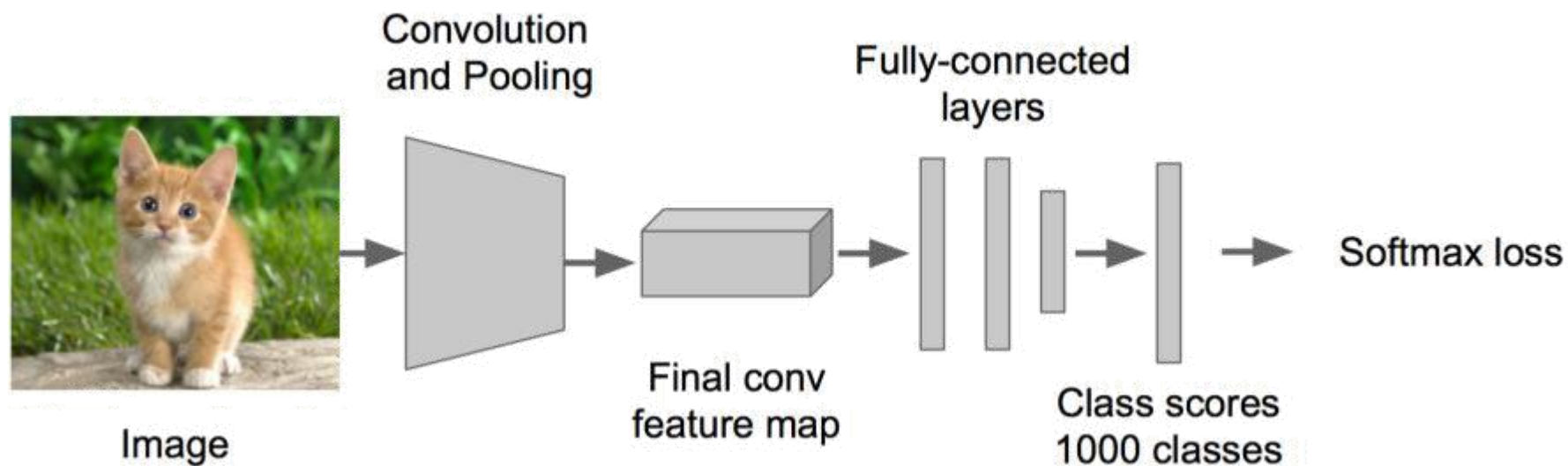
RCNN

⌚ 使用padding、填充
(真实内容填充、特殊
值填充)、缩放等操作,
将候选区域转换为
227*227的图像大小。



Figure 7: Different object proposal transformations. (A) the original object proposal at its actual scale relative to the transformed CNN inputs; (B) tightest square with context; (C) tightest square without context; (D) warp. Within each column and example proposal, the top row corresponds to $p = 0$ pixels of context padding while the bottom row has $p = 16$ pixels of context padding.

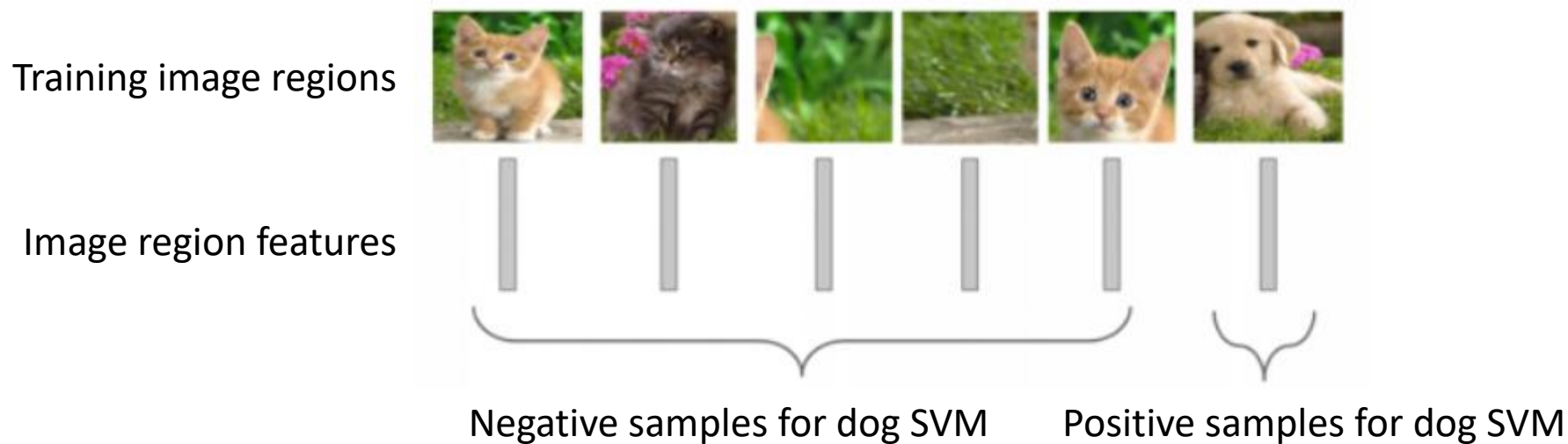
RCNN



RCNN

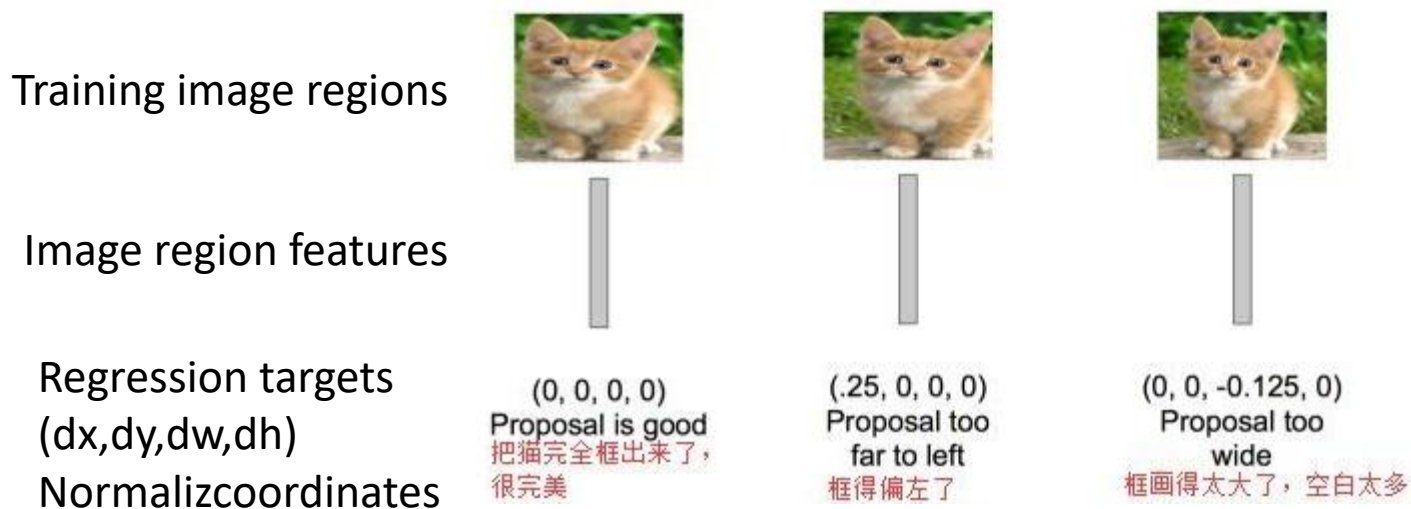
⌚ 对于进行大小缩放的ROI区域图像，使用CNN网络提取出区域图像对应的高阶特征信息(FC7层)，然后使用SVM算法将其划分为两个类别：

- ⌚ 正样本：包含当前类别的区域图像
- ⌚ 负样本：不包含当前类别的区域图像



RCNN

⌚ 对于进行大小缩放的ROI区域图像，使用CNN网络提取出区域图像对应的高阶特征信息(Conv5层)，使用回归器精细修正候选框位置：对于每一个类，训练一个线性回归模型去判定这个框是否框得完美。



RCNN_分类模型训练

- ⌚ 在RCNN模型训练中，主要有以下几点：
 - ⌚ 直接使用AlexNet作为前面的特征提取预训练模型；
 - ⌚ 将AlexNet的最后一层替换成 $N+1$ 个输出神经元的CNN softmax模型，然后使用fine-tuning训练，batch_size为128，其中32个正样本，96个负样本。最终得到当前任务上的**特征提取网络**。
 - ⌚ 在SVM训练阶段，假设要检测车辆，Ground Truth的Bounding Box当做正样本，和Ground Truth重叠IOU值低于0.3的bounding box当做负样本。比如提取出2000个候选框，然后高阶特征维度为4096，也就是可以得到一个 2000×4096 的特征向量矩阵，那么这个时候只需要训练一个 $4096 \times N$ 的权重系数 W 就可以完成SVM的训练(N 为分类类别数目)。

RCNN_分类模型训练

⌚ 为什么选择FC层的输出作为ROI的高阶特征？

⌚ 直接把CNN当做特征提取器，卷积层所学到的特征其实就是基础的共享特征提取层(位置信息、轮廓信息....)，而FC全连接是将特征任务相关的特征信息提取出来(将局部的高阶特征的合并成全局的)。

⌚ 为什么进行fine-tuning？

⌚ 如果不进行fine-tuning训练，相当于直接把AlexNet模型当做万金油使用，也就是不针对特定的任务，直接应用提取的特征，这种方式没有考虑到具体的任务相关特征。

RCNN_分类模型训练

- ⌚ 为什么在CNN softmax模型训练之外，需要单独训练一个SVM分类器呢？
 - ⌚ 这是因为svm训练和cnn训练过程中的正负样本定义方式不同，导致CNN softmax的输出比采用svm的精度低。在CNN softmax中，将IOU**大于等于0.5**的当做正样本，**IOU小于0.5**的当做负样本；而在SVM中，将Ground Truth当做正样本，**IOU小于0.3**的当做负样本。
 - ⌚ SVM分类器训练过程中，采用标准的hard negative mining方式进行训练，即：**难负例挖掘算法**，用途就是正负例数量不均衡，而负例分散代表性又不够的问题，hard negative就是每次把那些顽固的棘手的错误，再送回去继续练，练到你的成绩不再提升为止。这一个过程就叫做“hard negative mining”。
 - ⌚ CNN softmax模型的分类识别能力比较强，鲁棒性会更强（泛化能力更强，也就是对于区域中仅包含部分轮廓的情况，也可以产生一个当前类别的预测），对于精准的定位效果就不会特别的好。

RCNN_分类模型预测

- ⌚ 1. 对于输入图像，使用Selective Search提取2000个候选区域；
- ⌚ 2. 使用训练好的特征抽取网络对2000个ROI进行特征抽取，每个ROI输出一个4096维的特征向量(FC7层)；
- ⌚ 3. 使用N个SVM分类器对这4096维的特征向量进行预测分类，如果返回值+1(正例)表示当前ROI为当前类别，并计算属于当前类别的概率值作为置信度。

RCNN_回归模型训练

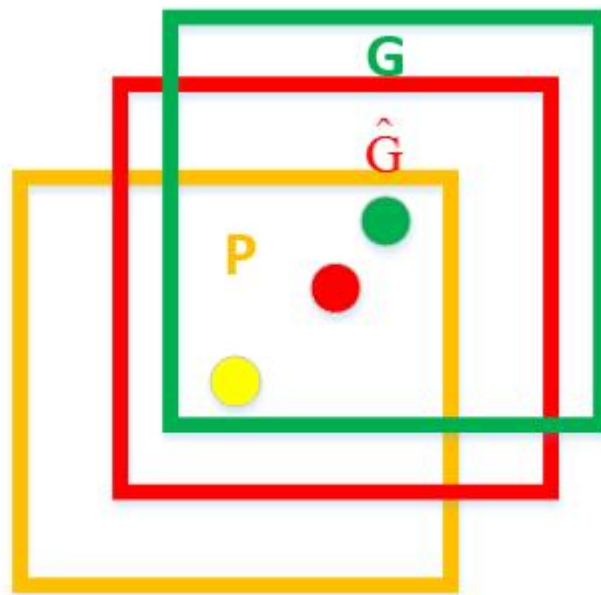
⌚ 为了进一步的提高定位的精确率，引入 Bounding Box Regression(BBR)进行定位边框的微调(基于DPM模型)。



- 如上图所示，绿色框为实际标准的卡宴车辆框，即Ground Truth；黄色框为 Selective Search算法得出的建议框，即Region Proposal。即使黄色框中物体，被分类器识别为卡宴车辆，但是由于绿色框和黄色框IoU值并不大，所以最后的目标检测精度并不高。采用回归器是为了对建议框进行校正，使得校正后的Region Proposal与Ground Truth更接近， 以提高最终的检测精度。

RCNN_回归模型训练

⌚ 如下图，黄色框口P表示建议框Region Proposal，绿色窗口G表示实际框Ground Truth，红色窗口 $\hat{G}$ 表示Region Proposal进行回归后的预测窗口，现在的目标是找到P到 $\hat{G}$ 的线性变换。



RCNN_回归模型训练

⌚ Region Proposal窗口表示:

$$P = (P_x, P_y, P_w, P_h)$$

⌚ 分别表示中心点坐标横轴值、中心点坐标纵轴值、窗口宽度、窗口高度。

RCNN_回归模型训练

⌚ Ground Truth窗口表示:

$$G = (G_x, G_y, G_w, G_h)$$

⌚ Region Proposal进行回归转换后的窗口表示:

$$\hat{G} = (\hat{G}_x, \hat{G}_y, \hat{G}_w, \hat{G}_h)$$

RCNN_回归模型训练

- ⌚ 定义四种变换函数，其中中心点通过对x和y进行平移变换得到，宽度w和高度h通过进行缩放变换得到。计算公式如下所示：

$$\hat{G}_x = P_w d_x(P) + P_x$$

$$\hat{G}_y = P_h d_y(P) + P_y$$

$$\hat{G}_w = P_w \exp(d_w(P))$$

$$\hat{G}_h = P_h \exp(d_h(P))$$

RCNN_回归模型训练

⌚ 结合上述转换函数，我们的目的是要求线性转换之后的边框和实际边框非常接近，那么也就是目标转换系数为(也就是模型训练值<offset box，位置偏移量>)：

$$G_x = t_x \times P_w + P_x$$

$$G_y = t_y \times P_h + P_y$$

$$G_w = e^{t_w} \times P_w$$

$$G_h = e^{t_h} \times P_h$$

$$t_x = (G_x - P_x) / P_w$$

$$t_y = (G_y - P_y) / P_h$$

$$t_w = \log(G_w / P_w)$$

$$t_h = \log(G_h / P_h)$$

回归模型对应的训练标签值

RCNN_回归模型训练

- ⌚ 每一个 $d_*(P)$ 【*表示x、y、w、h】都是CNN网络的高阶特征提取值 (Conv5层)的线性转换函数, 即:

$$d_*(P) = w_*^T \Phi(P)$$

- ⌚ 这里的w即需要学习的回归参数, 损失函数如下:

$$Loss = \arg \min \sum_{i=0}^N \left(t_*^i - w_*^T \Phi(P^i) \right)^2 + \lambda \|w_*\|^2$$

RCNN_回归模型训练

- ⌚ 为了提高样本框回归训练的有效性，对于每类样本中采用与Ground Truth 相交IoU大于0.6的Region Proposal作为样本对(P,G)进行BBR的训练。之所以选择**IoU大于0.6**的进行回归微调，其主要原因是如果边框间隔太远，那么其转换关系可能就不是非线性转换，使用线性转换的方式有点不work。
- ⌚ 对于每个类别均训练一个BBR线性转换结构。也就是RCNN中最终由N个BBR线性转换。

RCNN_回归模型预测

- ⌚ 使用RCNN的分类模型进行预测，得到当前图像上各个RoI属于各个类别的概率值(置信度)，形状为:2000*(N+1)的矩阵。
- ⌚ 对于每个类别(N个类别)，基于给定的概率阈值，选择超过阈值的边框作为当前的临时预测边框(得到边框的候选xywh + 预测类别 + 预测概率)；
- ⌚ 对于每个临时预测边框区域在Conv5层的输出高阶特征值，使用对应预测类别的BBR线性转换对RoI位置进行微调。
- ⌚ 针对所有位置修正后的预测边框，采用非极大值抑制的方式，以置信度高的区域为基础，剔除重叠位置的区域，最终得到的就是各个预测区域以及区域对应的类别。

RCNN

VOC 2010 test	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv	mAP
DPM v5 [20] [†]	49.2	53.8	13.1	15.3	35.5	53.4	49.7	27.0	17.2	28.8	14.7	17.8	46.4	51.2	47.7	10.8	34.2	20.7	43.8	38.3	33.4
UVA [39]	56.2	42.4	15.3	12.6	21.8	49.3	36.8	46.1	12.9	32.1	30.0	36.5	43.5	52.9	32.9	15.3	41.1	31.8	47.0	44.8	35.1
Regionlets [41]	65.0	48.9	25.9	24.6	24.5	56.1	54.5	51.2	17.0	28.9	30.2	35.8	40.2	55.7	43.5	14.3	43.9	32.6	54.0	45.9	39.7
SegDPM [18] [†]	61.4	53.4	25.6	25.2	35.5	51.7	50.6	50.8	19.3	33.8	26.8	40.4	48.3	54.4	47.1	14.8	38.7	35.0	52.8	43.1	40.4
R-CNN	67.1	64.1	46.7	32.0	30.5	56.4	57.2	65.9	27.0	47.3	40.9	66.6	57.8	65.9	53.6	26.7	56.5	38.1	52.8	50.2	50.2
R-CNN BB	71.8	65.8	53.0	36.8	35.9	59.7	60.0	69.9	27.9	50.6	41.4	70.0	62.0	69.0	58.1	29.5	59.4	39.3	61.2	52.4	53.7

Table 1: Detection average precision (%) on VOC 2010 test. R-CNN is most directly comparable to UVA and Regionlets since all methods use selective search region proposals. Bounding-box regression (BB) is described in Section C. At publication time, SegDPM was the top-performer on the PASCAL VOC leaderboard. [†]DPM and SegDPM use context rescoring not used by the other methods.

VOC 2007 test	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv	mAP
R-CNN pool ₅	51.8	60.2	36.4	27.8	23.2	52.8	60.6	49.2	18.3	47.8	44.3	40.8	56.6	58.7	42.4	23.4	46.1	36.7	51.3	55.7	44.2
R-CNN fc ₆	59.3	61.8	43.1	34.0	25.1	53.1	60.6	52.8	21.7	47.8	42.7	47.8	52.5	58.5	44.6	25.6	48.3	34.0	53.1	58.0	46.2
R-CNN fc ₇	57.6	57.9	38.5	31.8	23.7	51.2	58.9	51.4	20.0	50.5	40.9	46.0	51.6	55.9	43.3	23.3	48.1	35.3	51.0	57.4	44.7
R-CNN FT pool ₅	58.2	63.3	37.9	27.6	26.1	54.1	66.9	51.4	26.7	55.5	43.4	43.1	57.7	59.0	45.8	28.1	50.8	40.6	53.1	56.4	47.3
R-CNN FT fc ₆	63.5	66.0	47.9	37.7	29.9	62.5	70.2	60.2	32.0	57.9	47.0	53.5	60.1	64.2	52.2	31.3	55.0	50.0	57.7	63.0	53.1
R-CNN FT fc ₇	64.2	69.7	50.0	41.9	32.0	62.6	71.0	60.7	32.7	58.5	46.5	56.1	60.6	66.8	54.2	31.5	52.8	48.9	57.9	64.7	54.2
R-CNN FT fc ₇ BB	68.1	72.8	56.8	43.0	36.8	66.3	74.2	67.6	34.4	63.5	54.5	61.2	69.1	68.6	58.7	33.4	62.9	51.1	62.5	64.8	58.5
DPM v5 [20]	33.2	60.3	10.2	16.1	27.3	54.3	58.2	23.0	20.0	24.1	26.7	12.7	58.1	48.2	43.2	12.0	21.1	36.1	46.0	43.5	33.7
DPM ST [28]	23.8	58.2	10.5	8.5	27.1	50.4	52.0	7.3	19.2	22.8	18.1	8.0	55.9	44.8	32.4	13.3	15.9	22.8	46.2	44.9	29.1
DPM HSC [31]	32.2	58.3	11.5	16.3	30.6	49.9	54.8	23.5	21.5	27.7	34.0	13.7	58.1	51.6	39.9	12.4	23.5	34.4	47.4	45.2	34.3

Table 2: Detection average precision (%) on VOC 2007 test. Rows 1-3 show R-CNN performance without fine-tuning. Rows 4-6 show results for the CNN pre-trained on ILSVRC 2012 and then fine-tuned (FT) on VOC 2007 trainval. **Row 7 includes a simple bounding-box regression (BB) stage that reduces localization errors** (Section C). Rows 8-10 present DPM methods as a strong baseline. The first uses only HOG, while the next two use different feature learning approaches to augment or replace HOG.

RCNN

模块4: Bounding Box回归模型

- * 对SS提供的ROI进行精化
- * 基于AlexNet的Conv5层输出的特征进行线性转换
- * IoU阈值0.6(>0.6)才进行训练
- * 每个类别一个BB回归模型
- * 使用NMS对SS提供的ROI进行剔除

模块3: 线性SVMs分类器

- * 基于AlexNet的FC7层输出的特征(4096)进行分类
- * 每个类别一个SVM模型
- * 样本IoU阈值0.3(GT $\rightarrow$ 正, $<0.3 \rightarrow$ 负)

模块2: AlexNet网络

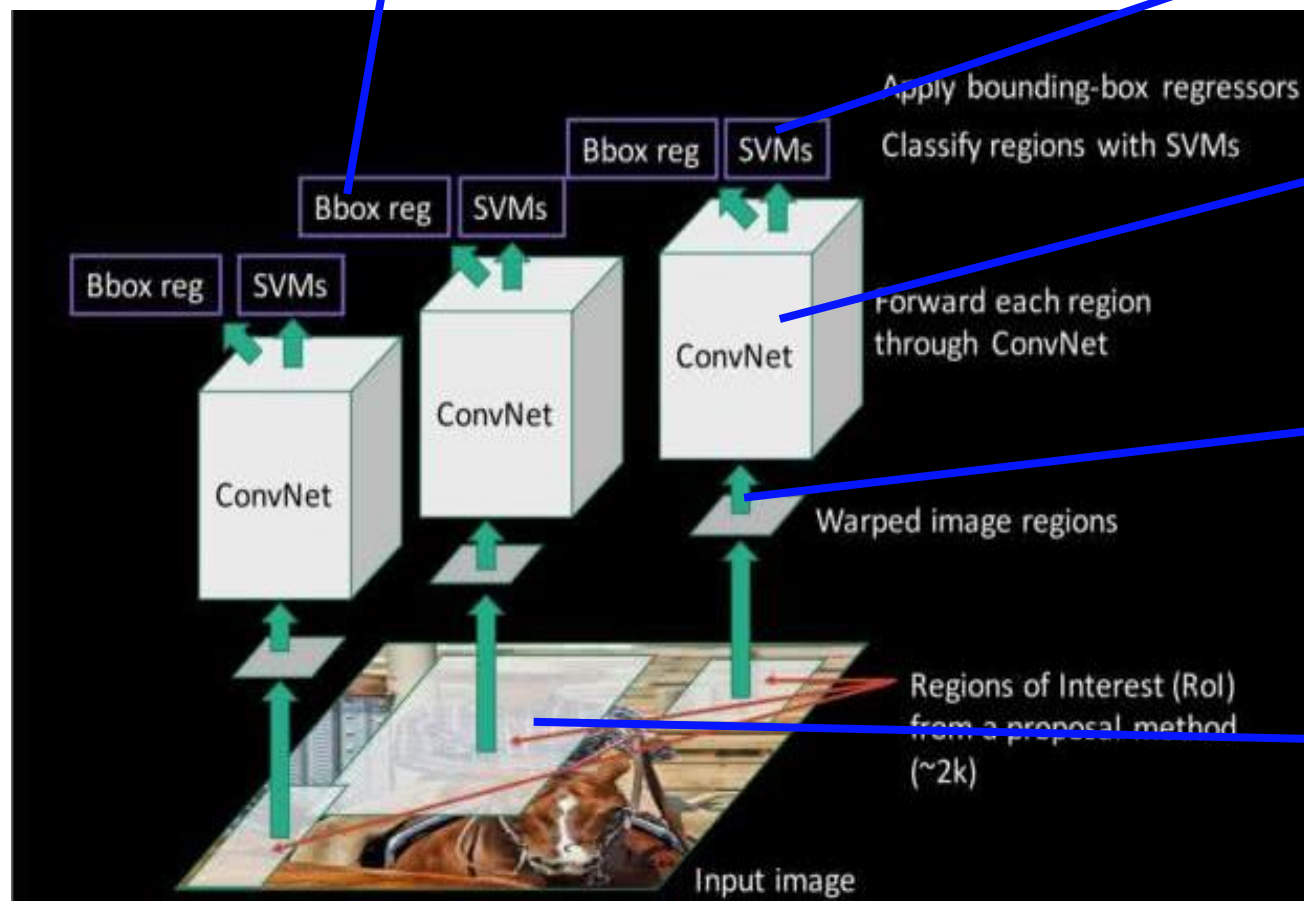
- * 预训练(加载执行训练好的AlexNet模型)
- * fine-tune微调(为了训练特征提取功能)
- * 样本IoU阈值0.5(GT $+ \geq 0.5 \rightarrow$ 正, $<0.5 \rightarrow$ 负)

ROI预处理

- * Bounding Box膨胀(16px)
- * 尺寸缩放变换 227×227

模块1: Selective Search(SS)获取ROI

- * 2000个区域Region Proposals
- * 均包含可能的物体, 和分类无关



RCNN

- ⌚ R-CNN采用AlexNet;
- ⌚ R-CNN采用Selective Search技术生成Region Proposal;
- ⌚ R-CNN在ImageNet上先进行预训练, 然后利用成熟的权重参数在 PASCAL VOC数据集上进行fine-tune;
- ⌚ R-CNN用CNN抽取特征, 然后用一系列的SVM做类别预测;
- ⌚ R-CNN的bbox位置回归基于DPM的灵感, 基于R-CNN提取出来的特征训练了一个线性回归模型;

RCNN

⌚ R-CNN已经不是最先进的目标检测模型，但是它是深度学习应用目标检测的奠基者，在RCNN中的很多缺陷问题，也在后续的目标检测网络结构中得以解决，其主要问题如下：

- ⌚ **计算量大**：R-CNN虽然不再是穷举，但依然有两千个左右的候选框，这些候选框都需要进行CNN操作，计算量依然很大，其中有不少其实是重复计算；
- ⌚ **训练测试分为多步**：区域提名、特征提取、分类、回归都是断开的训练的过程，中间数据还需要单独保存；
- ⌚ **训练的空间和时间代价很高**：卷积出来的特征需要先存在硬盘上，这些特征需要几百G的存储空间；
- ⌚ **执行效率慢**：GPU上处理一张图片需要13秒，CPU上则需要53秒。

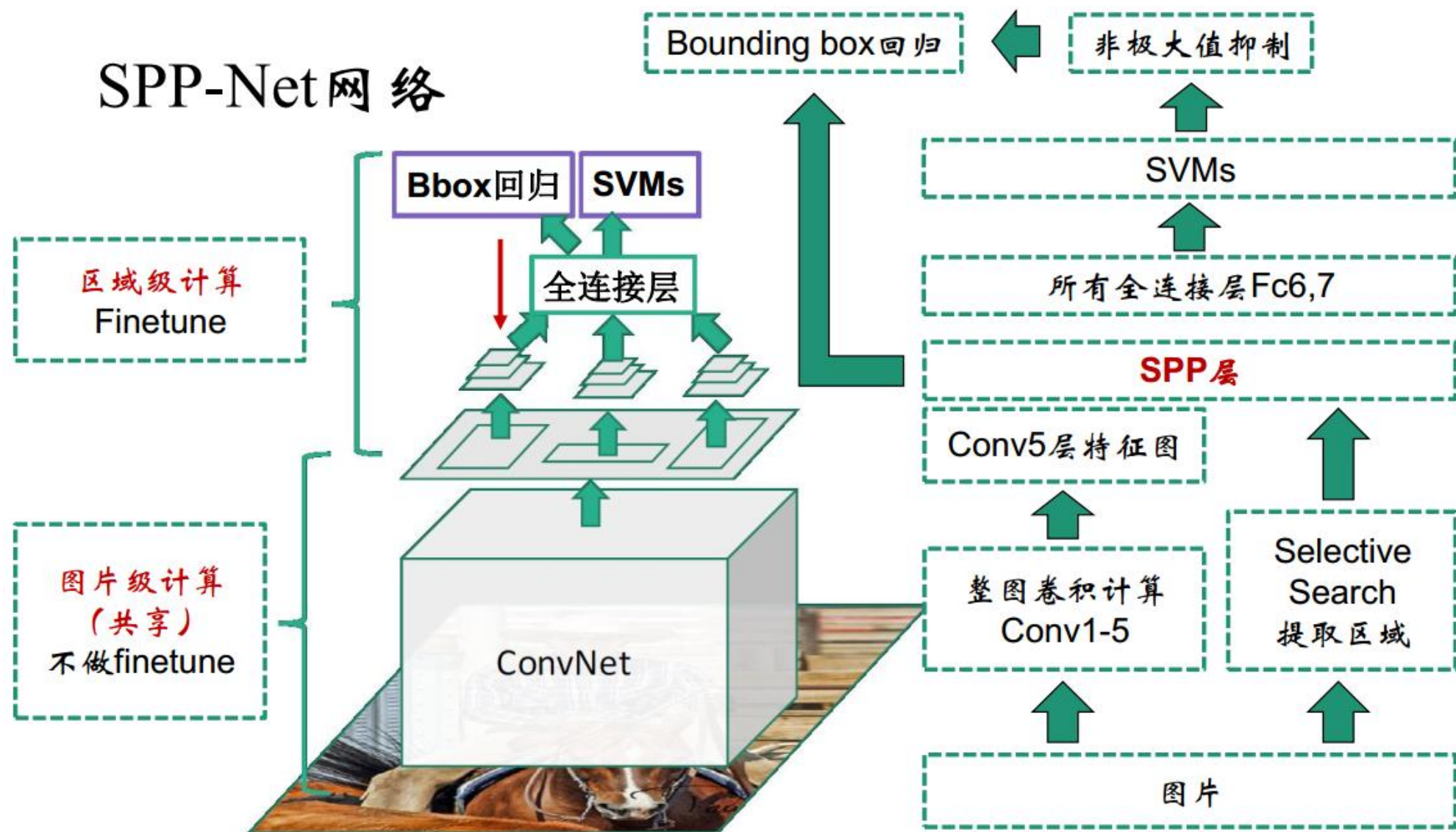
SPP Net

- ⌚ 2015年IEEE论文《Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition》,是对R-CNN的一种优化, SPP即Spatial Pyramid Pooling(空间金字塔池), 是一种基于图像金字塔结构的目标检测网络结构。
- ⌚ 其网络主要修正的是R-CNN网络的效率问题, 也就是主要更改的是**R-CNN中每个RoI区域均需要通过一个CNN提取高阶特征的这个阶段**的结构。

SPP Net

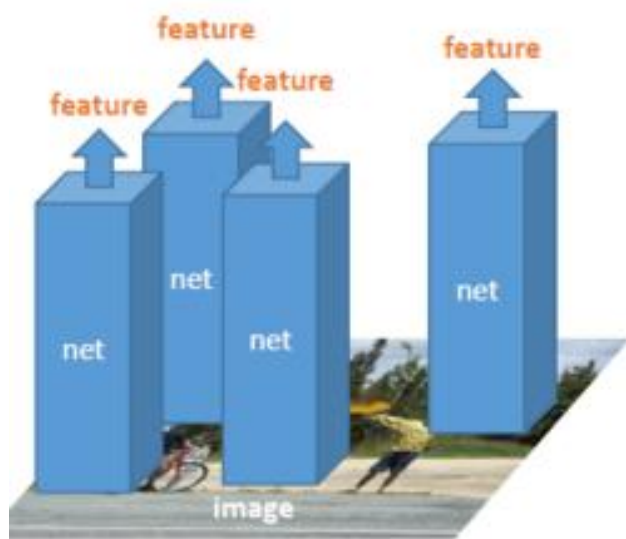
- ⌚ R-CNN的作用比较强，其主要缺陷(结合SPP Net)就是效率问题：
 - ⌚ **计算量大，速度慢**：每张图像的每个Propasal均需要通过CNN提取高阶特征、SVM进行分类等操作。
 - ⌚ **图像失真**：每个Propasal区域在输入CNN网络之前，需要进行warp拉伸/crop压缩操作，可能会影响图像质量以及模型效果。
- ⌚ SPP Net优化改进：
 - ⌚ 直接送入整个图像，所有区域**共享卷积计算(一遍)**
 - ⌚ 在Conv5层提取所有区域的高阶特征
 - ⌚ 引入空间金字塔池(**S**patial **P**yramid **P**ooling)
 - ⌚ 为不同尺寸的区域，在Conv5层上提取不同的特征，然后映射到尺寸固定的全连接上。

SPP Net

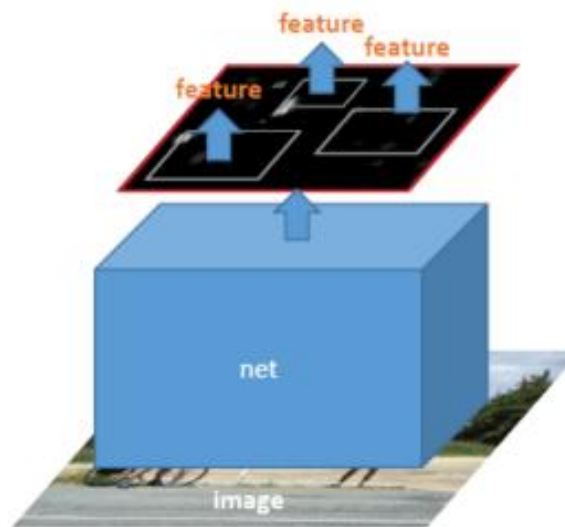


SPP Net

- ⌚ 相比R-CNN需要对每个proposal做卷积，SPP Net只做一次卷积，将相应的sub-image映射到full-image卷积后的feature map上从而得到sub-image通过卷积层得到的feature。



R-CNN
2000 nets on image regions



SPP-net
1 net on full image

SPP Net

🕒 卷积/池化操作的feature map大小变化公式:

The diagram illustrates the formula for calculating the output size of a feature map after a convolution or pooling operation. The formula is presented as a mathematical equation with arrows pointing from descriptive labels to its components.

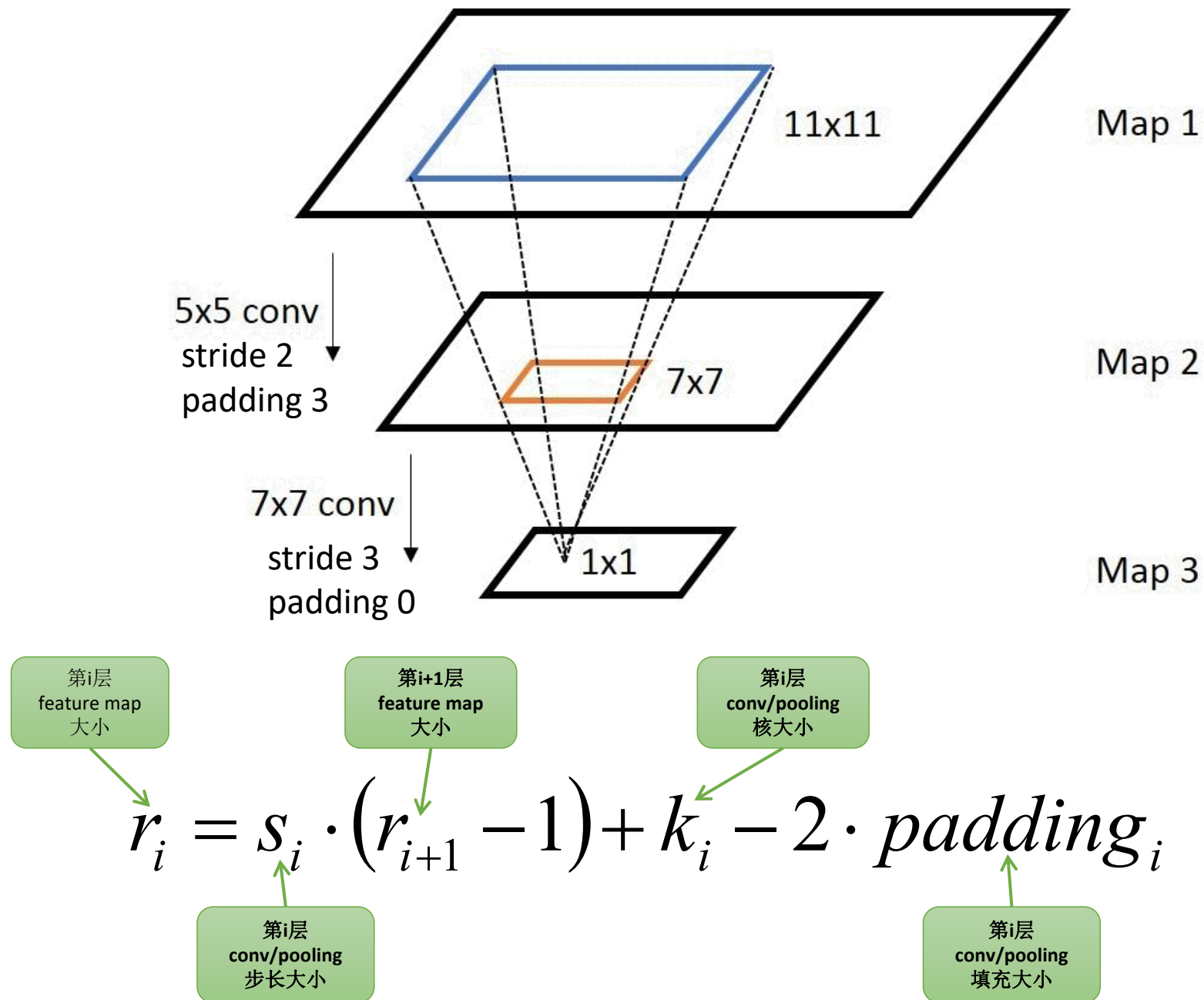
$$output_{size} = \frac{input_{size} - kernel_{size} + stride_{size} + 2 * padding}{stride_{size}}$$

The components of the formula are labeled as follows:

- Conv/Pooling 输入大小** (Conv/Pooling input size) points to $input_{size}$.
- Conv/Pooling 核大小** (Conv/Pooling kernel size) points to $kernel_{size}$.
- Conv/Pooling 的填充Padding** (Conv/Pooling padding) points to $padding$.
- Conv/Pooling 输出大小** (Conv/Pooling output size) points to $output_{size}$.
- Conv/Pooling 步长** (Conv/Pooling stride) points to $stride_{size}$ in both the numerator and denominator.

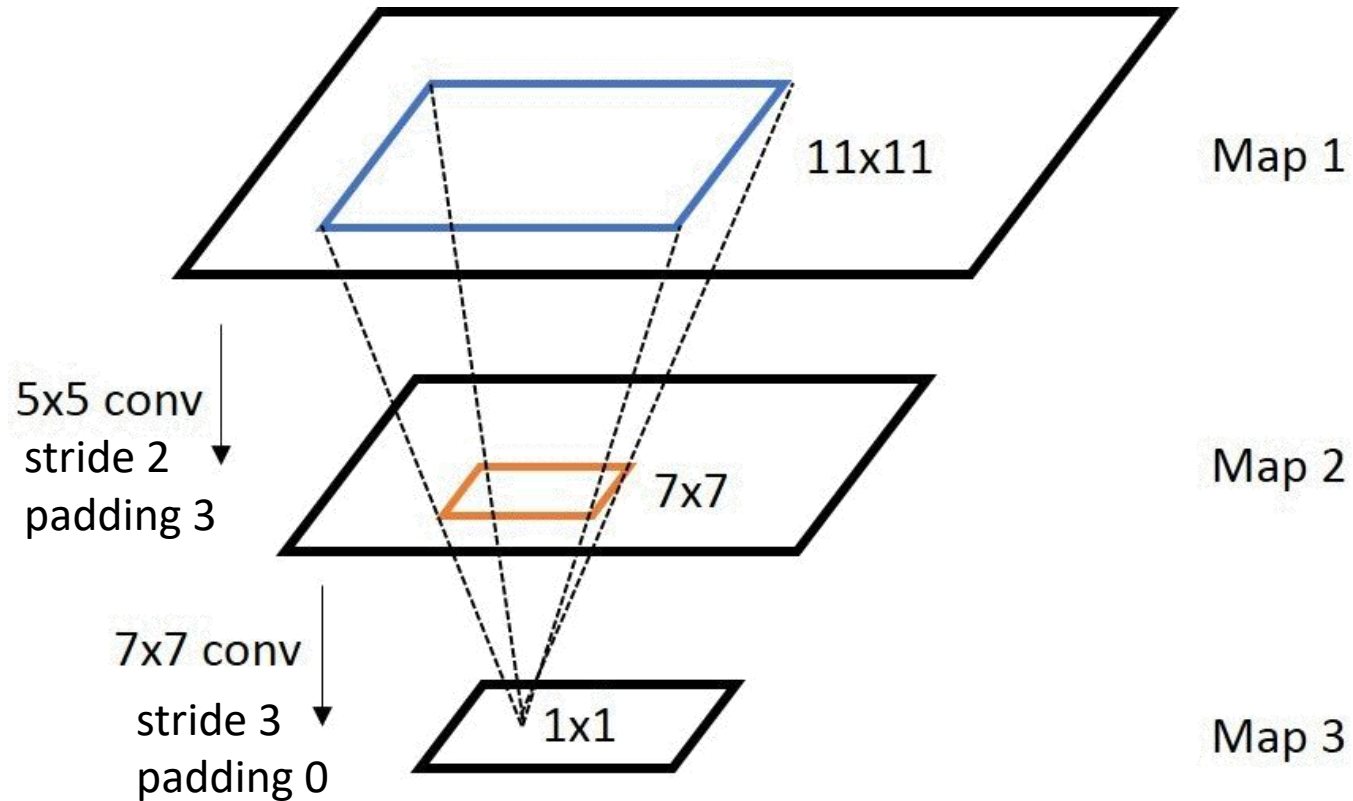
SPP Net_感受野

🕒 卷积神经网络CNN中，某一层输出结果中一个元素所对应的输入层的区域大小，被称作感受野receptive field。感受野的大小是由kernel size, stride, padding, outputsize一起决定的。



SPP Net_感受野

⌚ 和感受野映射类似，感受野坐标的映射(一般考虑中心点映射)<下标从0开始>。



$$p_i = s_i \cdot p_{i+1} + \left(\frac{k_i - 1}{2} - padding \right)$$

第i层
点坐标值

第i层
conv/pooling
步长大小

第i+1层
点坐标值

第i层
conv/pooling
核大小

第i层
conv/pooling
填充大小

$$p_2 = 3p_3 + \left(\frac{7-1}{2} - 0 \right) = 3p_3 + 3$$

$$p_1 = 2p_2 + \left(\frac{5-1}{2} - 3 \right) = 2p_2 - 1$$

$$p_1 = 6p_3 + 5$$

SPP Net_感受野

- 🏆 原图像大小: 7*7
- 🏆 窗口大小: 3*3
- 🏆 步长为: 2
- 🏆 填充为: 0
- 🏆 新图像大小: 3*3

1	2	3	4	5	6	7
9	8	7	6	5	4	3
1	3	5	7	9	0	2
4	6	8	5	3	1	2
1	2	5	3	6	2	1
1	1	6	5	3	2	1
1	2	3	5	6	2	3

-1	-1	-1
-1	8	-1
-1	-1	-1

0

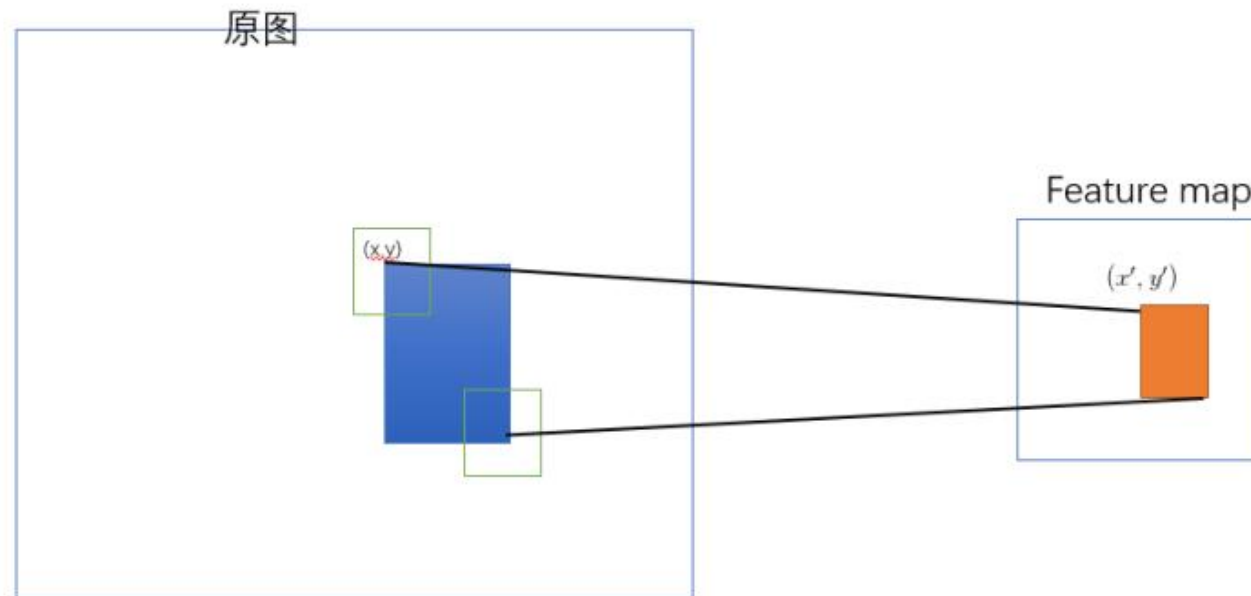
33	3	-5
19	-6	-17
-13	3	-8

$$p_i = s_i \cdot p_{i+1} + \left(\frac{k_i - 1}{2} - padding \right)$$

$$(1,2) \rightarrow \begin{cases} 2 * 1 + \frac{3-1}{2} = 3 \\ 2 * 2 + \frac{3-1}{2} = 5 \end{cases} \rightarrow (3,5)$$

SPP Net

- ⌚ 假设原始点坐标为 (x,y) , 那么 feature map 上的坐标点为 (x',y')
- ⌚ 假设padding大小均为核的一半。
- ⌚ 之所以左上角取+1, 右下角取-1的主要目的是为了获取的 feature map 区域更加的精确。以保障模型的准确性。(也可以不 ± 1)。



$$p_i = s_i \cdot p_{i+1} + \left(\frac{k_i - 1}{2} - padding \right) \quad padding = \left\lfloor \frac{k_i}{2} \right\rfloor$$

$$p_i = s_i \cdot p_{i+1} \quad S = \prod_{i=0} s_i \quad p_0 = S \cdot p_{i+1}$$

$$\begin{cases} x = S \cdot x' \\ y = S \cdot y' \end{cases} \rightarrow \begin{cases} x' = \lfloor x/S \rfloor + 1 \\ y' = \lfloor y/S \rfloor + 1 \end{cases} \quad \text{左上角点}$$
$$\begin{cases} x' = \lceil x/S \rceil - 1 \\ y' = \lceil y/S \rceil - 1 \end{cases} \quad \text{右下角点}$$

SPP Net_感受野

0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	0
0	9	8	7	6	5	4	3	0
0	1	3	5	7	9	0	2	0
0	4	6	8	5	3	1	2	0
0	1	2	5	3	6	2	1	0
0	1	1	6	5	3	2	1	0
0	1	2	3	5	6	2	3	0
0	0	0	0	0	0	0	0	0

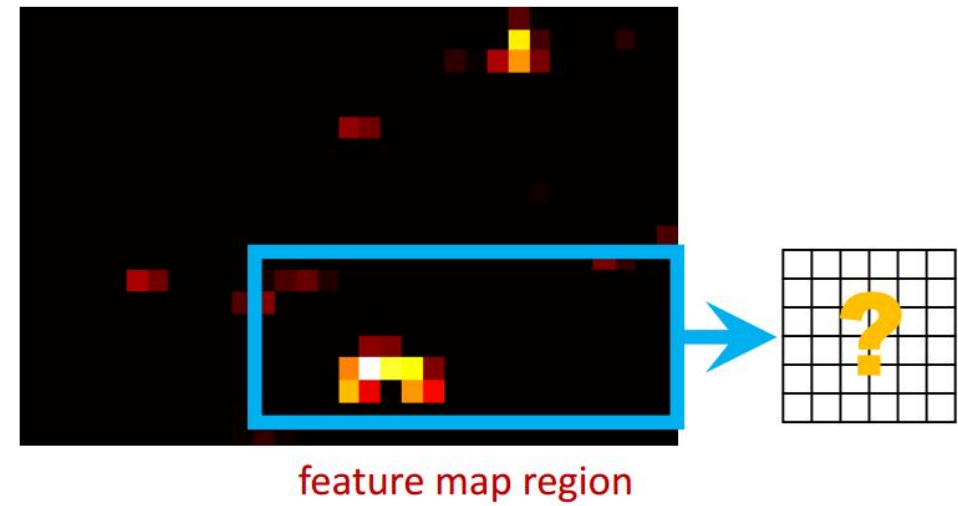
- 原图像大小: 7*7
- 窗口大小: 3*3
- 步长为: 2
- 填充为: 1
- 新图像大小: 4*4

-1	-1	-1
-1	8	-1
-1	-1	-1

0

-11			

SPP Net



SPP Net

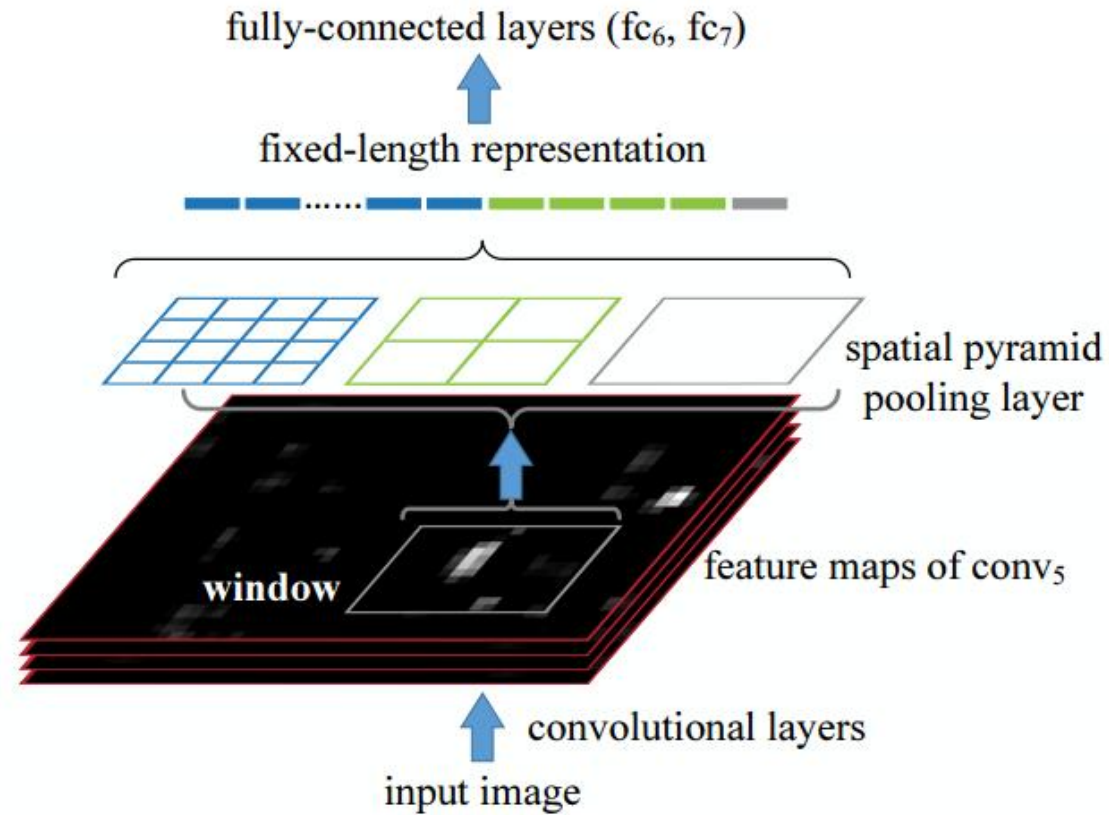


Figure 5: Pooling features from arbitrary windows on feature maps. The feature maps are computed from the entire image. The pooling is performed in candidate windows.

SPP Net

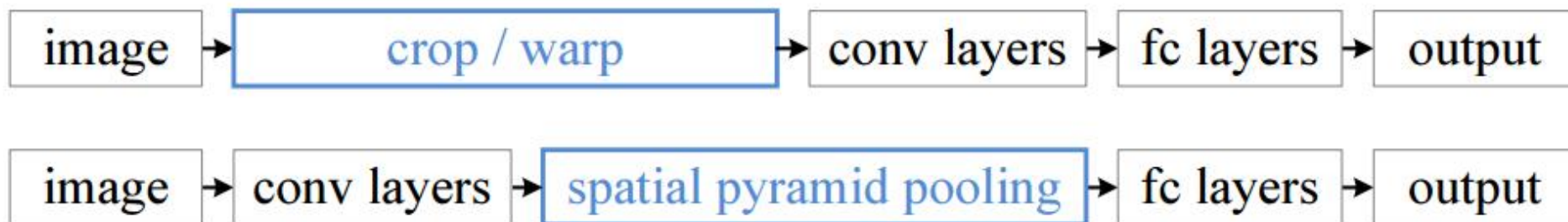
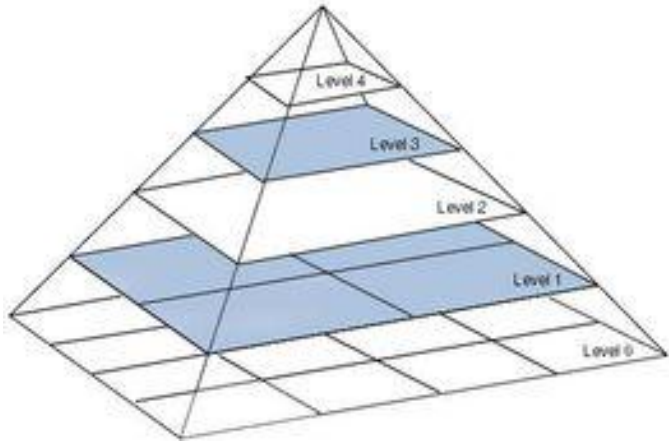


Figure 1: Top: cropping or warping to fit a fixed size. Middle: a conventional CNN. Bottom: our spatial pyramid pooling network structure.

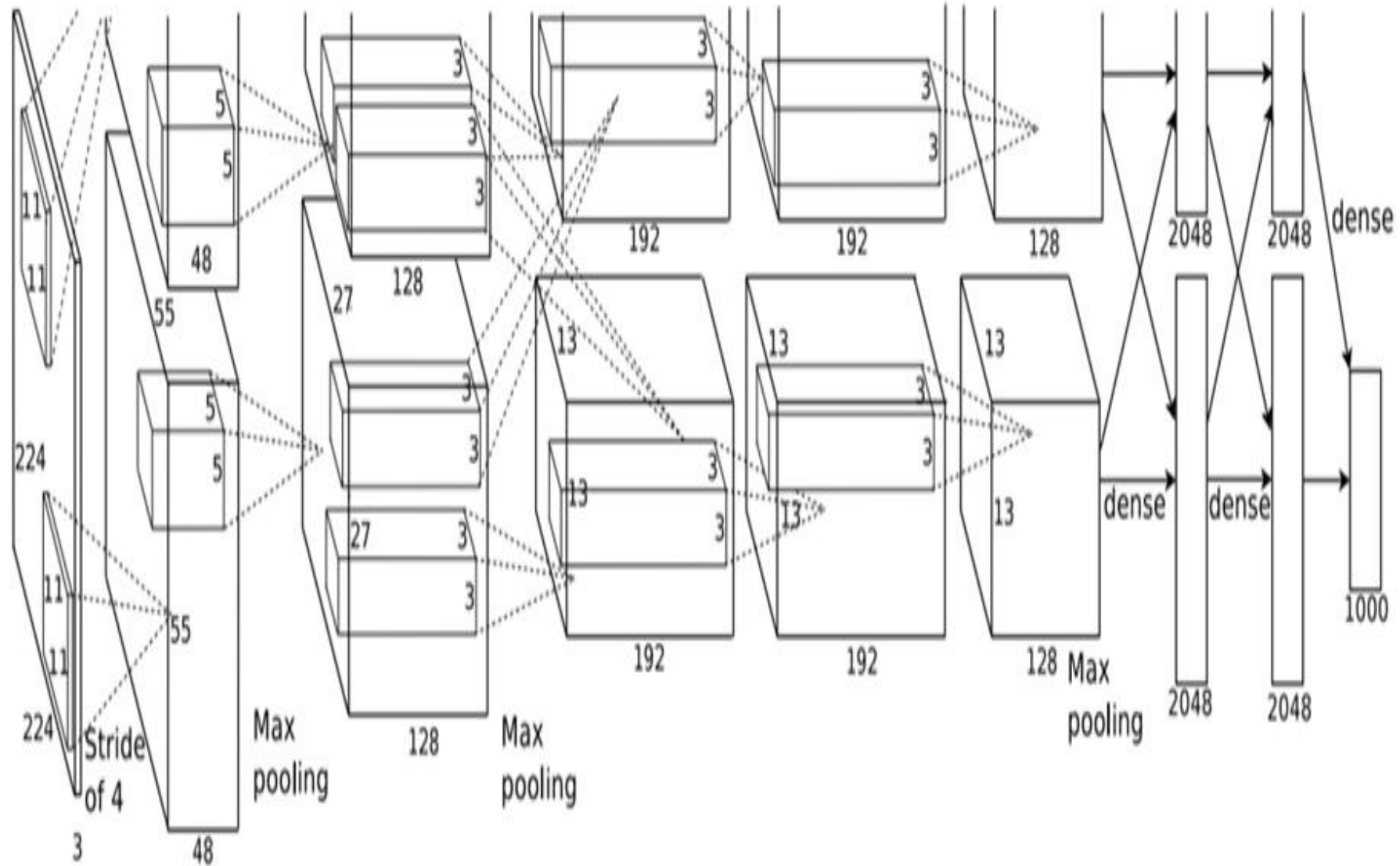
SPP Net

- ⌚ 在CNN卷积中，当输入的图像大小不一致的时候，如何解决？
 - ⌚ 方式一：对输入进行crop、padding、resize等操作，将其统一到相同大小。
 - ⌚ 方式二：在CNN之后，不进行FC全连接操作，将全连接转换为Global Average Pooling(GAP，即每个feature map输出一个均值作为特征值)操作。
 - ⌚ 方式三：在CNN之后，FC操作之前，加入SSP Layer操作。

SPP Net_图像金字塔



SPP Net



SPP Net

⌚ 在AlexNet网络中，现在输入两种格式的数据： $224*224*3$ 和 $180*180*3$ ；那么在Conv5层输出的Feature Map大小分别为： $13*13*256$ 和 $10*10*256$ ，那么经过Pooling操作后，输入FC层的大小分别为： $6*6*256$ 和 $4*4*256$ ；由于全连接层的神经元数目为4096，针对这两种不同的输入所需要的参数是不一样的，所以需要固定其FC的输入维度大小。

SPP Net

⌚ 空间金字塔池化

⌚ 替换Conv5层后的Pooling层

⌚ 3个Level、21个Bin

⌚ 4x4、2x2、1x1

⌚ Bin内使用Max-Pooling

⌚ size=向上取整(a/n)

⌚ stride=向下取整(a/n)

⌚ NOTE: a 为输入的feature map大小, n 为Bin大小(即Pooling输出feature map大小)
NOTE:实际上就是nn.
AdaptiveMaxPool2d的重叠

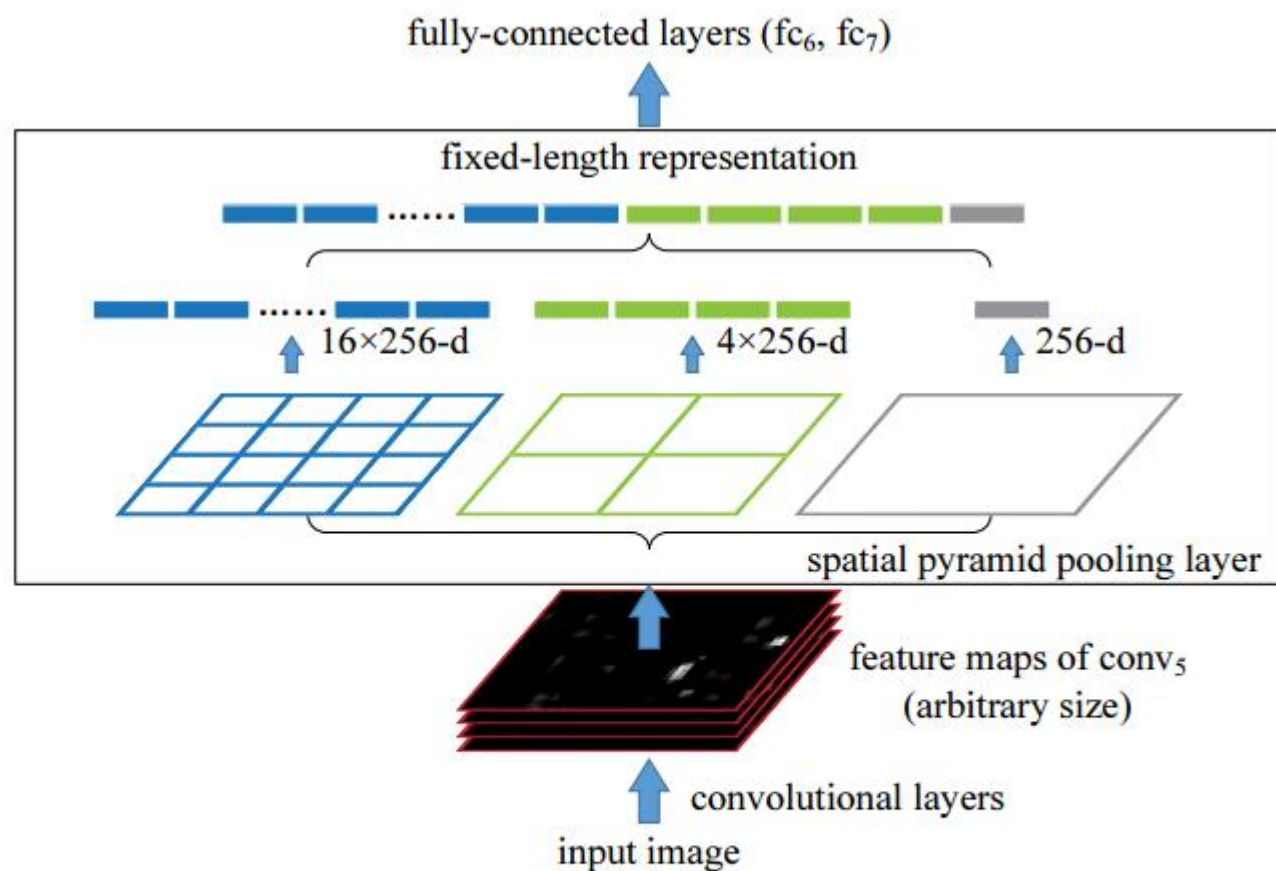


Figure 3: A network structure with a **spatial pyramid pooling layer**. Here 256 is the filter number of the conv₅ layer, and conv₅ is the last convolutional layer.

SPP Net

⌚ 当SSP Layer层输入的大小为 $13*13*256$ 的时候，其三个Level的结构如下：

⌚ Max Pooling Layer Level1：输出 $4*4*256$ ， $\text{size}=\text{ceil}(13/4)=4$ ， $\text{stride}=\text{floor}(13/4)=3$ ；

⌚ Max Pooling Layer Level2：输出 $2*2*256$ ， $\text{size}=\text{ceil}(13/2)=7$ ， $\text{stride}=\text{floor}(13/2)=6$ ；

⌚ Max Pooling Layer Level3：输出 $1*1*256$ ， $\text{size}=\text{ceil}(13/1)=13$ ， $\text{stride}=\text{floor}(13/1)=13$ ；

⌚ 最终输出大小为：concat+flatten($4*4*256$, $2*2*256$, $1*1*256$) ->
 $21*256$ -> $4096+1024+256=5376$ 维的特征向量

SPP Net

⌚ 当SSP Layer层输入的大小为 $10*10*256$ 的时候，其三个Level的结构如下：

⌚ Max Pooling Layer Level1：输出 $4*4*256$ ， $size=ceil(10/4)=3$ ， $stride=floor(10/4)=2$ ；

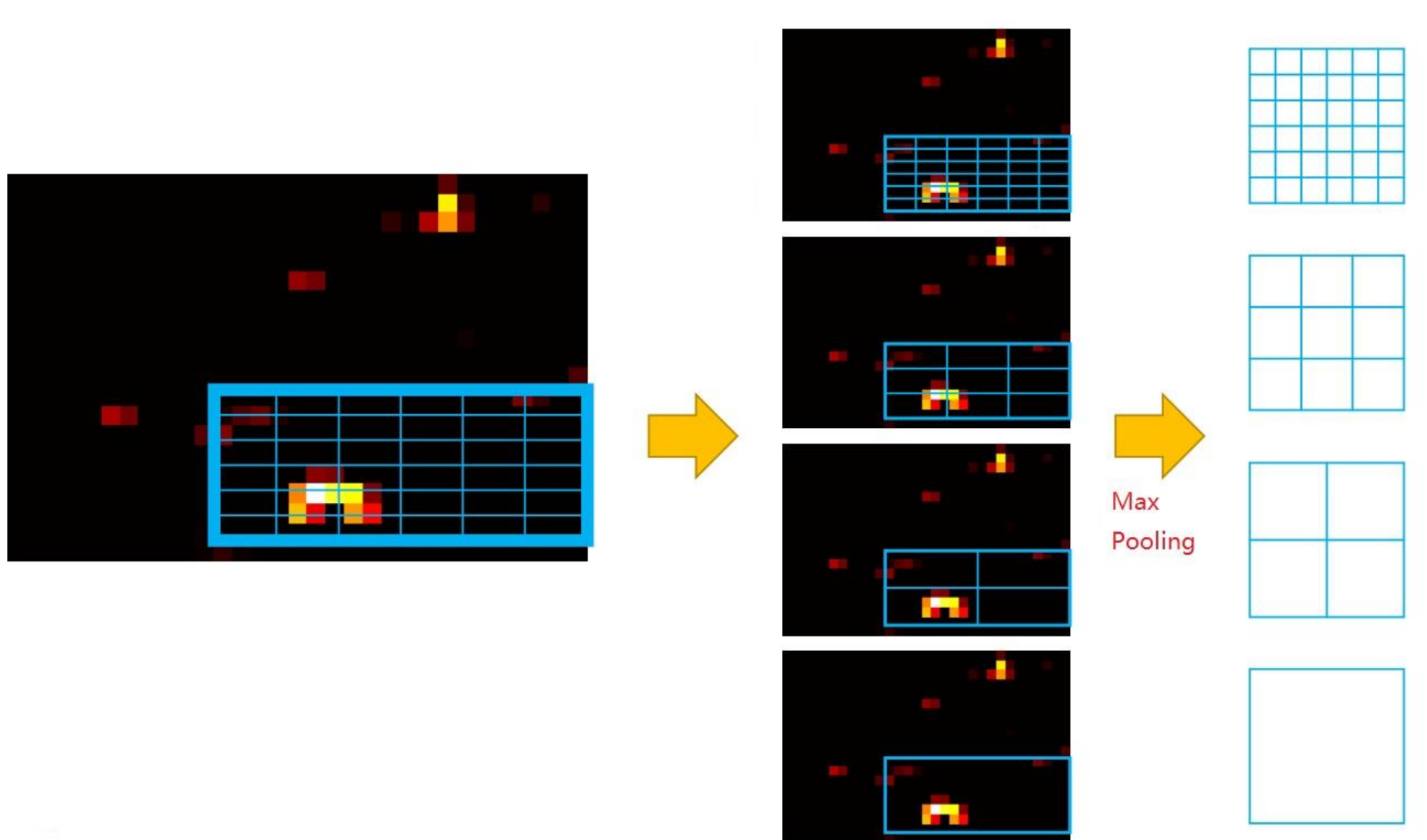
⌚ Max Pooling Layer Level2：输出 $2*2*256$ ， $size=ceil(10/2)=5$ ， $stride=floor(10/2)=5$ ；

⌚ Max Pooling Layer Level3：输出 $1*1*256$ ， $size=ceil(10/1)=10$ ， $stride=floor(10/1)=10$ ；

⌚ 最终输出大小为：concat+flatten($4*4*256$, $2*2*256$, $1*1*256$) -> $21*256$ -> $4096+1024+256=5376$ 维的特征向量

⌚ NOTE: 所有的Pooling层全部为VALID形式。

SPP Net



SPP Net_分类模型

- ⌚ 1. 加载在ImageNet数据集上pre-train的AlexNet模型;
- ⌚ 2. 更改最后一层卷积层Conv5后的Pooling成为SPP Layer;
- ⌚ 3. 根据业务(类别数目)更改最后一层输出层FC8的输出神经元大小(N);
- ⌚ 4. 使用新数据对FC6、FC7、FC8全连接层进行fine-tune训练;
 - ⌚ 使用**多阶段、多尺度的训练方式**, 在每一个epoch的时候, 我们先将图像放缩到一个size, 然后训练网络。训练完整后保存网络的参数, 然后resize 到另外一个尺寸, 并在之前权值的基础上再次训练模型。size取值范围: {224、256、300、360、448、560}。
- ⌚ NOTE: 预测和普通CNN网络一样。

SPP Net_检测模型训练_Net

- ⌚ 1. 和SPP Net的分类模型的前3步一样，更改网络结构, 输出类别 $N+1$;
- ⌚ 2. 将图像大小resize到size大小，然后输入到前面的CNN特征提取模型中，得到各个scale尺度下的图像feature map，size取值范围: {480, 576, 688, 864, 1200, min(w,h)};
- ⌚ 3. 使用Selective Search框出2000个左右的ROI区域;
- ⌚ 4. 在六个scale尺度图像中，找出ROI区域大小最接近 224×224 像素的scale尺度(不同尺度的原始图形上找);
- ⌚ 5. 在对应scale尺度对应的feature map上获取ROI区域对应的高阶特征;
- ⌚ 6. 将ROI的高阶特征信息通过SPP Layer提取为固定大小的vector向量;

SPP Net_检测模型训练_Net

⌚ 7. 使用ROI对应的vector向量训练FC6、FC7以及FC8的模型参数(fine-tune训练)。

⌚ NOTE:

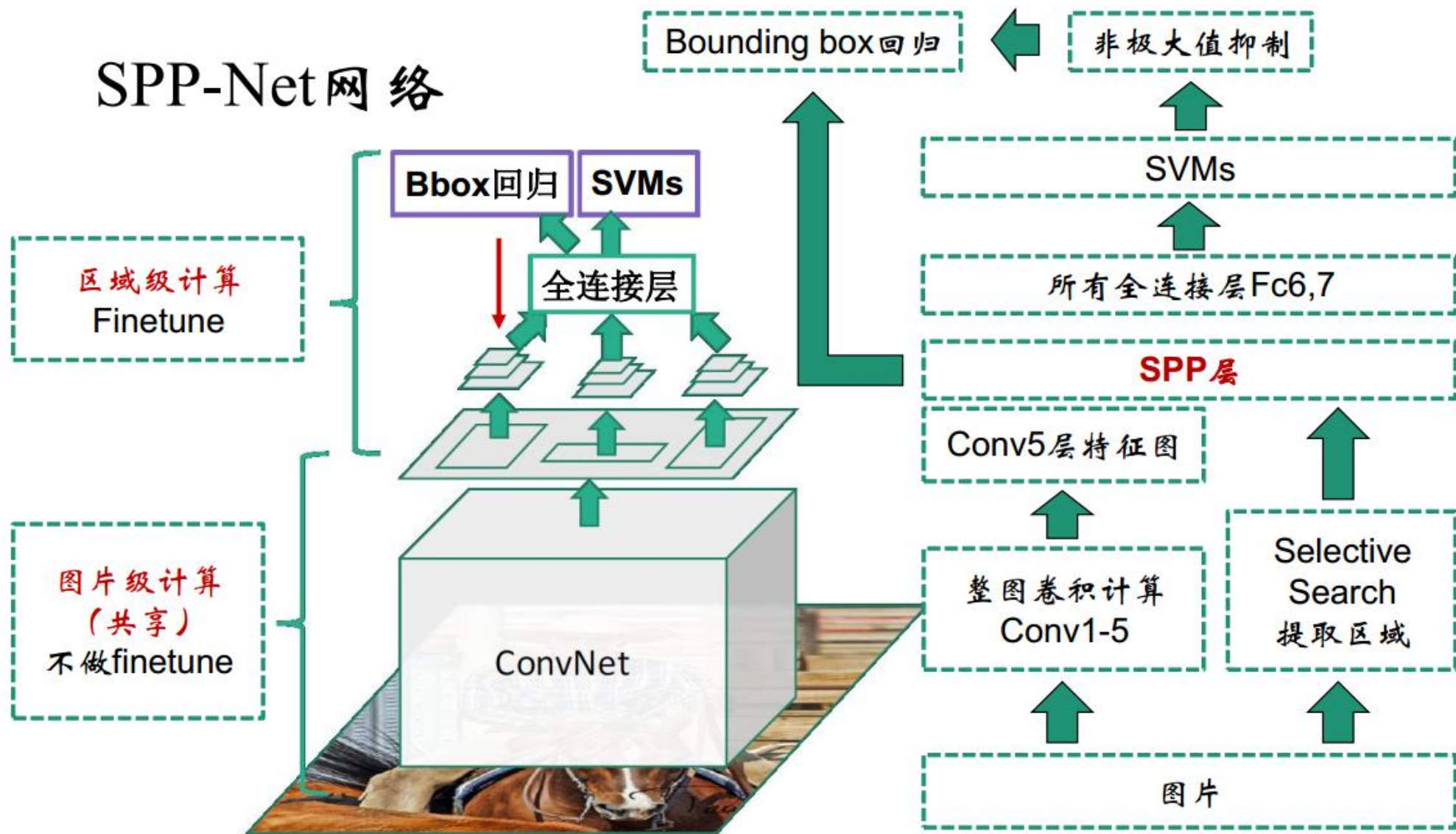
⌚ Ground Truth + IOU ≥ 0.5 的区域作为正例; IOU < 0.5 的区域作为负例;

⌚ 每个训练批次中25%的正例样本;

SPP Net_检测模型训练_SVM_BBR

- ⌚ 当Net网络训练完成后，SVM分类器和BBR回归器和R-CNN的结构完全一样。
- ⌚ 使用FC7层的输出特征属性作为SVM分类器的输入特征属性；
- ⌚ 使用Conv5经过SPP Net转换后的特征属性作为BBR回归器的输入特征属性；
- ⌚ 在SVM中，使用Ground Truth作为正例，使用IoU小于0.3的作为负例进行模型训练；
- ⌚ 在BBR中，使用IoU大于等于0.6的ROI区域框进行模型训练。

SPP Net



SPP Net

⌚ SPP Net相比于R-CNN，有以下几个优点：

⌚ 使用full-image卷积仅需要对原图像经过一次卷积操作就提取完所有区域的高阶特征属性；

⌚ 使用SPP结构支持候选框大小可大可小，解决了R-CNN中候选框crop/warp所带来的图像失真的问题。

⌚ SPP Net的缺点和R-CNN还是类似的：

⌚ 继续沿用R-CNN结构，特征提取、SVM分类器、BB回归器单独训练；

⌚ SPP Net没法fine tune训练前面的Conv部分的特征提取网络(效率低)。

Fast R-CNN

⌚ Fast R-CNN是在R-CNN的基础上融入了SPP Net网络优点之后的一个改进网络, 相比于R-CNN和SPP Net具有以下优点:

- ⌚ 训练/预测运行速度更快;
- ⌚ mAP效果评估指标更高;
- ⌚ 实现端到端(**end-to-end**)的**"单阶段"**模型训练;
 - ⌚ 多任务损失函数(Multi-task loss)
- ⌚ 所有层的参数都可以fine tune重训练;
- ⌚ 不需要离线存储特征文件;
- ⌚ **采用SVD简化FC全连接层的计算量;**
- ⌚ 使用Softmax替换SVM分类器;

Fast R-CNN

⌚ Fast R-CNN引入三种新技术:

⌚ **RoI Pooling Layer**

⌚ **Multi-task loss**(多任务损失函数)

⌚ **SVD**(类似连续的FC层<没有激活>)

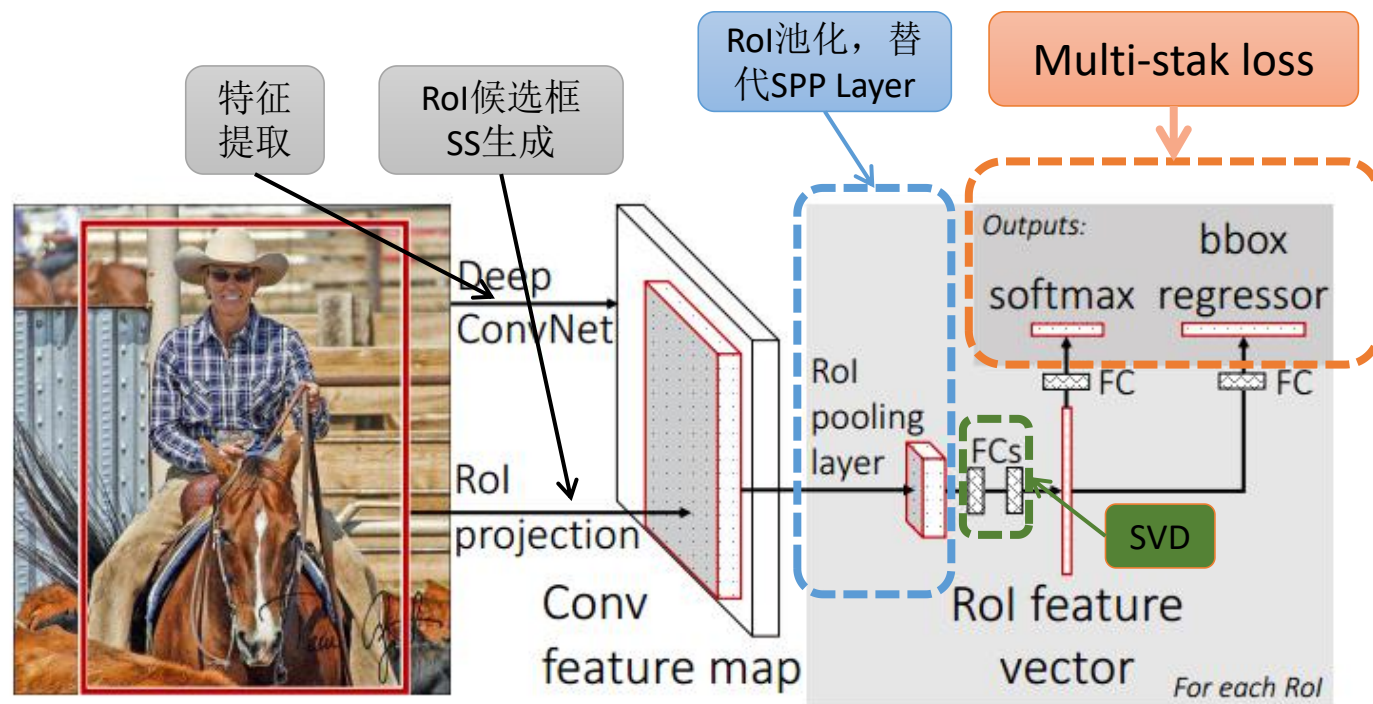


Figure 1. Fast R-CNN architecture. An input image and multiple regions of interest (RoIs) are input into a fully convolutional network. Each RoI is pooled into a fixed-size feature map and then mapped to a feature vector by fully connected layers (FCs). The network has two output vectors per RoI: softmax probabilities and per-class bounding-box regression offsets. The architecture is trained end-to-end with a multi-task loss.

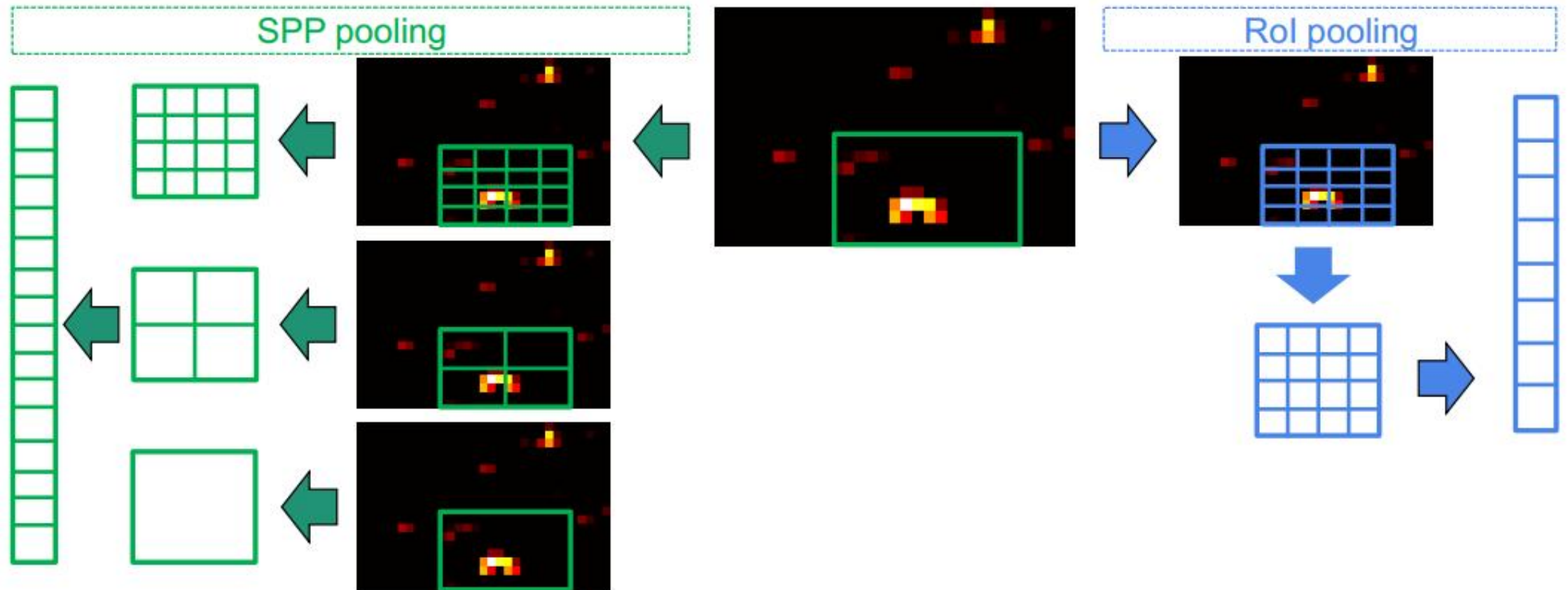
Fast R-CNN

⌚ ROI Pooling Layer

- ⌚ 空间金字塔结构SPP的单层特例;
- ⌚ 将ROI区域卷积高阶特征转换为HxW的结构;
- ⌚ 内部使用Max Pooling。



Fast R-CNN



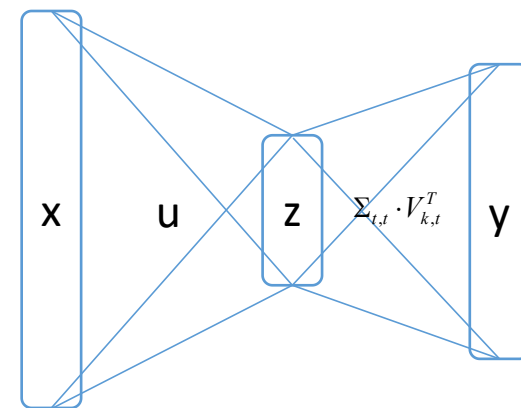
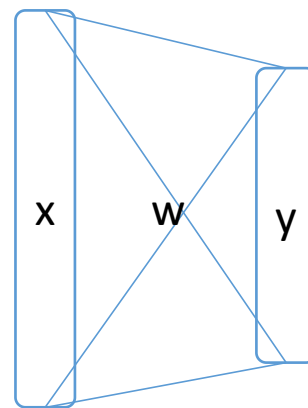
Fast R-CNN

⌚ 在R-CNN和SPP Net中，SS提取的ROI区域比较多，所以后面的全连接操作比较多，差不多得进行2K次全连接操作，因此计算量有点大，故Fast R-CNN中采用SVD分解加速全连接计算。在实现上，相当于将一个全连接拆分为两个全连接。

$$Y_{m,k} = X_{m,n} \cdot W_{n,k}$$

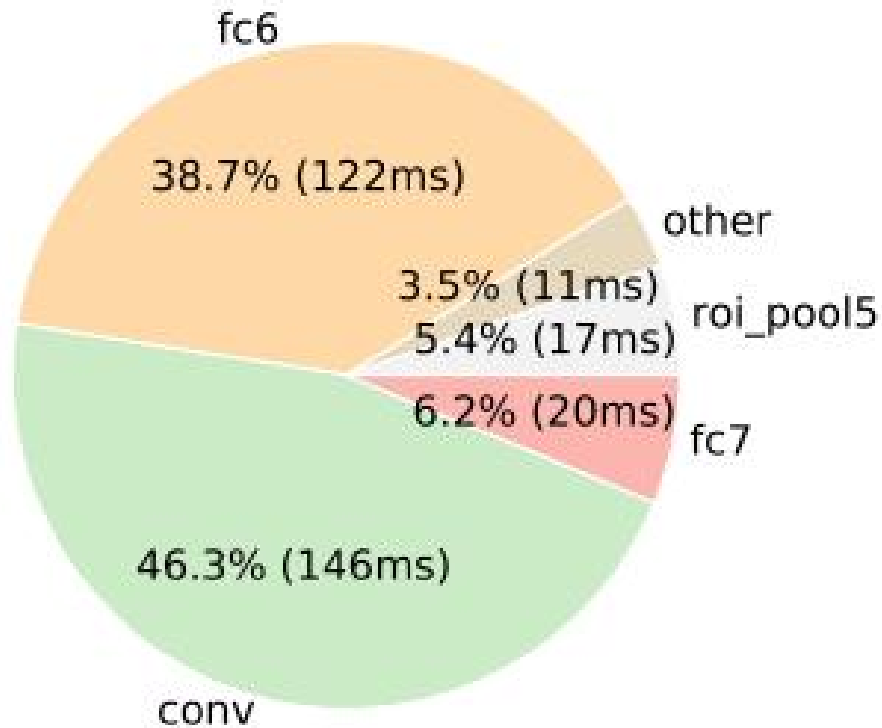
$$W_{n,k} = U \Sigma V^T \approx U_{n,t} \Sigma_{t,t} V_{k,t}^T$$

$$Y_{m,k} = X_{m,n} \cdot W_{n,k} = \underbrace{X_{m,n} \cdot U_{n,t}}_{Z_{m,t}} \cdot \underbrace{\Sigma_{t,t} \cdot V_{k,t}^T}_{Y_{m,k}}$$



Fast R-CNN

Forward pass timing
mAP 66.9% @ 320ms / image



Forward pass timing (SVD)
mAP 66.6% @ 223ms / image

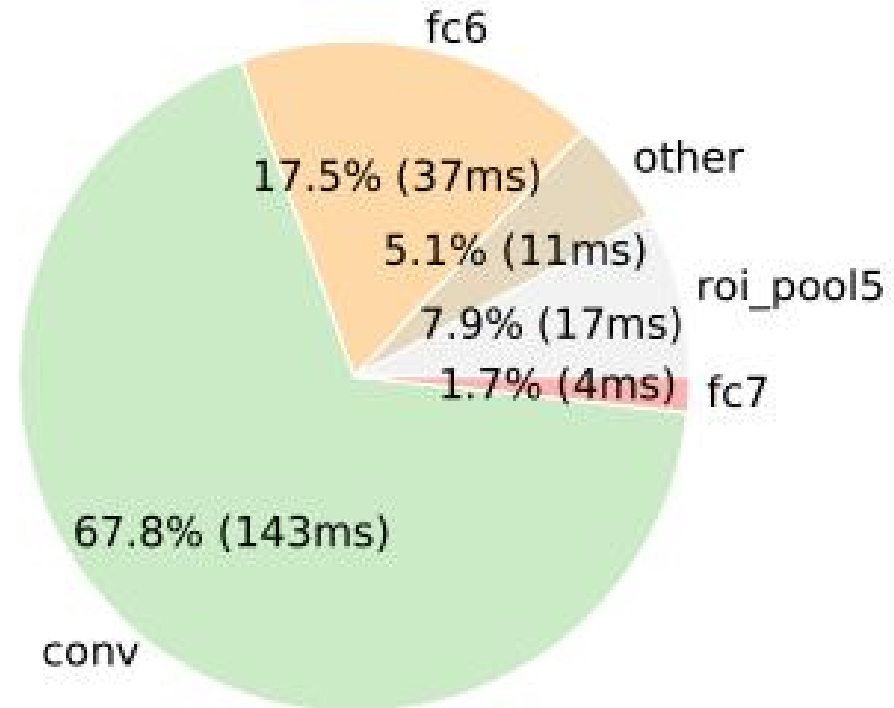


Figure 2. Timing for VGG16 before and after truncated SVD. Before SVD, fully connected layers fc6 and fc7 take 45% of the time.

Fast R-CNN

- ⌚ Multi-task loss: 包含分类分支和回归分支, 其中分类返回输出的是K+1个概率分布p, 回归分支输出的是预测位置(box offset偏移系数值)。

$$L(p, u, t^u, v) = L_{\text{cls}}(p, u) + \lambda[u \geq 1]L_{\text{loc}}(t^u, v)$$

- ⌚ 指示函数 $[u \geq 1]$

- ⌚ 当u=0的时候(当前候选框为背景/负例), 函数值为0;
- ⌚ 当u>=1的时候(当前候选框为物体/正例), 函数值为1;

Fast R-CNN

⌚ 分类器Loss:

$$L_{\text{cls}}(p, u) = -\log p_u$$

⌚ 每个ROI的概率分布

$$p = (p_0, \dots, p_K)$$

⌚ u: 训练阶段中Bounding Box的对应的类别值。

⌚ NOTE: u=0表示背景。

Fast R-CNN

⌚ 回归器Loss

$$L_{\text{loc}}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - v_i)$$

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

⌚ u: 训练阶段中Bounding Box的对应的类别值。 $v_x = (G_x - P_x) / P_w$

⌚ 偏差目标v: $v = (v_x, v_y, v_w, v_h)$ $v_y = (G_y - P_y) / P_h$

⌚ 预测偏差t: $t^u = (t_x^u, t_y^u, t_w^u, t_h^u)$ $v_w = \log(G_w / P_w)$

$$v_h = \log(G_h / P_h)$$

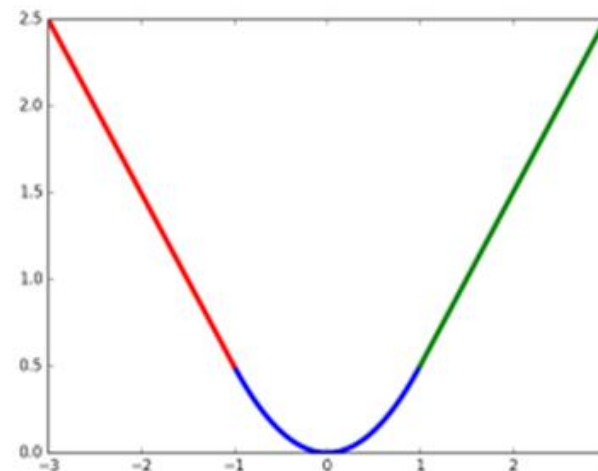
Fast R-CNN

⌚ 为什么采用Smooth L1 Loss损失函数呢？

⌚ Smooth L1 Loss相比于L2 Loss，对于**离群点更不敏感**，当预测值和实际值差值比较大的时候，使用L2 Loss容易出现梯度爆炸的问题，而使用Smooth L1 Loss不容易出现梯度爆炸。

⌚ Smooth L1 Loss相比于L1 Loss，在预测值接近实际值的位置($x=0$ 位置)，是连续可导的，收敛速度会更快。

⌚ Smooth L1 Loss的功能类似Huber Loss。



Fast R-CNN

- ⌚ 在pre train的基础模型(前面的Conv特征提取使用ImageNet数据集上训练好的AlexNet或者VGG网络)上进行fine tune训练操作

- ⌚ Mini-Batch Sampling抽样

- ⌚ 分级抽样法(Hierarchical Sampling)

- ⌚ 每个Batch抽取N个图像，然后从N个图像中抽取R个ROI区域组成训练数据

- ⌚ 论文建议：Batch尺寸(128) = 每个Batch的图像数目(2) * 每个图像的ROI区域数目(64)

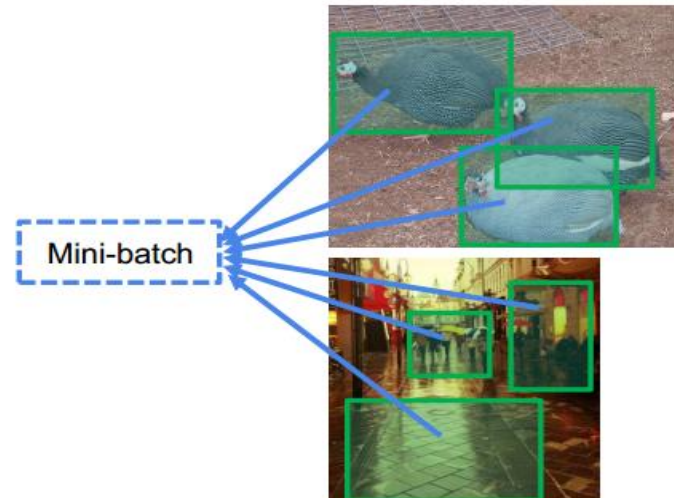
- ⌚ 论文建议：每个Batch中，至少25%的正例样本。

- ⌚ 充分利用卷积层的共享计算。

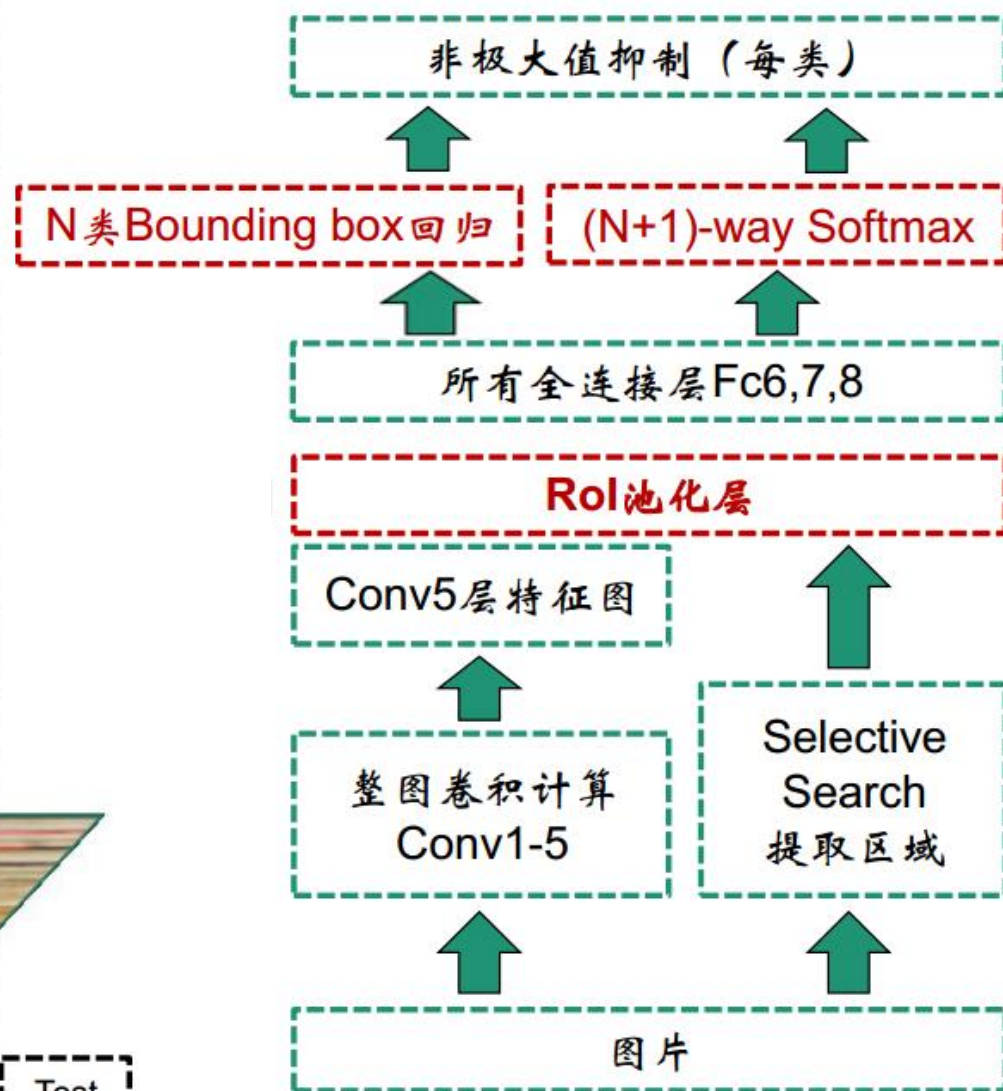
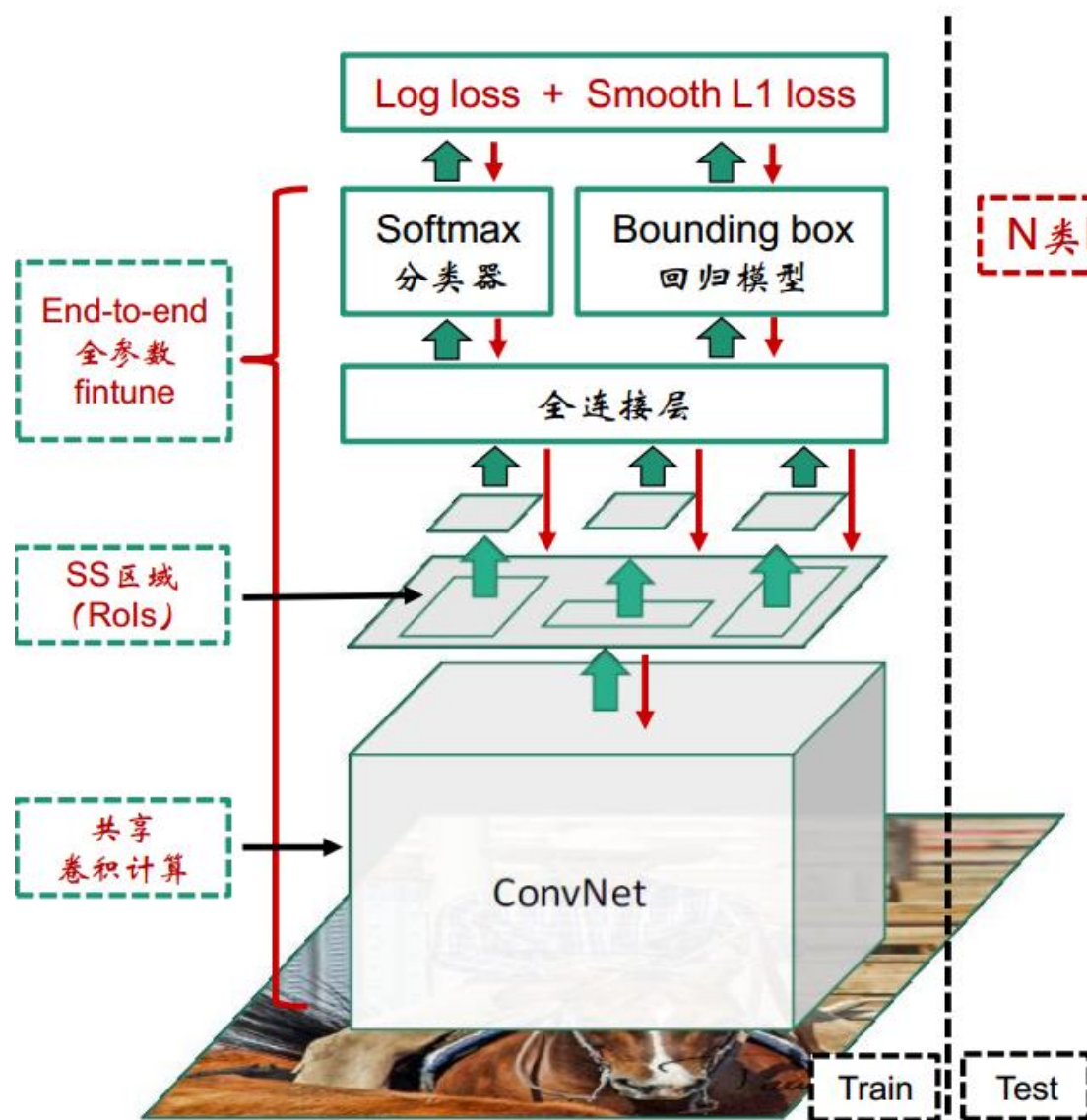
- ⌚ RoI分类基于和Ground Truth的重叠

- ⌚ 物体/正例：IoU ≥ 0.5

- ⌚ 背景/负例：IoU < 0.5



Fast R-CNN



Fast R-CNN

⌚ 为什么使用Softmax替换SVM分类器？

⌚ 因为在R-CNN中，微调操作是一个独立的分类损失函数，模型的求解是一个独立的过程，所以求解的模型容易过拟合，故需要单独训练一个SVM分类器来进行明确的分类操作。

⌚ 而在Fast R-CNN中，由于损失函数是由两部分构成的：分类损失和回归损失；这相当于在softmax中引入了类间竞争，效果会更好，另外论文中测试效果如下：

method	classifier	S	M	L
R-CNN [9, 10]	SVM	58.5	60.2	66.0
FRCN [ours]	SVM	56.3	58.7	66.8
FRCN [ours]	softmax	57.1	59.2	66.9

Table 8. Fast R-CNN with softmax vs. SVM (VOC07 mAP).

Fast R-CNN

⌚ 如何处理尺度不变性问题？也就是不同大小的图像输入网络也可以正确的被识别。

⌚ brute-force learning (single scale):

⌚ 使用某个给定scale对不同的数据进行训练预测，以期望网络能够自动学习尺度不变形特点。论文建议scale=600, 长边限制在1000以内，等比例缩放；

⌚ image pyramids (multi-scale):

⌚ 训练阶段随机从图像金字塔(建议scale={480,576,688,864,1200}, 长边限制2000以内，等比例缩放)中采样图像，预测的时候缩放为图像金字塔中最相似的尺寸进行预测。

	SPPnet ZF		S		M		L
scales	1	5	1	5	1	5	1
test rate (s/im)	0.14	0.38	0.10	0.39	0.15	0.64	0.32
VOC07 mAP	58.0	59.2	57.1	58.4	59.2	60.7	66.9

Fast R-CNN

method	train set	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	persn	plant	sheep	sofa	train	tv	mAP
SPPnet BB [11] [†]	07 \ diff	73.9	72.3	62.5	51.5	44.4	74.4	73.0	74.4	42.3	73.6	57.7	70.3	74.6	74.3	54.2	34.0	56.4	56.4	67.9	73.5	63.1
R-CNN BB [10]	07	73.4	77.0	63.4	45.4	44.6	75.1	78.1	79.8	40.5	73.7	62.2	79.4	78.1	73.1	64.2	35.6	66.8	67.2	70.4	71.1	66.0
FRCN [ours]	07	74.5	78.3	69.2	53.2	36.6	77.3	78.2	82.0	40.7	72.7	67.9	79.6	79.2	73.0	69.0	30.1	65.4	70.2	75.8	65.8	66.9
FRCN [ours]	07 \ diff	74.6	79.0	68.6	57.0	39.3	79.5	78.6	81.9	48.0	74.0	67.4	80.5	80.7	74.1	69.6	31.8	67.1	68.4	75.3	65.5	68.1
FRCN [ours]	07+12	77.0	78.1	69.3	59.4	38.3	81.6	78.6	86.7	42.8	78.8	68.9	84.7	82.0	76.6	69.9	31.8	70.1	74.8	80.4	70.4	70.0

Table 1. **VOC 2007 test** detection average precision (%). All methods use VGG16. Training set key: **07**: VOC07 trainval, **07 \ diff**: **07** without “difficult” examples, **07+12**: union of **07** and VOC12 trainval. [†]SPPnet results were prepared by the authors of [11].

	Fast R-CNN			R-CNN			SPPnet
	S	M	L	S	M	L	[†] L
train time (h)	1.2	2.0	9.5	22	28	84	25
train speedup	18.3×	14.0×	8.8×	1×	1×	1×	3.4×
test rate (s/im)	0.10	0.15	0.32	9.8	12.1	47.0	2.3
▷ with SVD	0.06	0.08	0.22	-	-	-	-
test speedup	98×	80×	146×	1×	1×	1×	20×
▷ with SVD	169×	150×	213×	-	-	-	-
VOC07 mAP	57.1	59.2	66.9	58.5	60.2	66.0	63.1
▷ with SVD	56.5	58.7	66.6	-	-	-	-

	layers that are fine-tuned in model L			SPPnet L
	≥ fc6	≥ conv3_1	≥ conv2_1	≥ fc6
VOC07 mAP	61.4	66.9	67.2	63.1
test rate (s/im)	0.32	0.32	0.32	2.3

卷积神经网络典型CNN-VGGNet

⌚ VGG Net

INPUT: [224x224x3]
CONV3-64: [224x224x64]
CONV3-64: [224x224x64]
POOL2: [112x112x64]
CONV3-128: [112x112x128]
CONV3-128: [112x112x128]
POOL2: [56x56x128]
CONV3-256: [56x56x256]
CONV3-256: [56x56x256]
CONV3-256: [56x56x256]
POOL2: [28x28x256]
CONV3-512: [28x28x512]
CONV3-512: [28x28x512]
CONV3-512: [28x28x512]
POOL2: [14x14x512]
CONV3-512: [14x14x512]
CONV3-512: [14x14x512]
CONV3-512: [14x14x512]
POOL2: [7x7x512]
FC: [1x1x4096]
FC: [1x1x4096]
FC: [1x1x1000]

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

最优模式

VGG-Net结构

Fast R-CNN

- ⌚ 基本解决了R-CNN的绝大多数问题，但是还是有一些问题没有解决，具体如下：
 - ⌚ 解决了SPP Net网络中的fine tune的问题(局部训练);
 - ⌚ 解决了R-CNN网络中各阶段独立模型训练的问题(特征提取、SVM分类器、BBR回归器);
 - ⌚ 使用Full Image的方式一次性提取图像的Feature Map，降低了提取RoI区域特征属性的运行时间;
 - ⌚ 引入RoI Pooling Layer层，对于不同大小的RoI候选框产生相同维度大小的特征属性向量。
- ⌚ **Fast R-CNN中的主要问题就是使用SS产生候选区域块ROI区域在整个检测过程中耗时太长(SS候选框获取2~3s，特征分类0.3s)。**

Faster R-CNN

- ⌚ 集成**Region Proposal Network(RPN)**网络
 - ⌚ $\text{Faster R-CNN} = \text{Fast R-CNN} + \text{RPN}$
 - ⌚ 使用**RPN**网络取代**外部的候选框生成(eg: Selective Search)**模块
 - ⌚ **解决Fast R-CNN的性能瓶颈**
 - ⌚ 候选框生成网络RPN和检测网络Fast R-CNN**共享卷积层计算**
 - ⌚ Region proposals量少质优(~300)
 - ⌚ 利用NMS去除低质量的候选框
 - ⌚ 高precision、高recall

Faster R-CNN

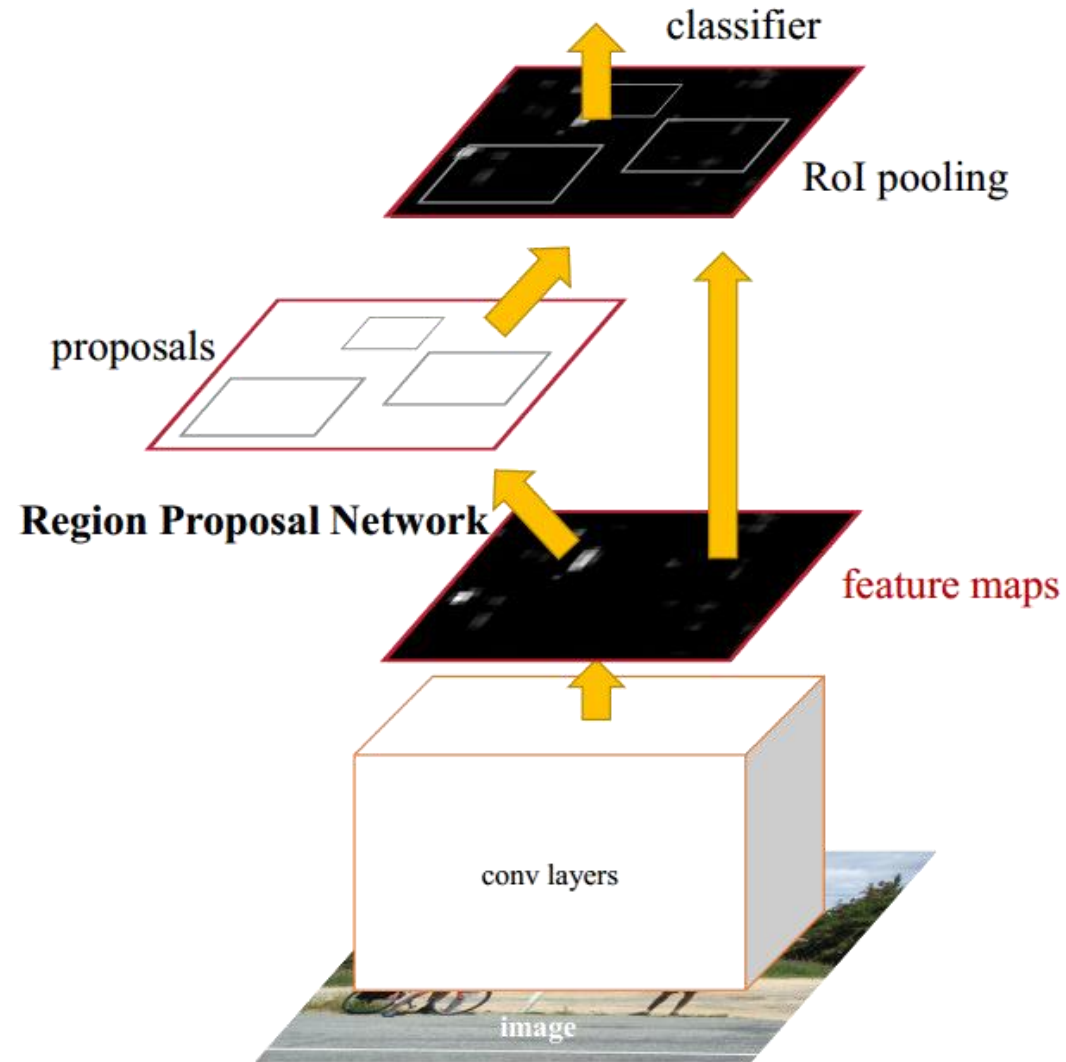
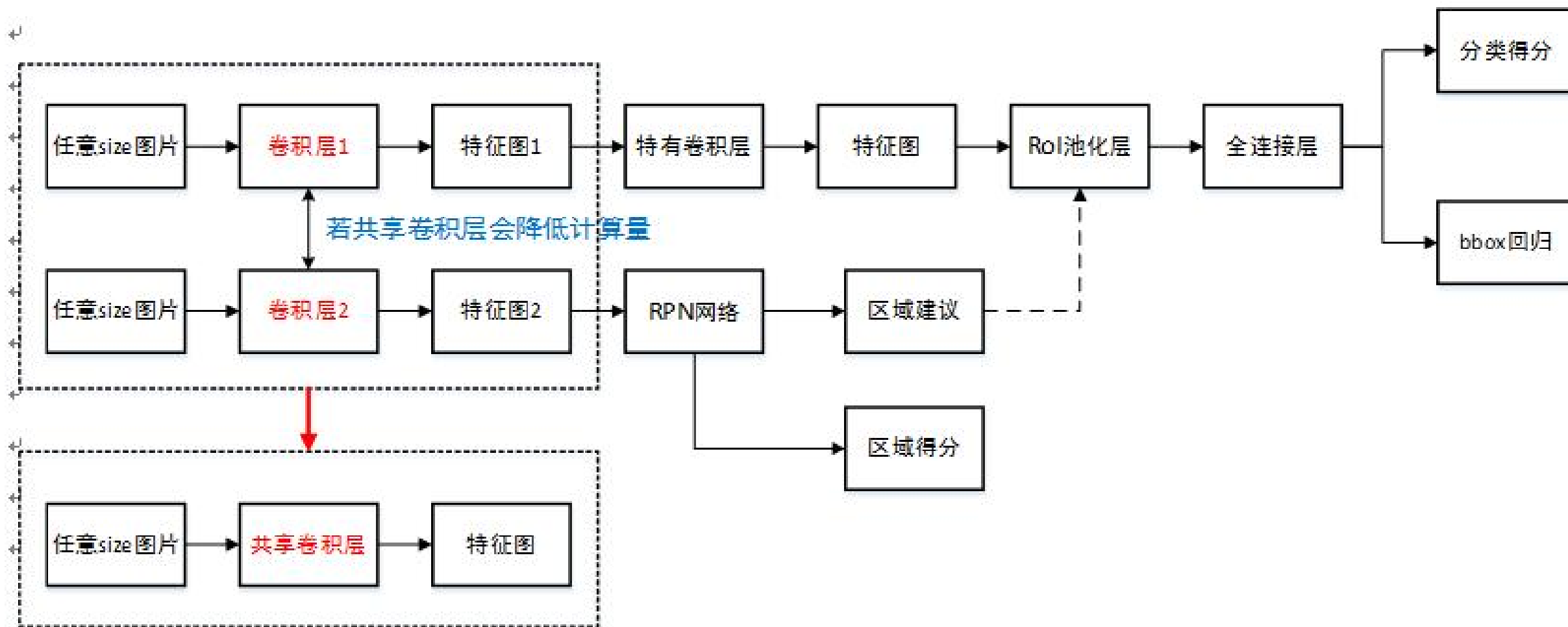
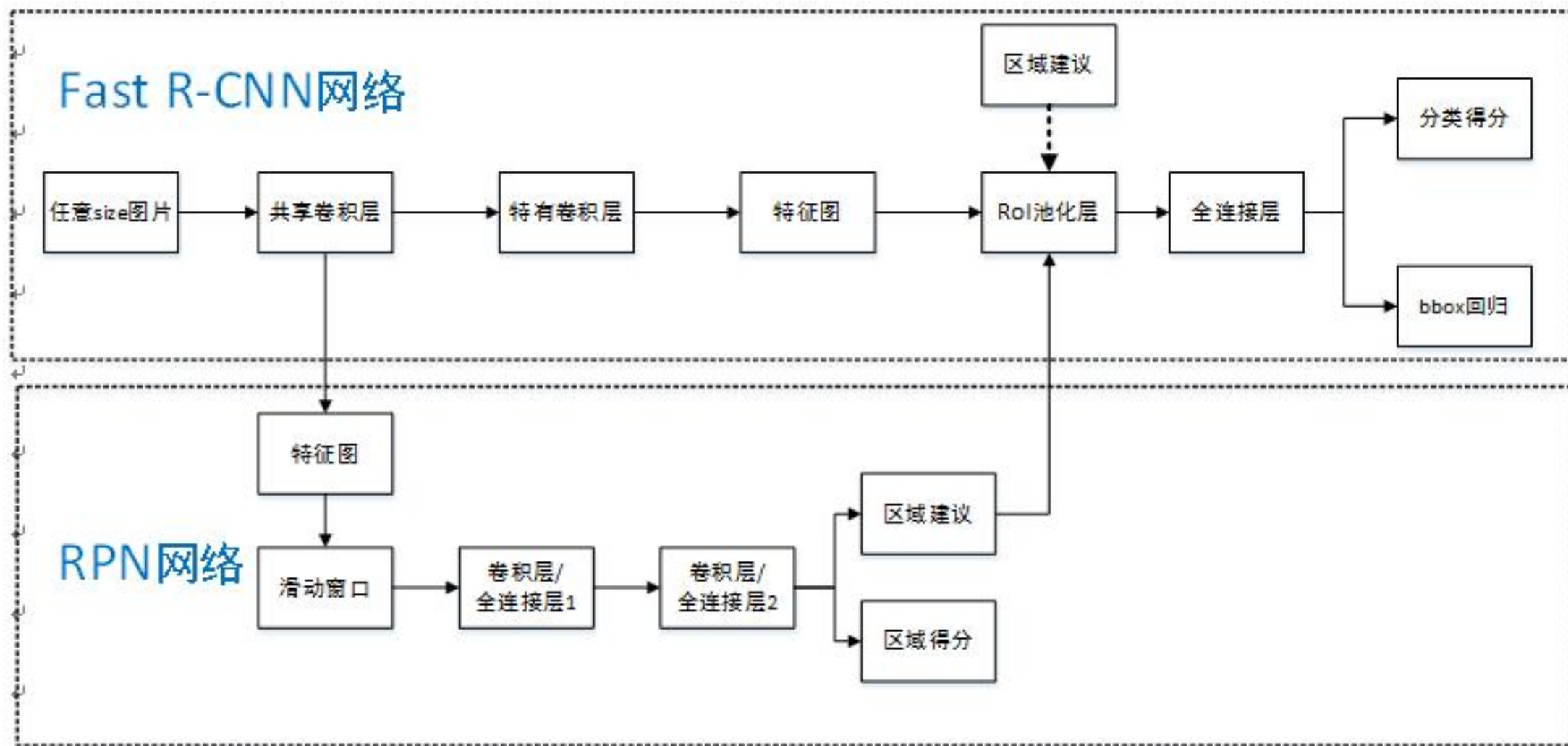


Figure 2: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the 'attention' of this unified network.

Faster R-CNN



Faster R-CNN



Fast R-CNN和RPN理解

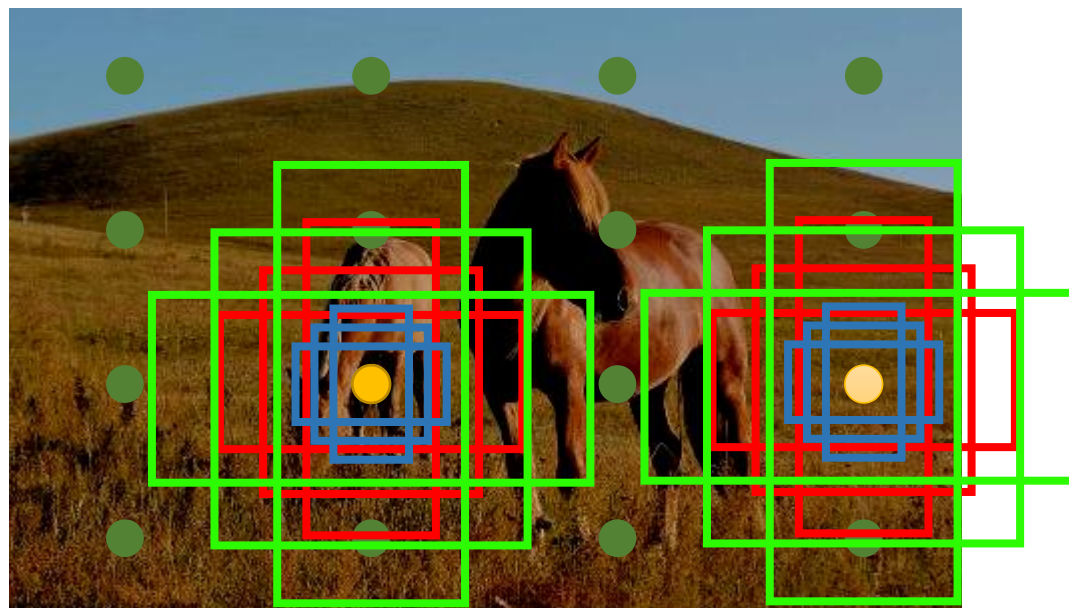
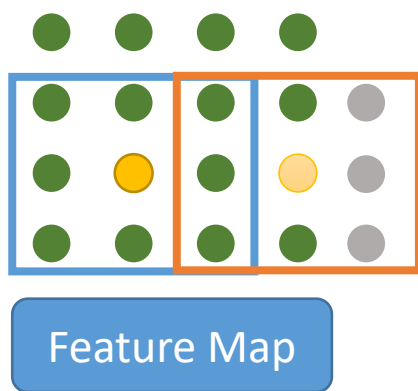
⌚ Fast R-CNN:

- ⌚ 结构简单理解：基于给定的**候选框**，通过一个深度学习网络，判断这个候选框里面是否有物体以及对应物体的类型信息(分类分支)；如果有确定的类别物体，那么该候选框和真实边框之间的偏移信息(回归分支)
- ⌚ **候选框使用外部算法(eg:SS)生成**；而且每个候选框大小不定，所以需要使用RoI池化；**结果输出就是最终预测边框**；

⌚ RPN:

- ⌚ 结构简单理解：基于给定的**初始框/Anchor Box**，通过一个深度学习网络，判断这个初始框里面是否有物体(分类分支)；如果有物体，那么该初始框和真实边框之间的偏移信息(回归分支)
- ⌚ **初始框使用规则方式生成**；**结果输出是下一阶段算法(Fast R-CNN)的输入候选框**；

Faster R-CNN RPN



原始图像

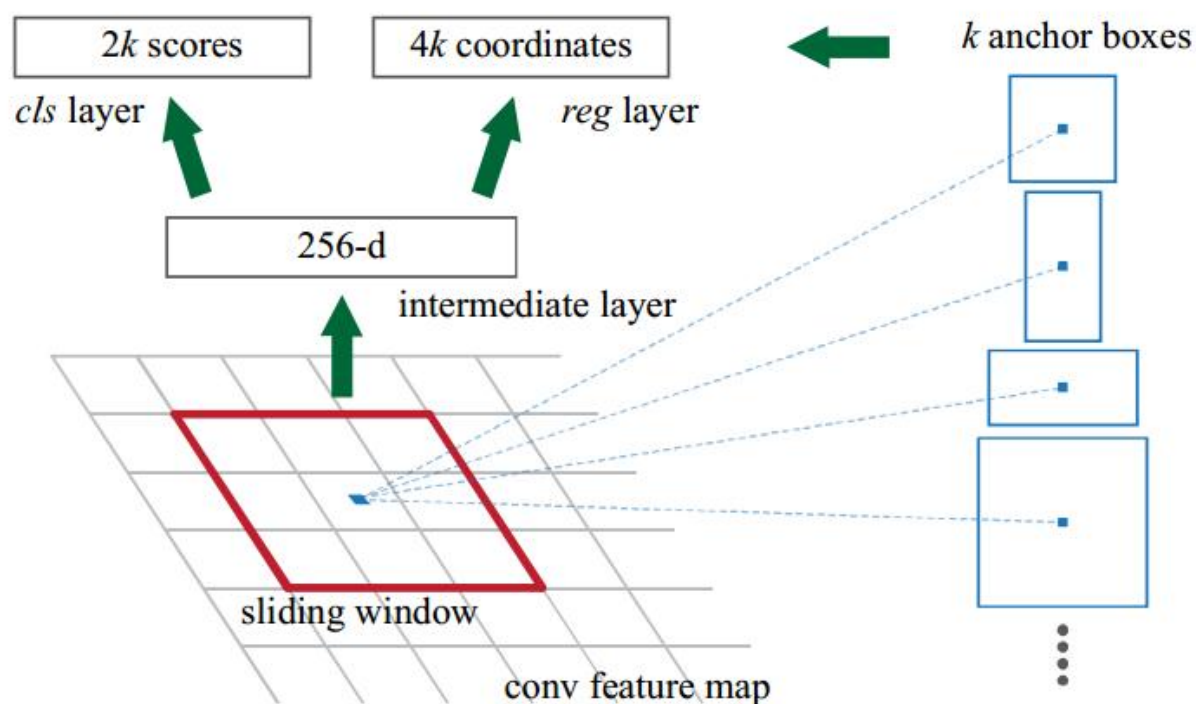
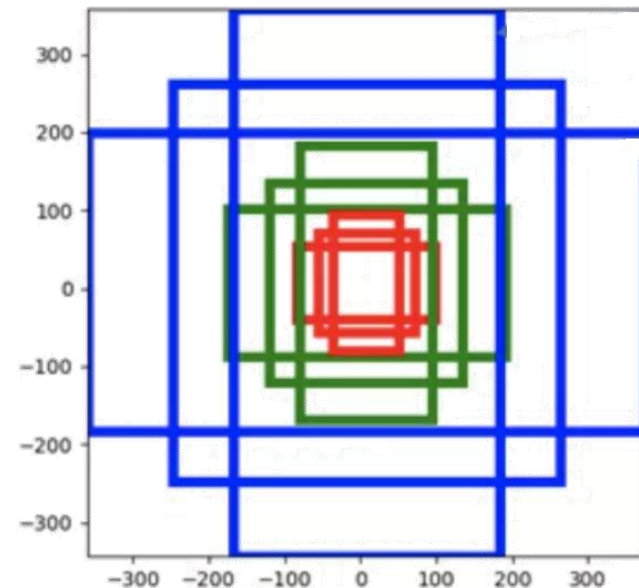
Faster R-CNN RPN

⌚ 一种全卷积网络(Fully Convolutional Network)

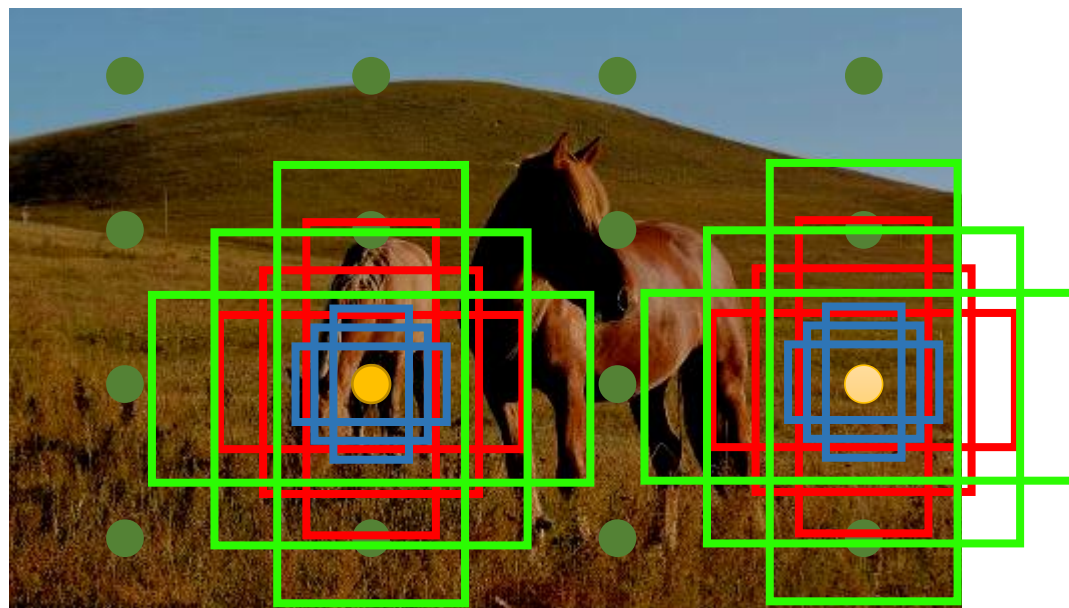
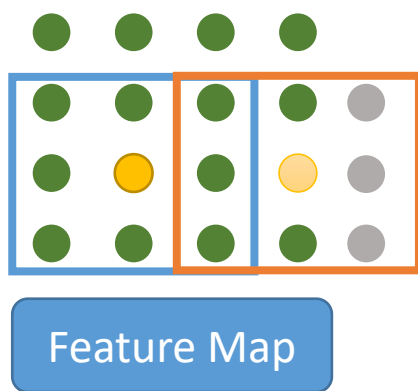
- ⌚ 输入图片的Conv5的特征($W*H$) --> $3*3/1$, 256-d卷积层
- ⌚ $1*1/1$, 4k-d卷积层 --> 输出k组proposal的offsets变量(x,y,w,h)
- ⌚ $1*1/1$, 2k-d卷积层 --> 输出k组置信值(object score, non-object score)

⌚ 参考框Anchor box

- ⌚ $k=9$, 每个区域获取k个anchor
- ⌚ 3个尺度scale, (128,256,512)
- ⌚ 3给宽高比ratio
 - ⌚ 1:1, 1:2, 2:1
- ⌚ Anchor Box数量: $W*H*k$

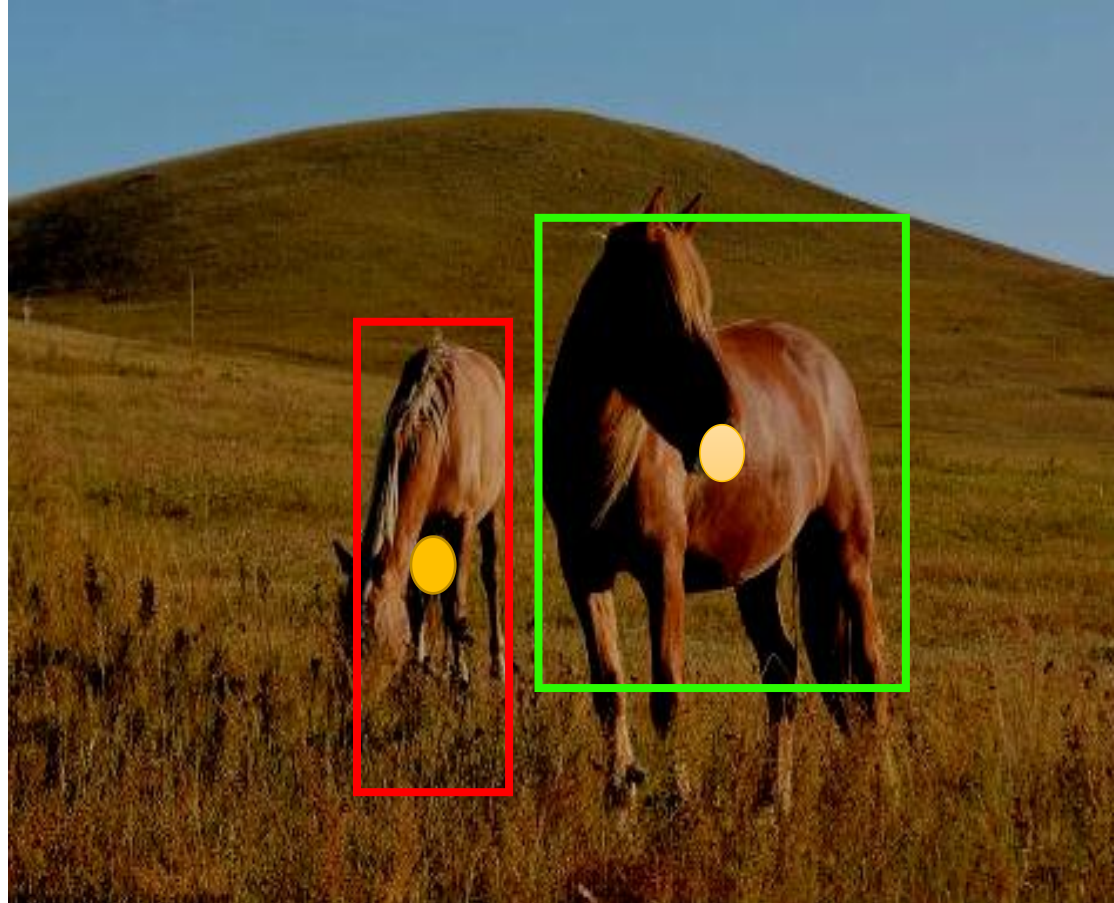


Faster R-CNN RPN



原始图像

Faster R-CNN RPN



Faster R-CNN RPN

- ⌚ RPN实际上是Faster R-CNN中的一个小的神经网络，通过这个网络来生成候选区域框ROI;
- ⌚ 通过 $n \times n$ (论文中为 3×3)的卷积对通用卷积层的输出feature map来提取当前区域256维的高阶特征，然后基于这个高阶特征来分别构建两个分支，第一个分支cls layer分类层的主要功能是判断reference boxes/anchor box是前景物体还是背景，第二个分支reg layer回归层的主要功能是对于reference boxes/anchor box的坐标和候选框(Region Proposal Box)之间的坐标之间的线性转换关系做一个预测(类似R-CNN的回归预测结构)。

Faster R-CNN RPN

⌚ Anchor: 锚点, 对于一个sliding window而言, 可以产生 k 个原始proposal候选框, 即 k 个reference boxes, 每个reference boxes可以用一个scale、一个aspect_ratio和一个sliding window中的锚点来唯一确定。对于 $W \times H$ 的feature map而言, 总共有 $W \times H \times k$ 个reference boxes。反向映射到原始图像上, 相当于是以对应锚点为中心的 k 个候选框, 所以可以认为Anchor box是初步候选框; 并且在feature map上进行sliding window的滑动, 相当于让候选框出现在原始图像上的任意位置, 并且通过scale和aspect_ratio的操作, 让Anchor产生的候选框具有**尺度不变性**的特点。

Faster R-CNN RPN

⌚ Anchor box:

$$(x_a, y_a, w_a, h_a)$$

Center Coordinates: 区域框的中心坐标

⌚ Predicted box (RPN产生的RoI区域框):

$$(x, y, w, h)$$

Width and Height: 区域框的宽度和高度

⌚ Ground truth box:

$$(x^*, y^*, w^*, h^*)$$

Faster R-CNN RPN

- ⌚ 覆盖到feature map边界上的Anchor Boxes不参与RPN网络的训练;
- ⌚ 正样本定义规则:
 - ⌚ -1. 和Ground Truth Box的IoU重叠区域比最高的anchor boxes;
 - ⌚ -2. 和Ground Truth Box的IoU重叠区域比大于0.7的anchor boxes;
- ⌚ 负样本定义规则:
 - ⌚ 和所有的Ground Truth Box的IoU重叠区域比小于0.3的anchor boxes;
- ⌚ 训练RPN的损失函数有softmax loss和smooth l1 loss两部分组成, 类似Fast R-CNN。

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* \cdot L_{reg}(t_i, t_i^*)$$

Faster R-CNN RPN

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* \cdot L_{reg}(t_i, t_i^*)$$

⌚ Softmax Loss的计算需要anchors对应的ground truth标定结果和预测结果， p_i 表示第*i*个anchor box属于正例的概率值(包含object)，当第*i*个anchor box实际标注为正例，那么 p_i^* 为1，否则为0。其损失函数为基本的log loss损失。

$$L_{cls}(p_i, p_i^*) = -\left(p_i^* \ln(p_i) + (1 - p_i^*) \ln(1 - p_i)\right)$$

Faster R-CNN RPN

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* \cdot L_{reg}(t_i, t_i^*)$$

⌚ 对于回归部分，仅仅对正例anchors进行回归预测并产生损失值，采用 Smoth L1 Loss进行模型构建。在bounding box regression中，采用 anchors作为基准，期望通过BBR转换后的候选框基本上都是位于Ground Truth Box实际框附近的。

$$L_{reg}(t_i, t_i^*) = Smoth_L1_Loss(t_i - t_i^*)$$

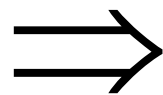
$$\begin{aligned} t_x &= (x - x_a) / w_a, t_y = (y - y_a) / h_a & t_x^* &= (x^* - x_a) / w_a, t_y^* = (y^* - y_a) / h_a \\ t_w &= \log(w / w_a), t_h = \log(h / h_a) & t_w^* &= \log(w^* / w_a), t_h^* = \log(h^* / h_a) \end{aligned}$$

Faster R-CNN RPN

⌚ 当我们RPN网络训练好后，候选框RoI的坐标为：

$$\begin{cases} x = t_x \cdot w_a + x_a \\ y = t_y \cdot h_a + y_a \\ w = e^{t_w} \cdot w_a \\ h = e^{t_h} \cdot h_a \end{cases}$$

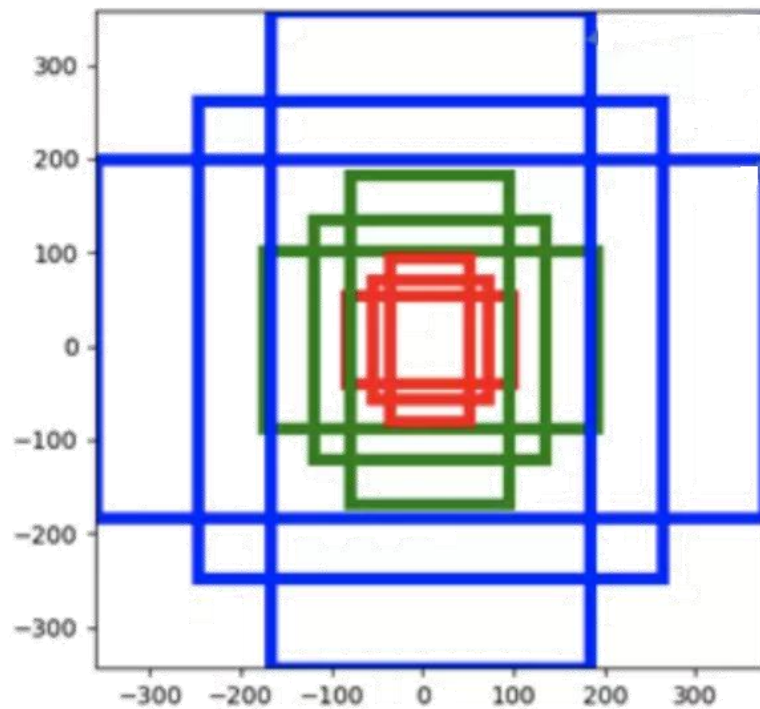
中心点坐标值+宽高值



$$\begin{cases} x_1 = x - 0.5 \cdot w \\ y_1 = y - 0.5 \cdot h \\ x_2 = x + 0.5 \cdot w \\ y_2 = y + 0.5 \cdot h \end{cases}$$

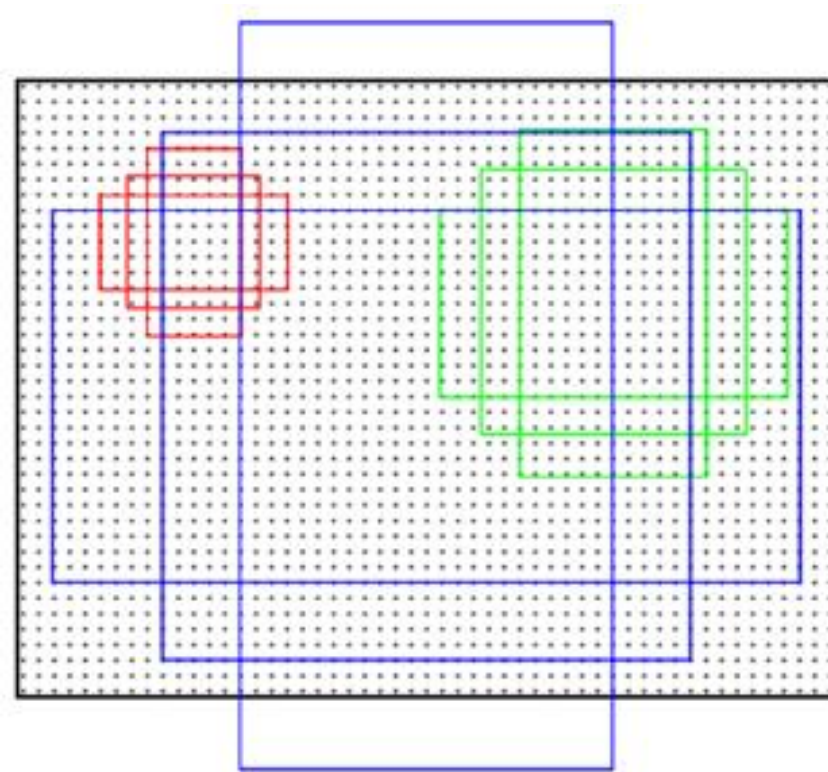
左上角+右下角坐标值

Faster R-CNN RPN



Anchor Boxes

回归转换



RoI

Faster R-CNN

- ⌚ 在RPN网络中，通过**RPN Classification**可以识别出**那些区域是前景物体、那些区域是背景**，通过**RPN bounding box regression**可以获取**前景区域的大致坐标(修正后的坐标)**。
- ⌚ 在RPN提取的RoI候选框区域后，类似Fast R-CNN，做一个Proposal Layer部分的结构，对区域位置做一个精准的位置回归，由于RPN中计算出来的 $H*W*K$ 个anchor box大多数是聚集在Ground Truth Box周围的候选框，所以在进行Proposal Layer部分的训练的时候，进行了RoI候选框的过滤操作(采用NMS非极大值抑制)。

Faster R-CNN

- ⌚ 对于RPN产生的候选框主要采用如下操作：
 - ⌚ 1. 移除超出边界的anchors(在图像内的anchor box区域宽度、高度低于最小区域大小的情况);
 - ⌚ 2. 选择出前景概率最大的N个proposal(论文中建议12000训练/6000预测);
 - ⌚ 3. 使用非极大值抑制(NMS)对候选框进行过滤(IoU阈值0.7);
 - ⌚ 4. 对NMS结果再做Top M的操作, 获取概率最大的M个proposal(论文中建议2000训练/300预测);
 - ⌚ 5. 将ROI和Ground Truth的IoU区域比在 $[0.5, 1]$ 的anchor当做正样本, 将IoU在 $[0.1, 0.5)$ 之间的anchor当做负样本, Bounding box regression仅使用正样本进行模型训练 (Proposal Layer网络中的训练)。

Faster R-CNN

- ⌚ 对于RPN网络和Proposal Layer两个不同网络结构的训练，其Faster R-CNN中提出了三种方式：
 - ⌚ **Alternating training**：类似Fast R-CNN, 交叉的训练RPN和Fast R-CNN;
 - ⌚ **Approximate joint training**：将RPN和Fast R-CNN合并成为一个网络结构，也就是相当于将RPN网络的loss和Fast R-CNN网络的loss合并成为一个loss值。这里的Approximate的意思是对于RPN bounding box regression部分的反向传播计算得到的梯度舍弃；
 - ⌚ **Non-Approximate joint training**：同Approximate joint training，区别在于不会舍弃RPN bounding box regression部分的梯度值，是将四个loss值累加起来来进行模型优化的。

Faster R-CNN

⌚ 4步 **Alternating training** 训练过程:

⌚ Step1 - 训练RPN网络

- ⌚ Initialized Convolutional with ImageNet Pre-trained model.
- ⌚ Initialize all new layers with Gaussian distribution(mean is 0.0, standard deviation is 0.01).
- ⌚ Batch size 256, One Image, 128 Positive Anchor; 128 Negative Anchor.
- ⌚ SGD, batch<60k --> learning rate is 0.001, next batch learning rate is 0.0001.

⌚ Step2 - 训练Fast R-CNN网络

- ⌚ Initialized Convolutional with ImageNet Pre-trained model.
- ⌚ Step1 RPN generate Region Proposals.
- ⌚ Change Convolutional kernel weights(fine-tune).

卷积
不共享

Faster R-CNN

⌚ 4步 **Alternating training** 训练过程:

⌚ Step3 - 调优RPN网络

- ⌚ Initialized Convolutional with Step2 Fast R-CNN;

- ⌚ fix the shared convolutional layers and fine-tune the RPN layers.

⌚ Step4 - 调优Fast R-CNN网络

- ⌚ fix the shard convolutional layers and fine-tune the unique layers of Fast R-CNN.

- ⌚ Note: 可以考虑多次重复, 但是实际上效果没有太大的提升。

卷积
共享

Faster R-CNN

⌚ **Approximate joint training**和**Non-Approximate joint training**:

- ⌚ RPN和Fast R-CNN网络中完全采用相同的损失函数。
- ⌚ 直接进行模型训练即可。

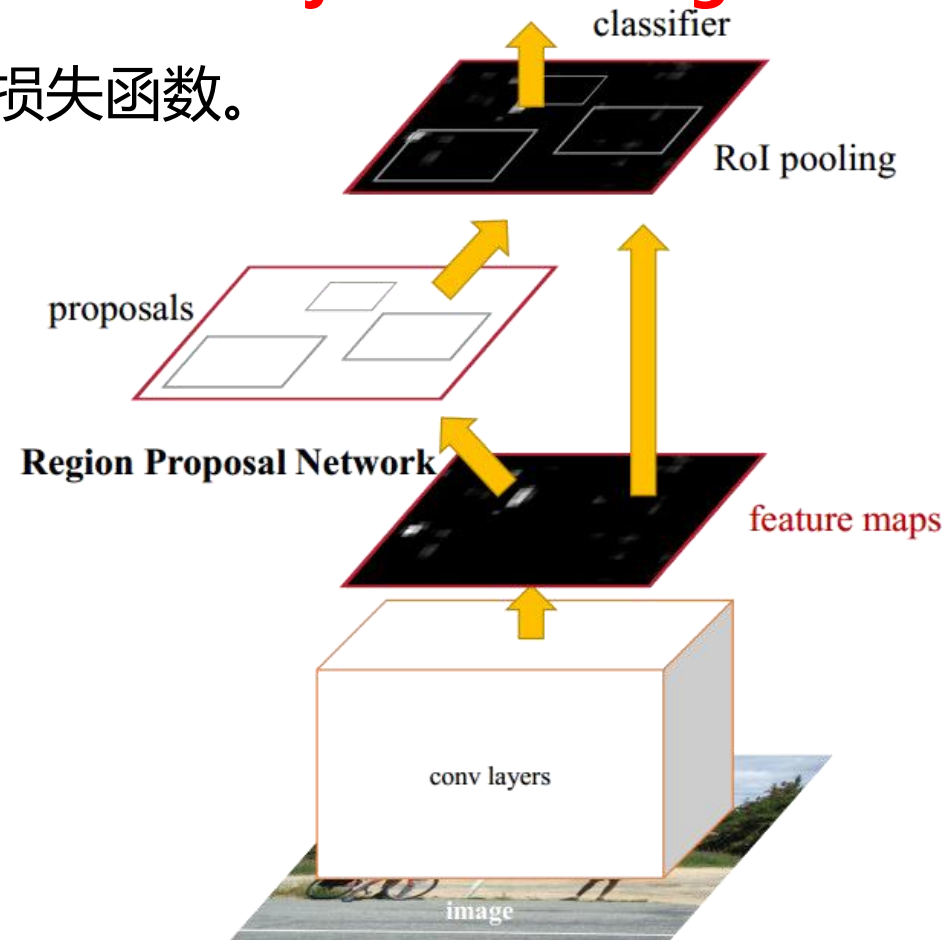


Figure 2: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the 'attention' of this unified network.

Faster R-CNN

⌚ 数据图像大小要求:

⌚ 单一尺度训练;

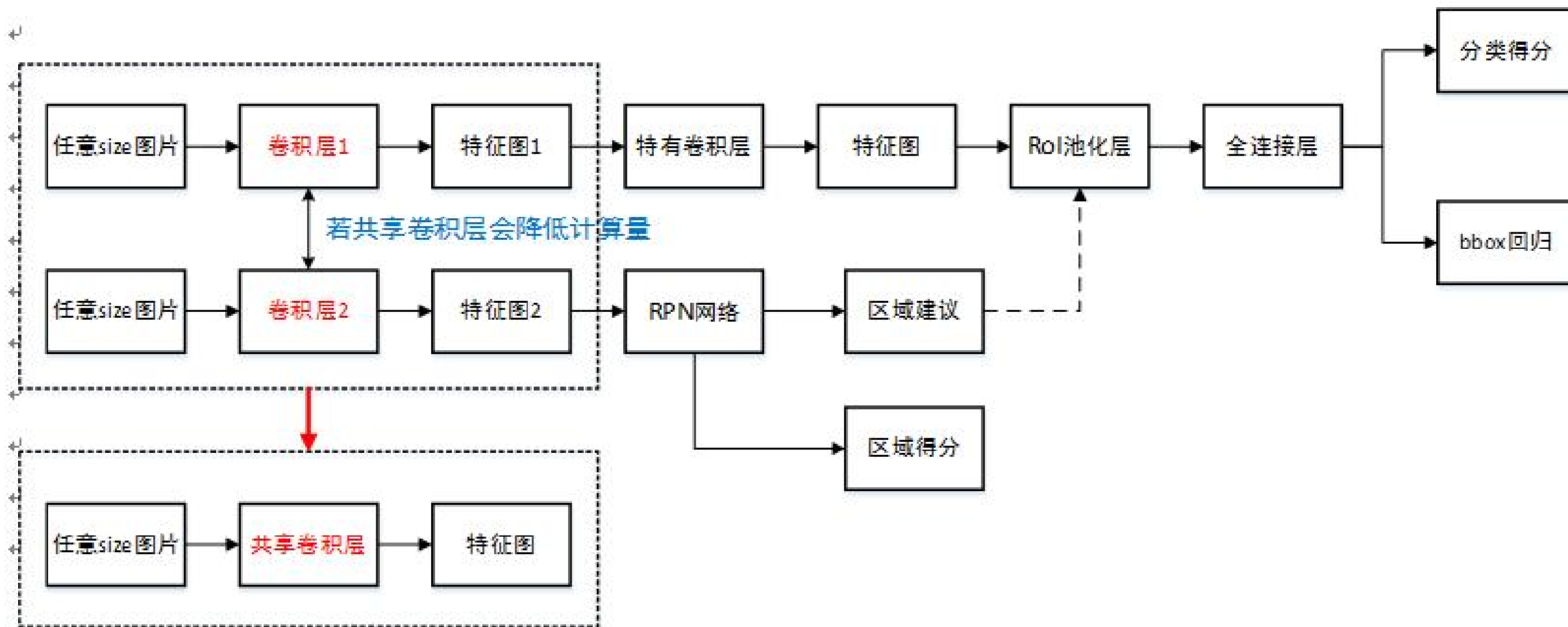
⌚ 要求输入的图像短边至少为600像素;

⌚ Anchor boxes还原到原始图像上后, 可以包含所有区域, 当600像素的情况下。

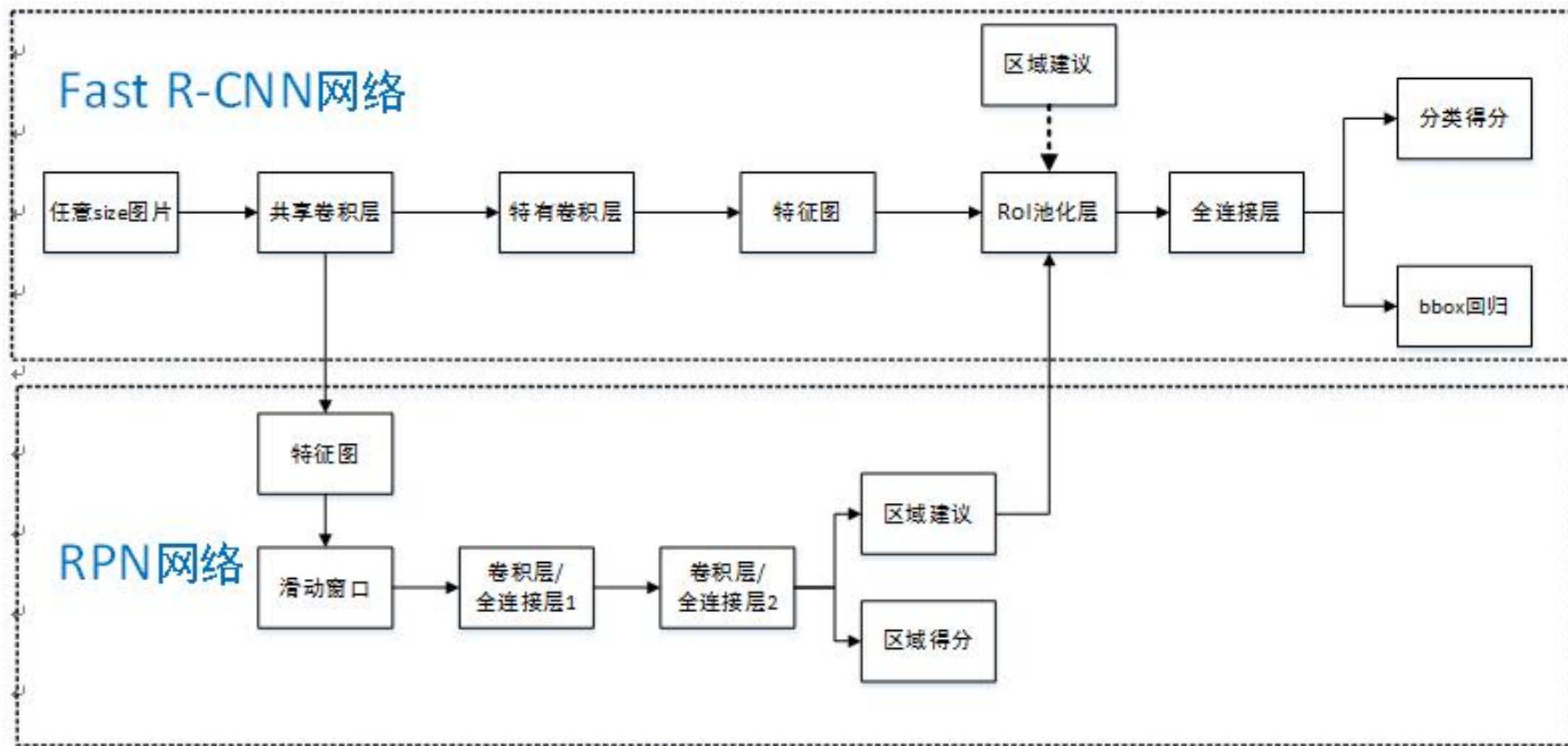
Faster R-CNN

- ⌚ 如果处理多尺度问题？即如何使 224×224 和 1080×720 的车辆同时在一个训练好的网络中都能正确识别？
 - ⌚ 图像金字塔：对伸缩到不同size的输入图像进行特征提取，有效但是费时；
 - ⌚ 滤波器金字塔或者滑动窗口金字塔：对输入图像采用不同的size大小的滤波器进行卷积操作，需要枚举比较麻烦；
 - ⌚ Anchors金字塔：在RPN网络中对feature map进行特征图滑窗，对滑窗中心进行多尺度、多宽高比的采样，并对采样结果进行回归、分类调整，从而达到仅依赖单一尺度图像、特征图以及卷积核，解决了多尺度问题。
 - ⌚ 相比于传统的窗口滑动(需要改动卷积核的尺寸大小)，Anchors是在feature map上做滑窗，计算量更少。

Faster R-CNN



Faster R-CNN



Faster R-CNN

method	# proposals	data	mAP (%)
SS	2000	07	66.9 [†]
SS	2000	07+12	70.0
RPN+VGG, unshared	300	07	68.5
RPN+VGG, shared	300	07	69.9
RPN+VGG, shared	300	07+12	73.2
RPN+VGG, shared	300	COCO+07+12	78.8

Table 6: Results on PASCAL VOC 2007 test set with Fast R-CNN detectors and VGG-16. For RPN, the train-time proposals for Fast R-CNN are 2000. RPN* denotes the unsharing feature version.

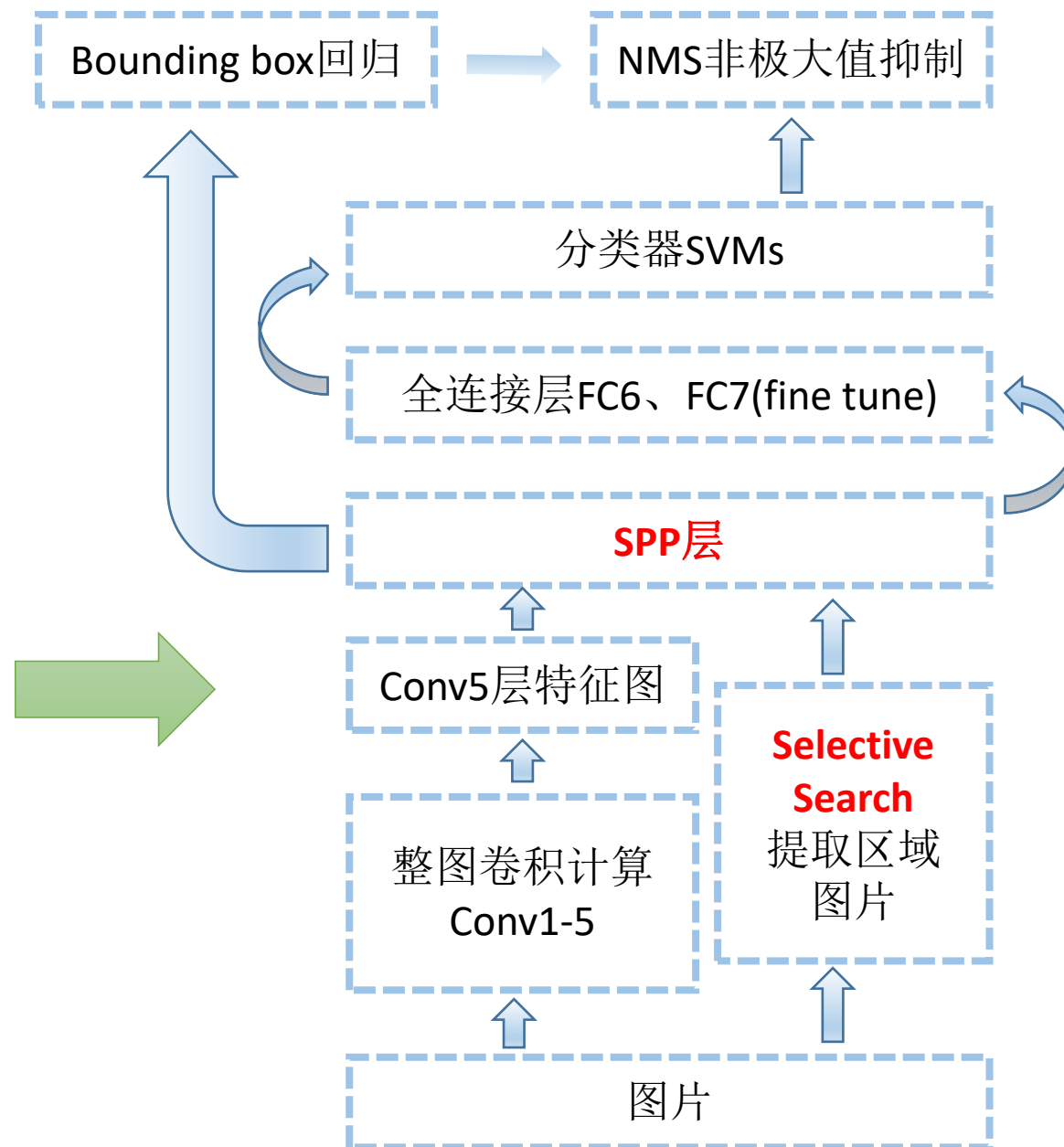
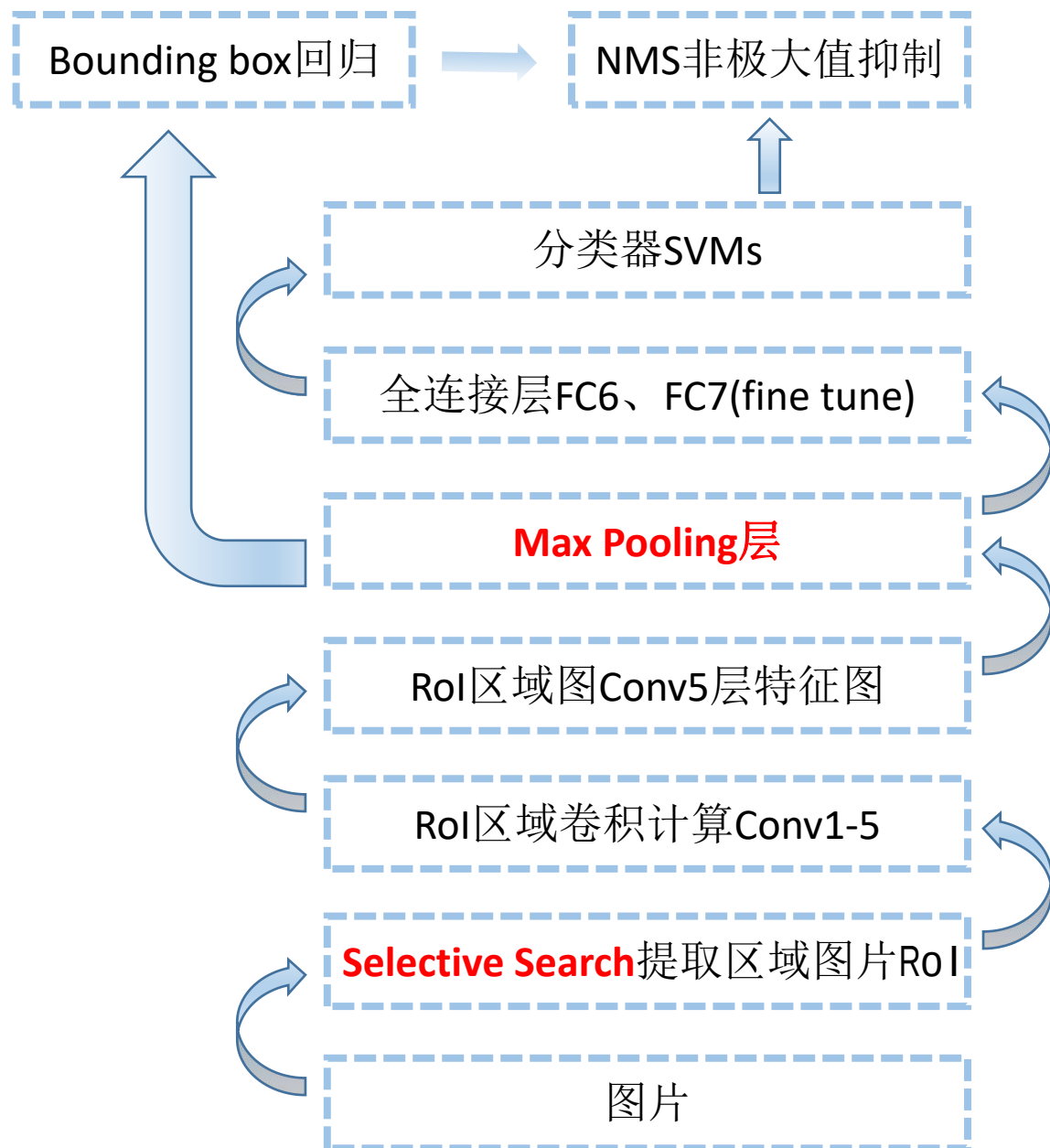
method	# box	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
SS	2000	07	66.9	74.5	78.3	69.2	53.2	36.6	77.3	78.2	82.0	40.7	72.7	67.9	79.6	79.2	73.0	69.0	30.1	65.4	70.2	75.8	65.8
SS	2000	07+12	70.0	77.0	78.1	69.3	59.4	38.3	81.6	78.6	86.7	42.8	78.8	68.9	84.7	82.0	76.6	69.9	31.8	70.1	74.8	80.4	70.4
RPN*	300	07	68.5	74.1	77.2	67.7	53.9	51.0	75.1	79.2	78.9	50.7	78.0	61.1	79.1	81.9	72.2	75.9	37.2	71.4	62.5	77.4	66.4
RPN	300	07	69.9	70.0	80.6	70.1	57.3	49.9	78.2	80.4	82.0	52.2	75.3	67.2	80.3	79.8	75.0	76.3	39.1	68.3	67.3	81.1	67.6
RPN	300	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
RPN	300	COCO+07+12	78.8	84.3	82.0	77.7	68.9	65.7	88.1	88.4	88.9	63.6	86.3	70.8	85.9	87.6	80.1	82.3	53.6	80.4	75.8	86.6	78.9

Table 5: **Timing** (ms) on a K40 GPU, except SS proposal is evaluated in a CPU. “Region-wise” includes NMS, pooling, fully-connected, and softmax layers. See our released code for the profiling of running time.

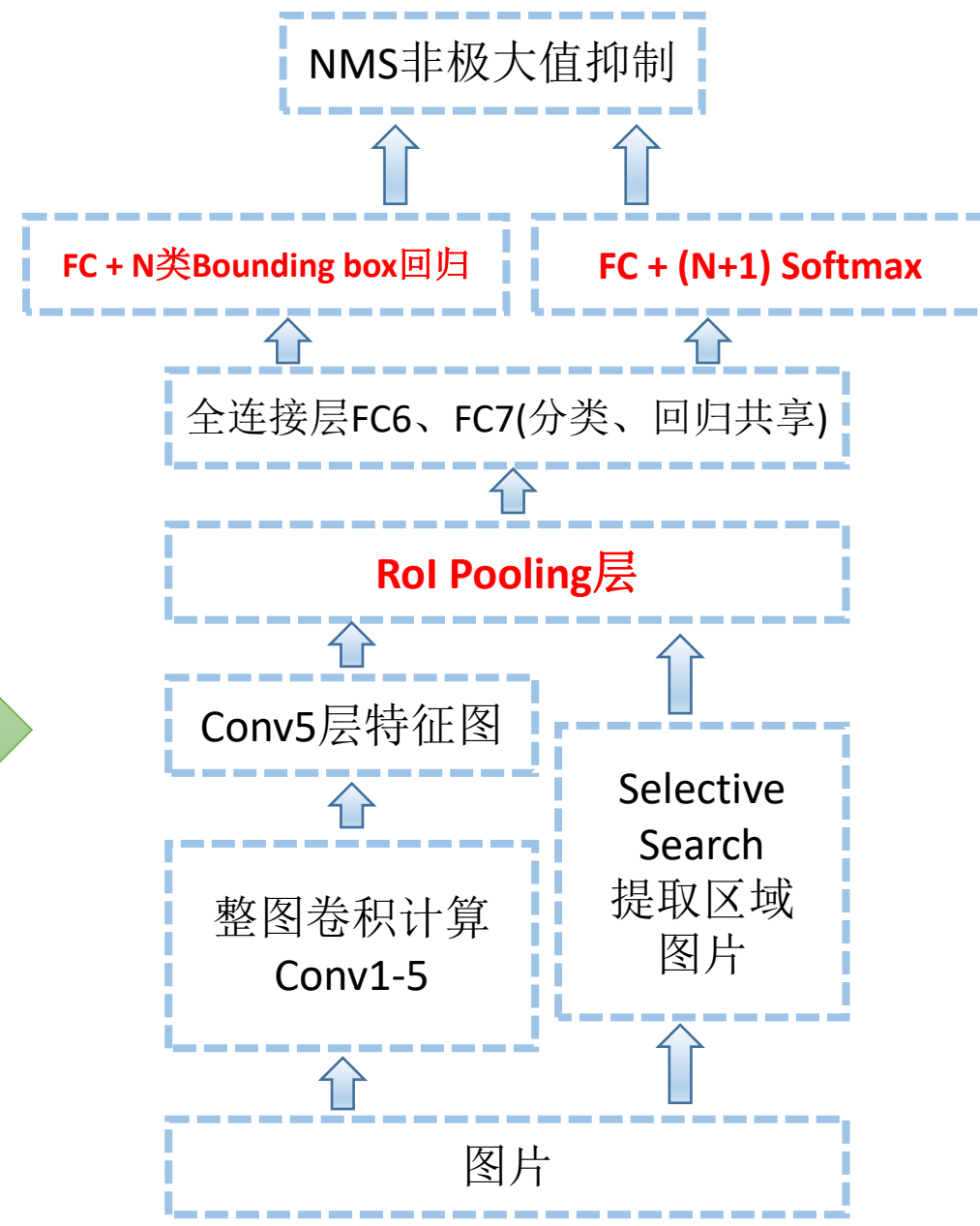
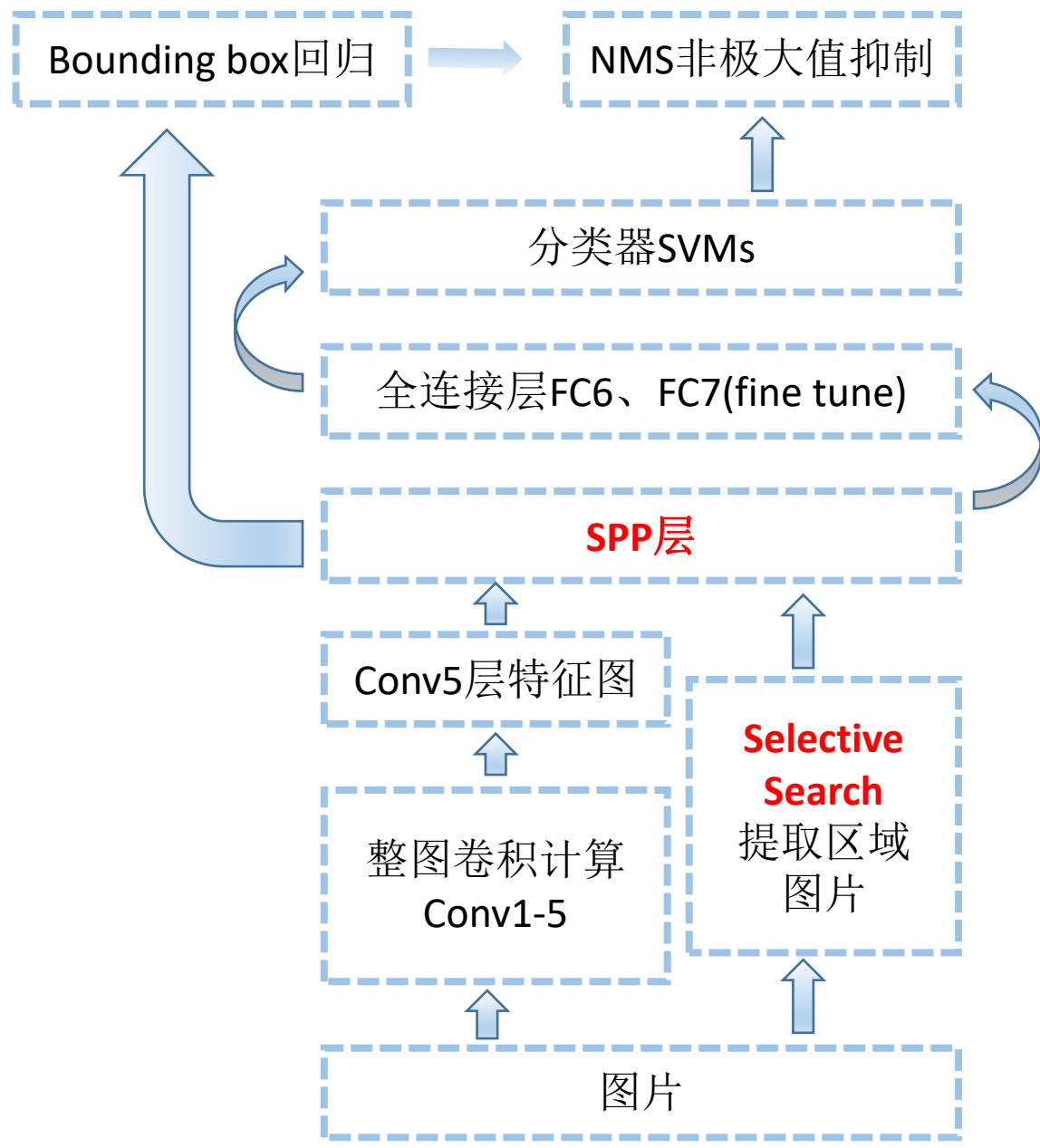
model	system	conv	proposal	region-wise	total	rate
VGG	SS + Fast R-CNN	146	1510	174	1830	0.5 fps
VGG	RPN + Fast R-CNN	141	10	47	198	5 fps
ZF	RPN + Fast R-CNN	31	3	25	59	17 fps

17fps:每秒预测17张图像

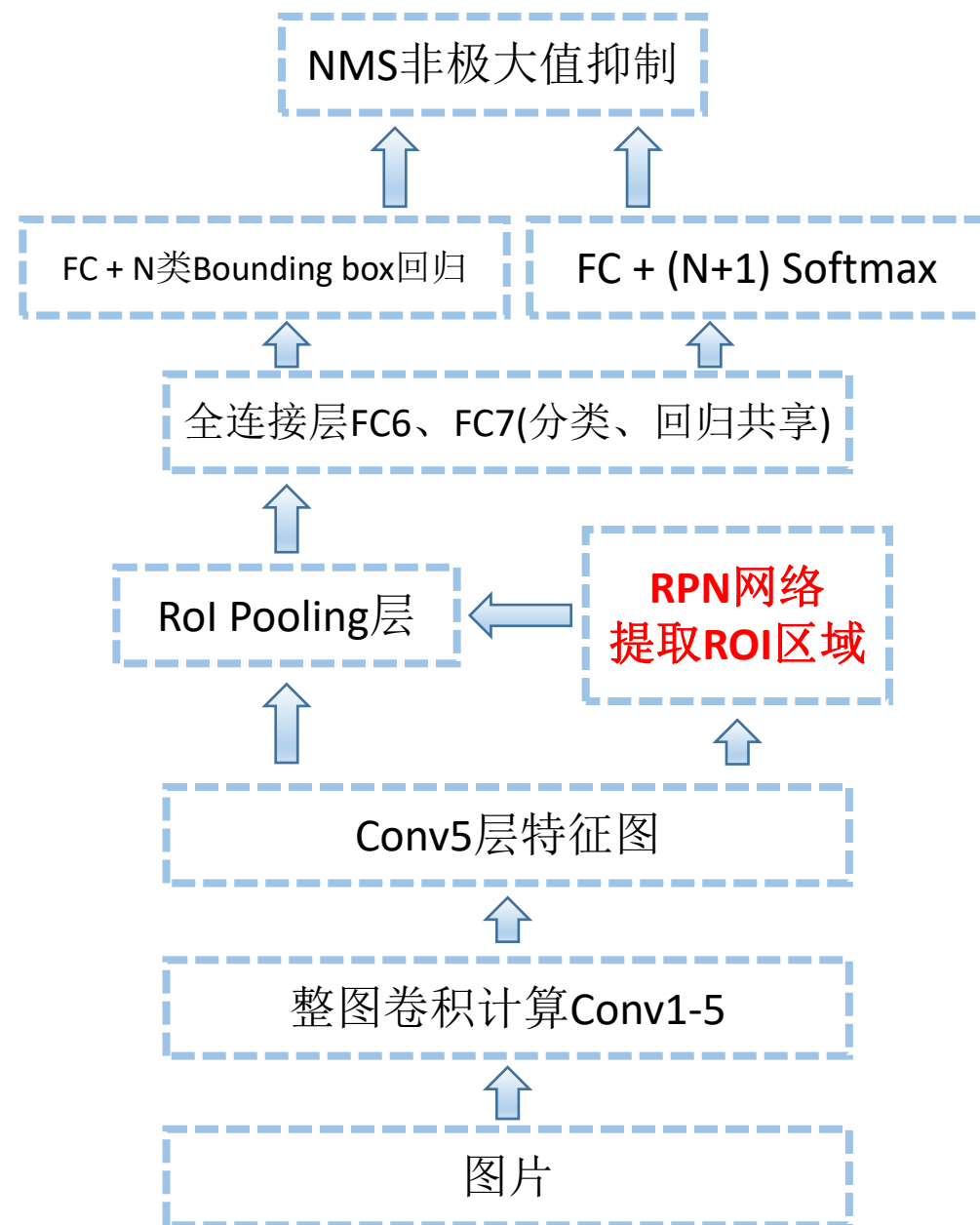
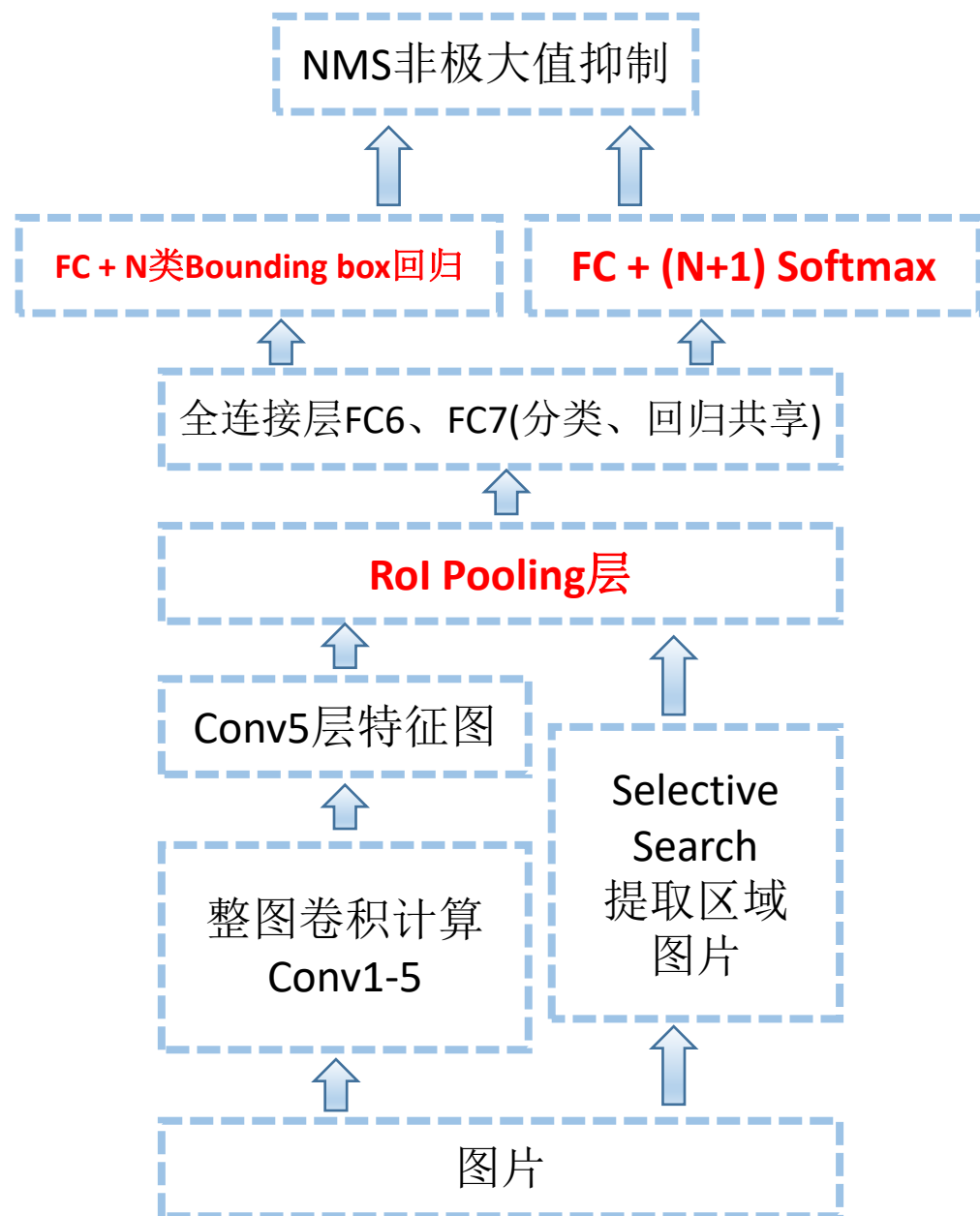
R-CNN vs SPP Net



SPP Net vs Fast R-CNN

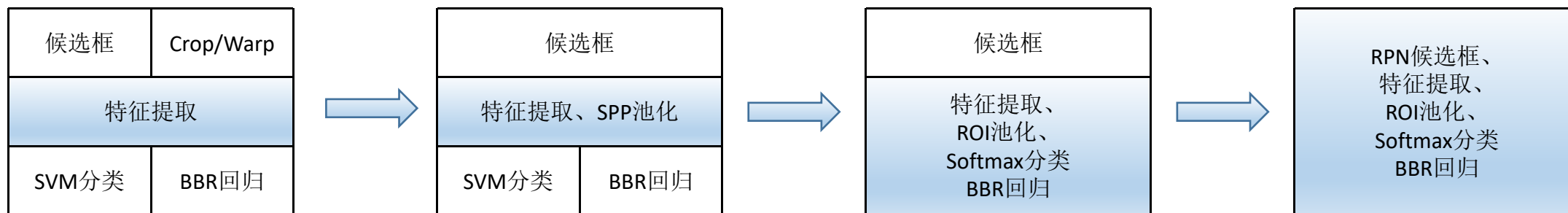


Fast R-CNN vs Faster R-CNN



R-CNN、SPP Net、Fast R-CNN、Faster R-CNN

⌚ 因为Faster-RCNN，这种基于CNN的real-time 的目标检测方法看到了希望，在这个方向上有了进一步的研究思路(one-stage发展方向)。至此，我们来看一下R-CNN网络的演进，如下图所示：



THANKS!